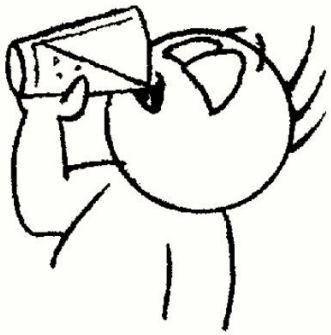
《编译原理》

第12讲b：语法分析–LR(k)

何冬杰
重庆大学



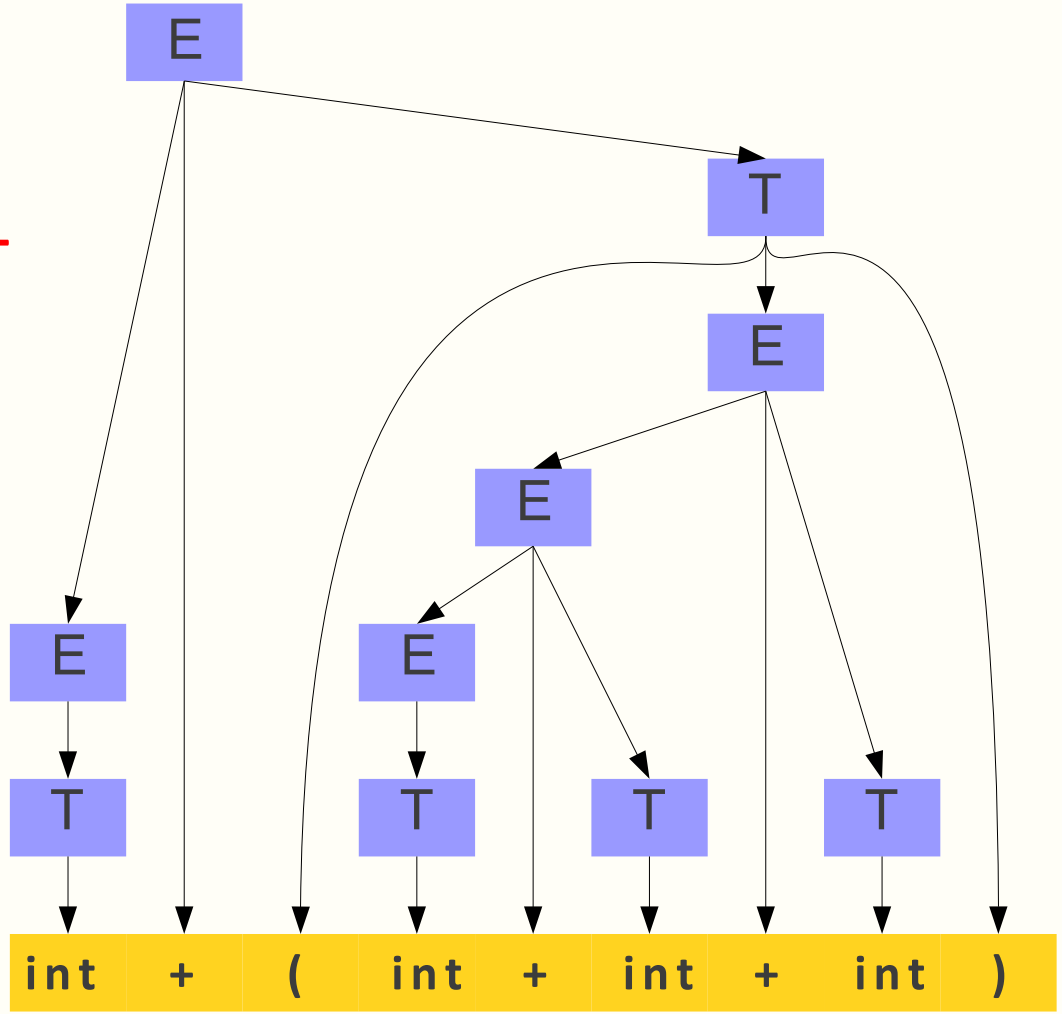
Bottoms Up!

What is Bottom-Up Parsing?

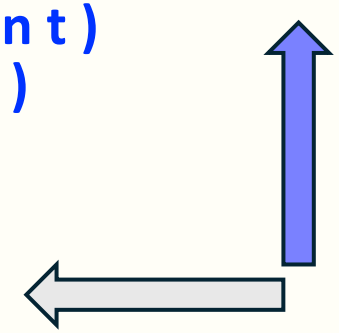
- **Idea**: Apply productions **in reverse** to convert the user's program to the start symbol.
- As with top-down, it could be done with a DFS or BFS, though this is rarely done in practice.
- We'll be exploring four **directional**, **predictive** bottom-up parsing techniques:
  - **Directional**: Scan the input from left-to-right.
  - **Predictive**: Guess which production should be inverted.

Views of a Bottom-Up Parse

$E \rightarrow T$
$E \rightarrow E + T$
$T \rightarrow int$
$T \rightarrow (E)$



$$
\begin{aligned}
& int + (int + int + int) \\
\Rightarrow\ & T + (int + int + int) \\
\Rightarrow\ & E + (int + int + int) \\
\Rightarrow\ & E + (T + int + int) \\
\Rightarrow\ & E + (E + int + int) \\
\Rightarrow\ & E + (E + T + int) \\
\Rightarrow\ & E + (E + int) \\
\Rightarrow\ & E + (E + T) \\
\Rightarrow\ & E + (E) \\
\Rightarrow\ & E + T \\
\Rightarrow\ & E
\end{aligned}
$$



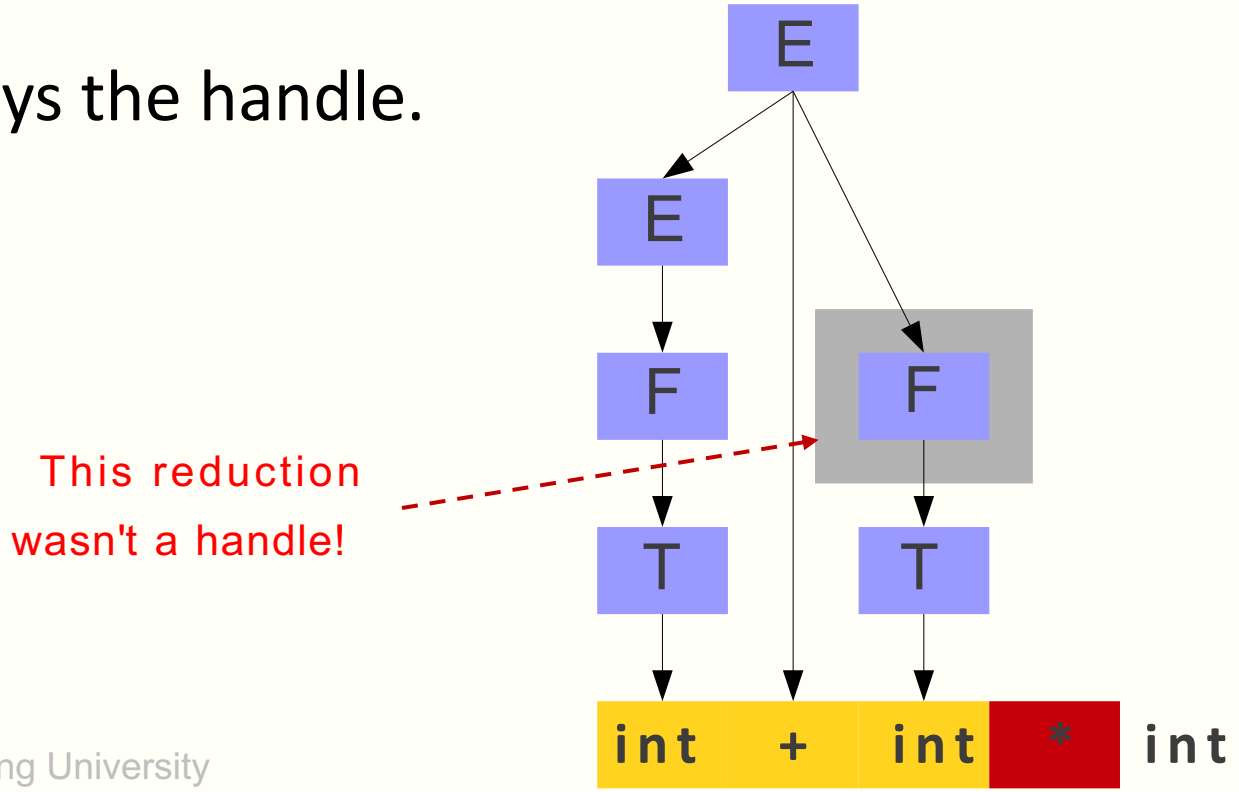
Bottom-Up Parsing **Reverses** a Rightmost Derivation.

Each step in this bottom-up parse is called a reduction. We reduce a substring of the sentential form back to a nonterminal.

Handles

- The **handle** of a parse tree T:
  - is the leftmost complete cluster of leaf nodes.
  - A left-to-right, bottom-up parse works by iteratively *searching* for a handle, then *reducing* the handle.
- The leftmost reduction isn't always the handle.

$E \rightarrow F$
$E \rightarrow E + F$
$F \rightarrow F * T$
$F \rightarrow T$
$T \rightarrow int$
$T \rightarrow (E)$



Finding Handles

- Where do we look for handles?
  - Where in the string might the handle be?
- How do we search for possible handles?
  - Once we know where to search, how do we identify candidate handles?
- How do we recognize handles?
  - Once we've found a candidate handle, how do we check that it really is the handle?

Q1: Where are Handles?

- Recall: A left-to-right, bottom-up parse traces a *rightmost* derivation in reverse.
- Each time we do a reduction, we are reversing a production applied to the *rightmost* nonterminal symbol.
- Suppose that our current sentential form is $\boldsymbol{\alpha\gamma\omega}$, where $\boldsymbol{\gamma}$ is the handle and $\mathbf{A} \rightarrow \boldsymbol{\gamma}$ is a production rule.
- After reducing $\boldsymbol{\gamma}$ back to $\mathbf{A}$, we have the string $\boldsymbol{\alpha}\mathbf{A}\boldsymbol{\omega}$.
- Thus $\boldsymbol{\omega}$ must consist purely of terminals, since otherwise the reduction we just did was not for the rightmost terminal.

Why This Matters

- Suppose we want to parse the string γ.
- We will break γ into two parts, α and ω, where
  - α consists of both terminals and non-terminals, and
  - ω consists purely of terminals.
- Our search for handles will concentrate purely in α.
- As necessary, we will start moving terminals from ω over into α.

Shift/Reduce Parsing

- The bottom-up parsers we consider are called **shift/reduce** parsers.
  - Contrast with the LL(1) **predict/match** parser.
- **Idea**: **Split** the input into *two parts*:
  - The *left* substring is our work area; all handles must be here.
  - The *right* substring is input we have not yet processed; it consists purely of terminals.
- At each point, decide whether to:
  - Move a terminal across the split (**shift**)
  - Reduce a handle (**reduce**)

A Sample Shift/Reduce Parse

E → F

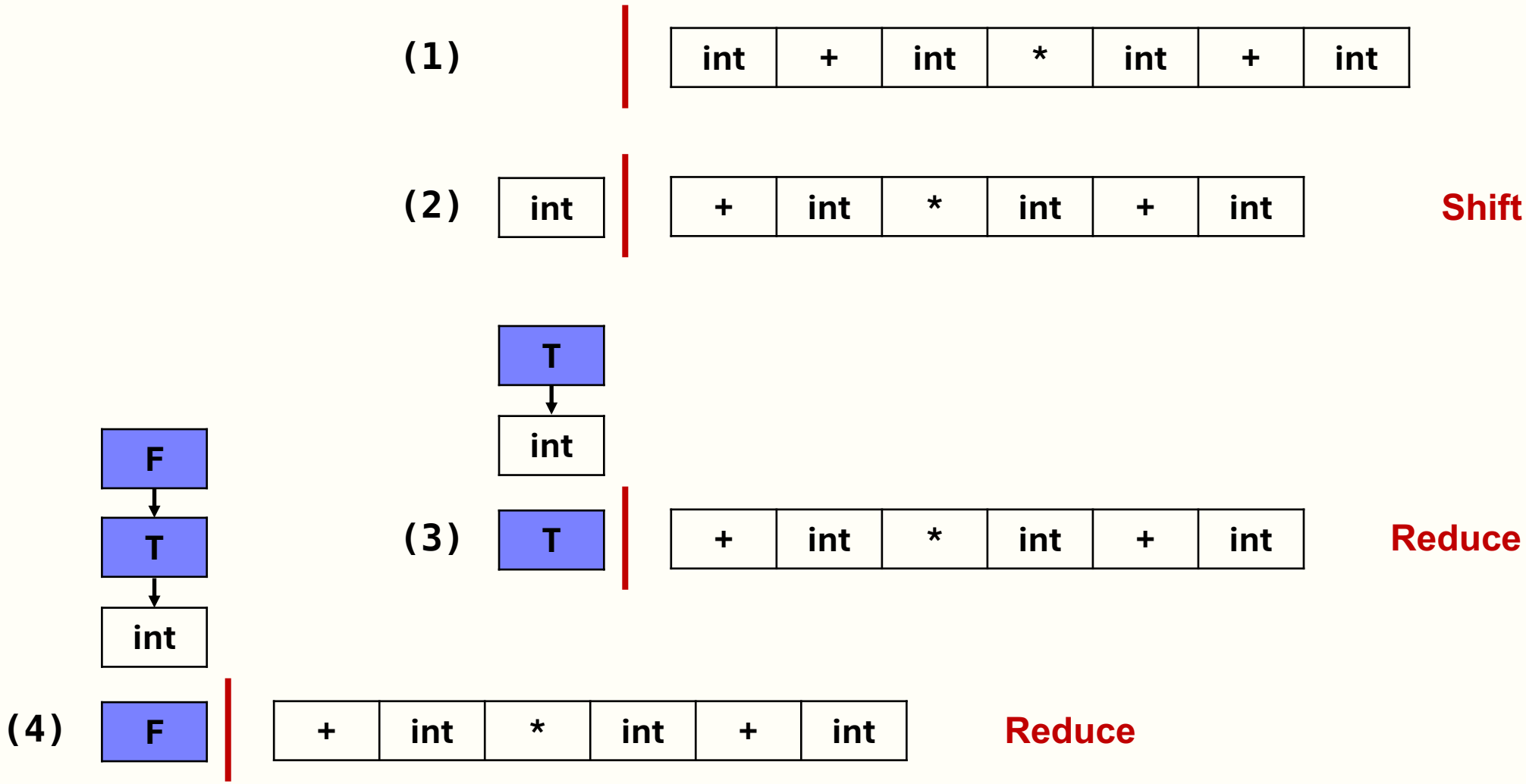
E → E + F

F → F * T

F → T

T → int

T → (E)



A Sample Shift/Reduce Parse

E → F

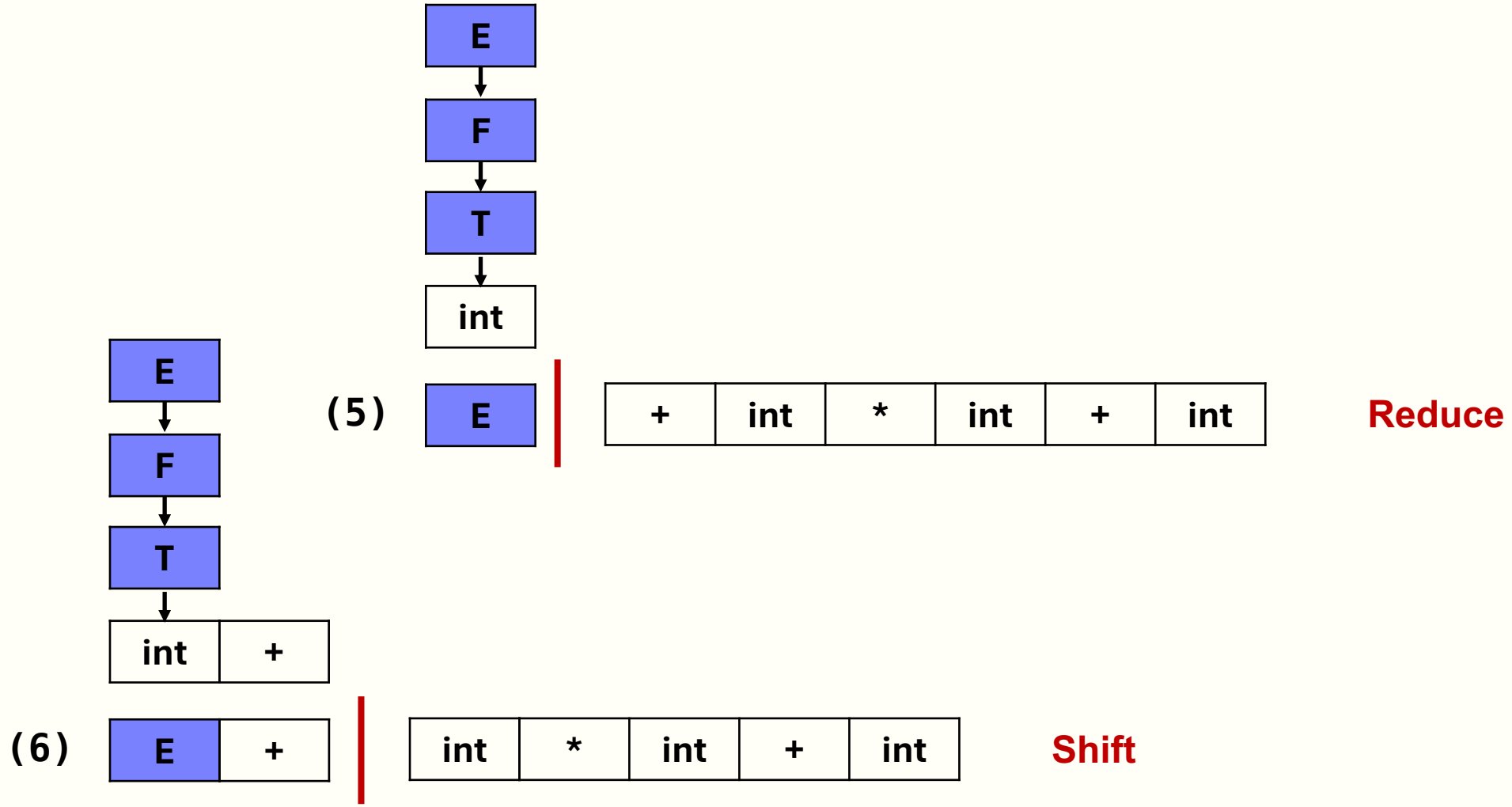
E → E + F

F → F * T

F → T

T → int

T → (E)



A Sample Shift/Reduce Parse

E → F

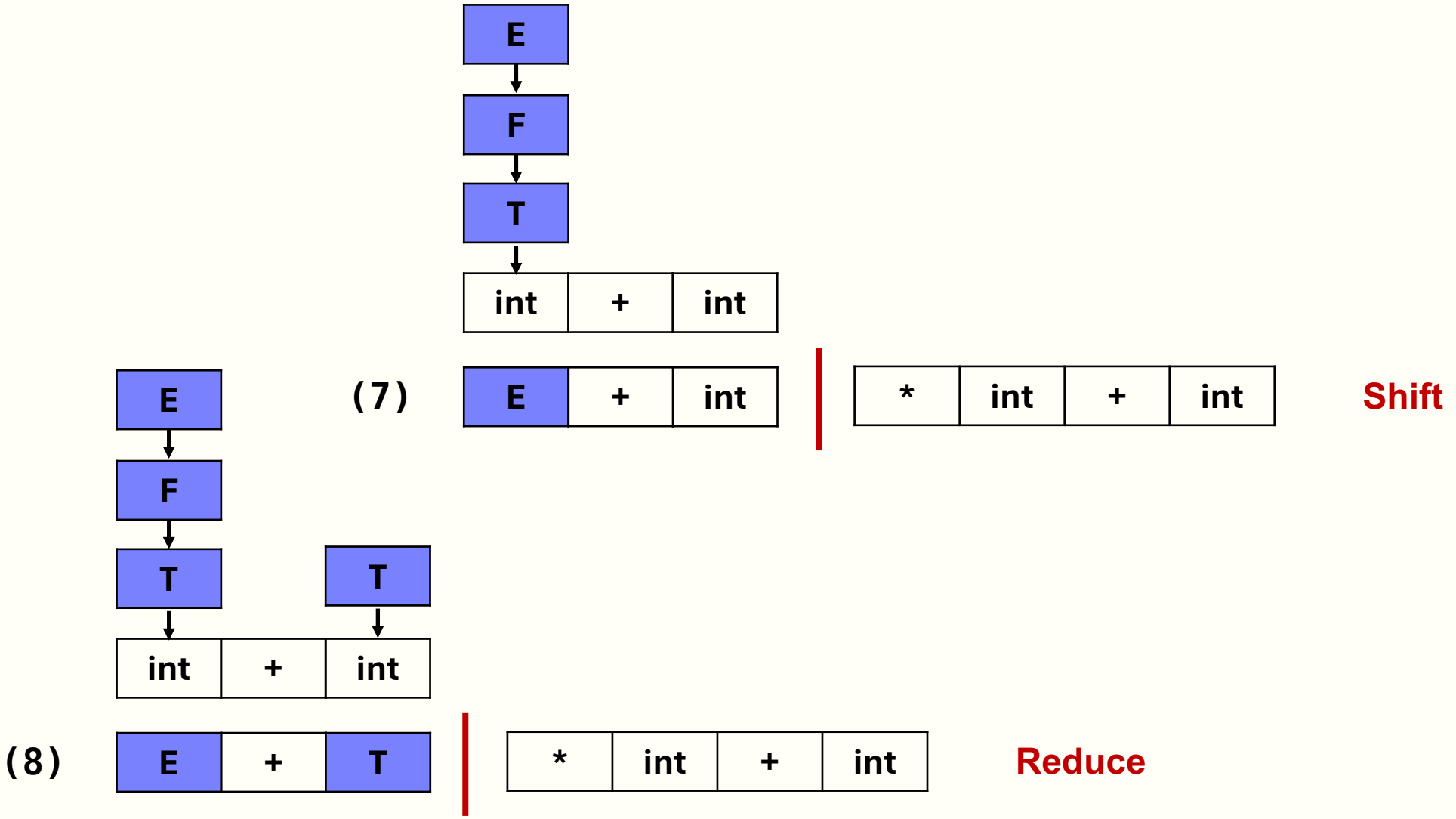
E → E + F

F → F * T

F → T

T → int

T → (E)



A Sample Shift/Reduce Parse

E → F

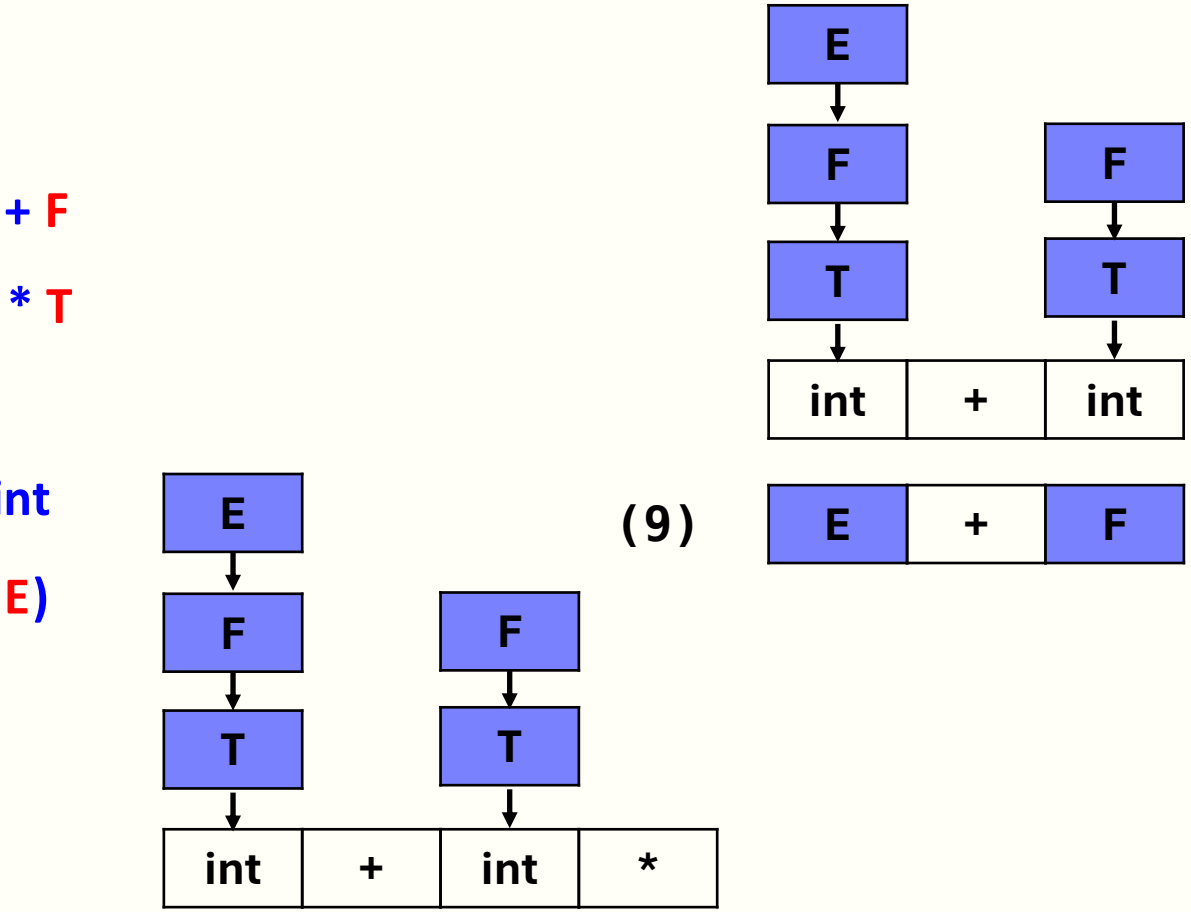
E → E + F

F → F * T

F → T

T → int

T → (E)



Reduce

Shift

A Sample Shift/Reduce Parse

E → F

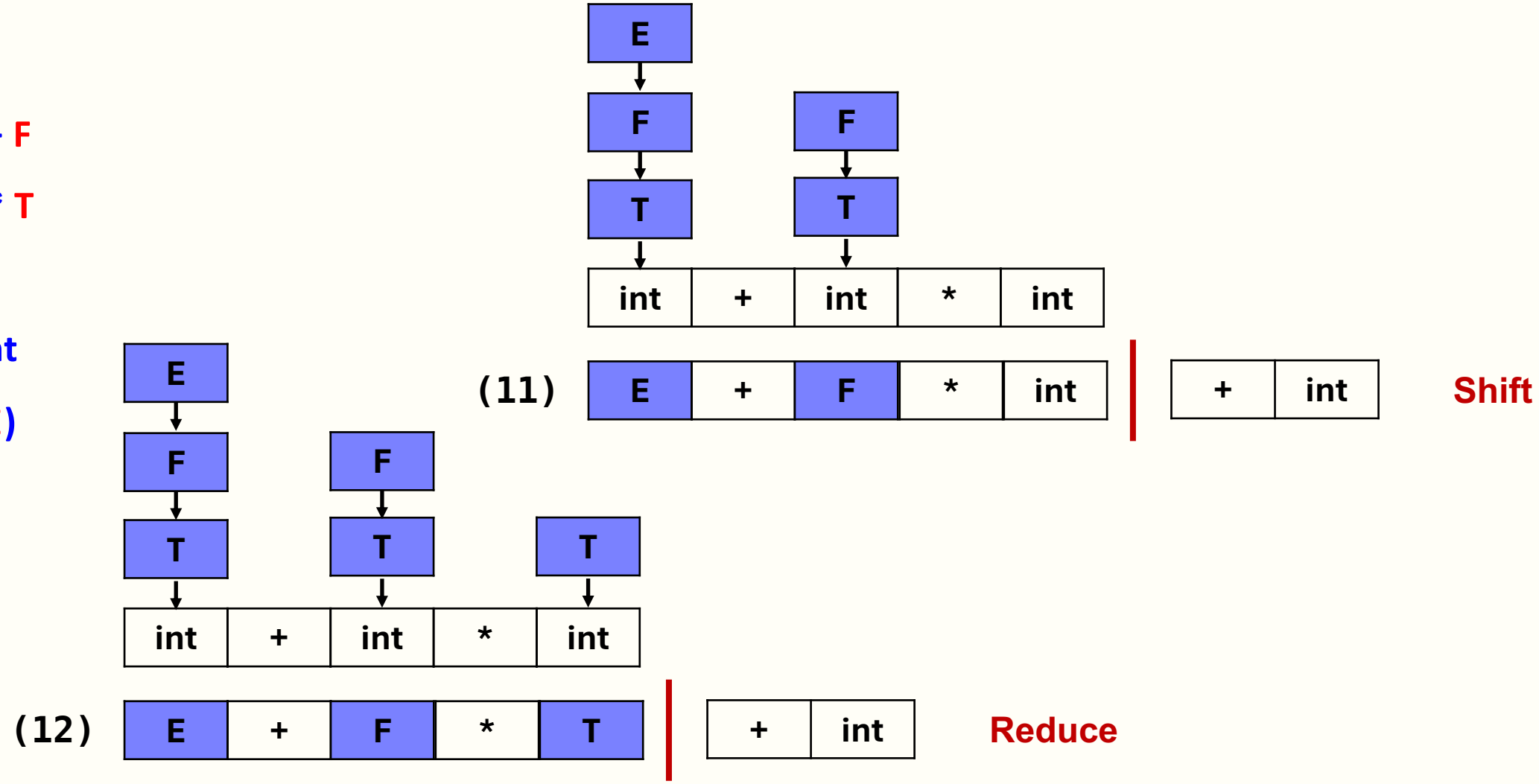
E → E + F

F → F * T

F → T

T → int

T → (E)



A Sample Shift/Reduce Parse

E → F

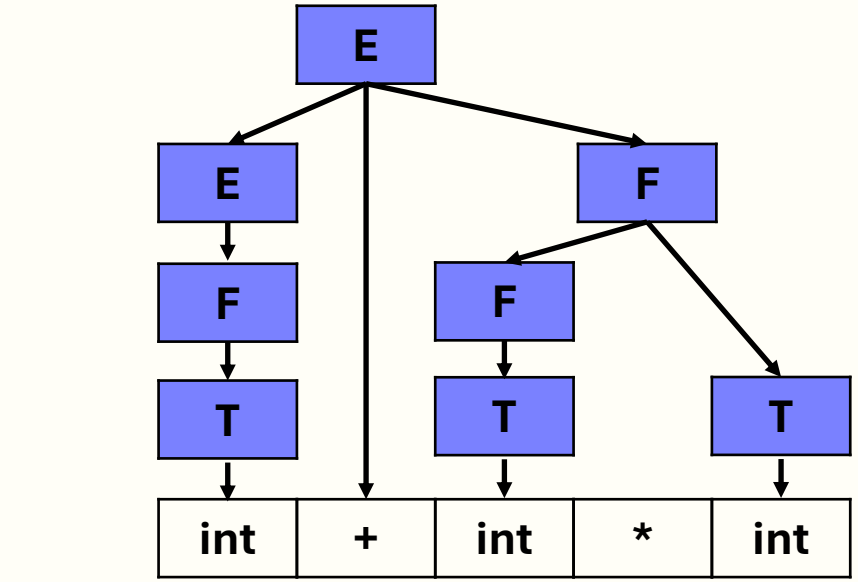
E → E + F

F → F * T

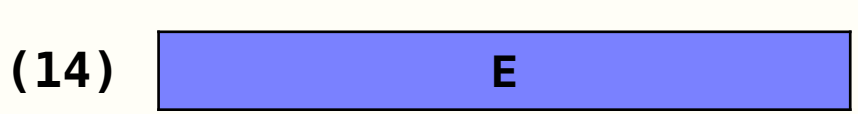
F → T

T → int

T → (E)

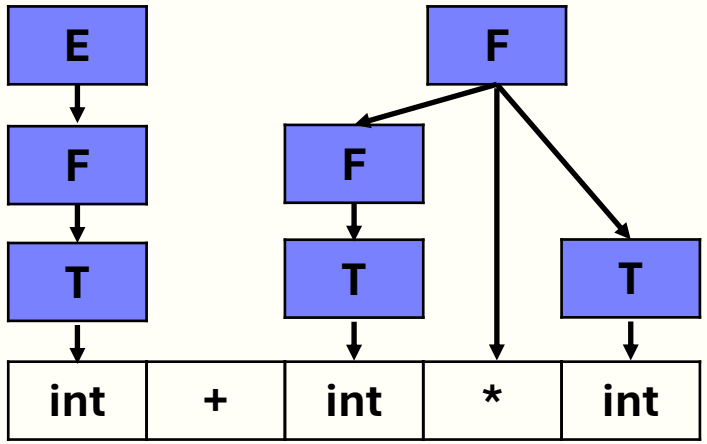


(14)



Reduce

+	int



(13)

E	+	F

Reduce

+	int

A Sample Shift/Reduce Parse

E → F

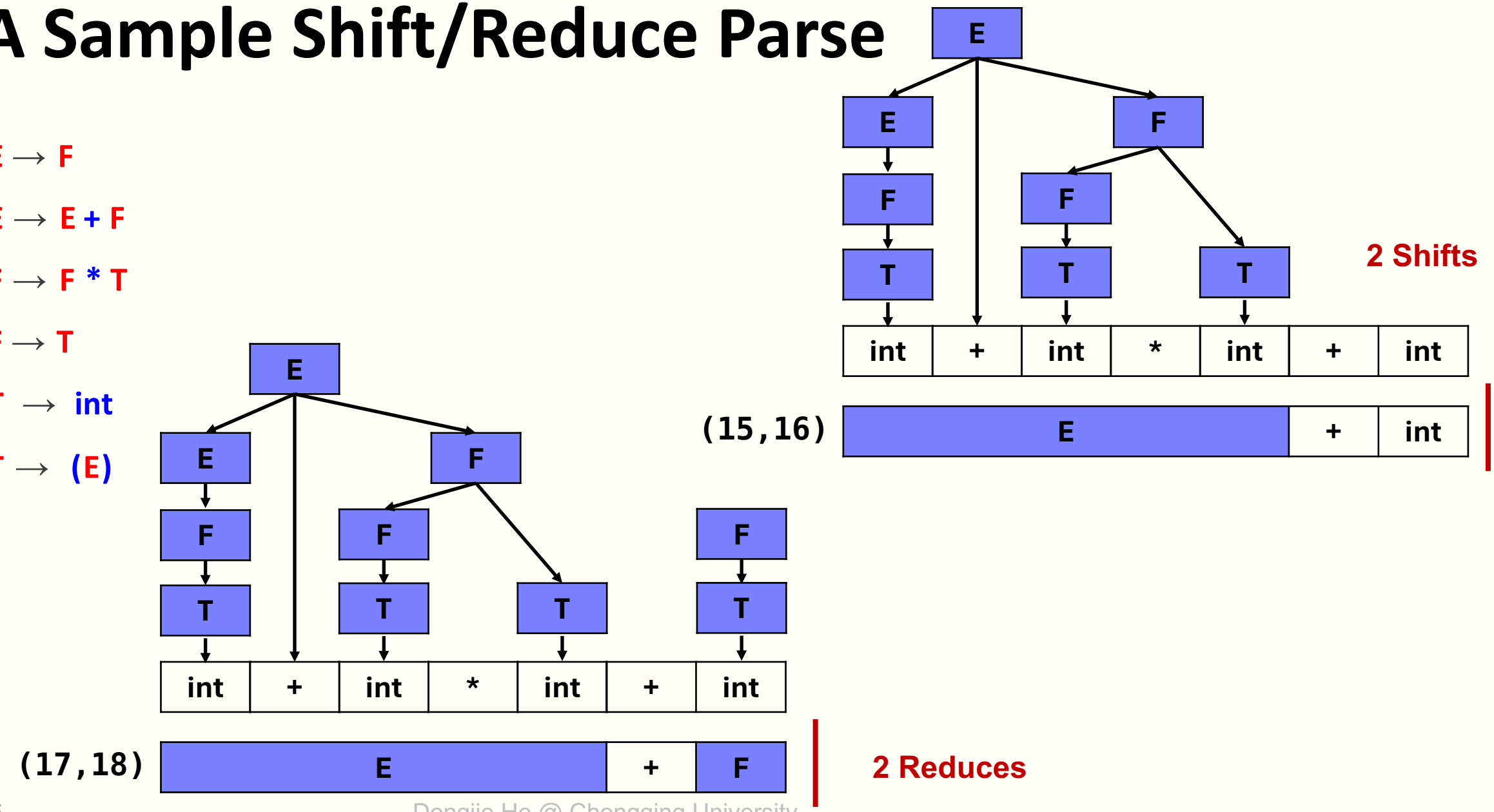
E → E + F

F → F * T

F → T

T → int

T → (E)



A Sample Shift/Reduce Parse

E → F

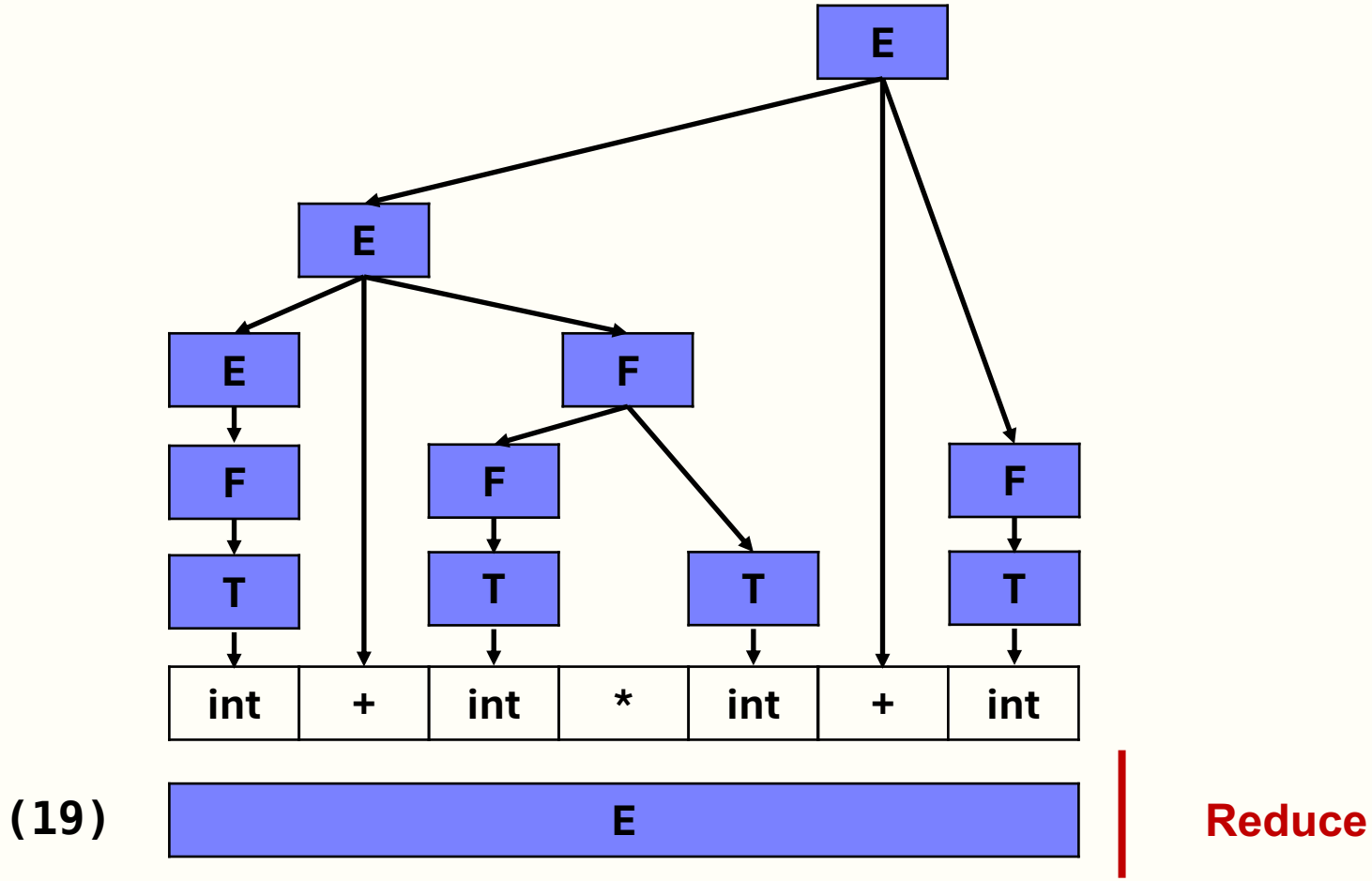
E → E + F

F → F * T

F → T

T → int

T → (E)



An Important Observation and Its Corollary

- **Observation**: all reductions occur at the right end of the left area.
- Inductive proof sketch:
  - **Base**: the first handle is reduced at the right end of the left area.
  - **Inductive Step**: After each reduction, subsequent shifts extend the left area until the next handle reaches its right end, where it can be reduced.

- **Corollary**: we never need to shift from the left to the right.
  - No need to “uncover” something to do a reduction.
- Consequently, shift/reduce parsing means
  - **Shift**: Move a terminal from the right to the left area.
  - **Reduce**: Replace some number of symbols at the right side of the left area.

Simplifying our Terminology

- All activity in a shift/reduce parser is at the far right end of the left area.

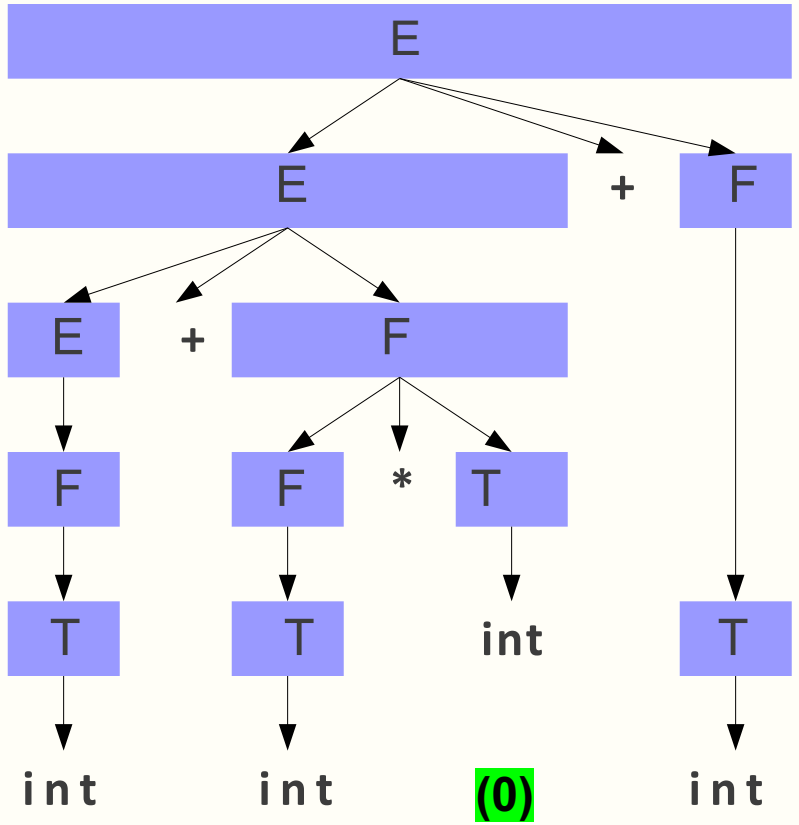
- **Idea**: Represent the left area as a **stack**.
- **Shift**: Push the next terminal onto the stack.
- **Reduce**: Pop some number of symbols from the stack, then push the appropriate nonterminal.

Q2: How do we search for handles?

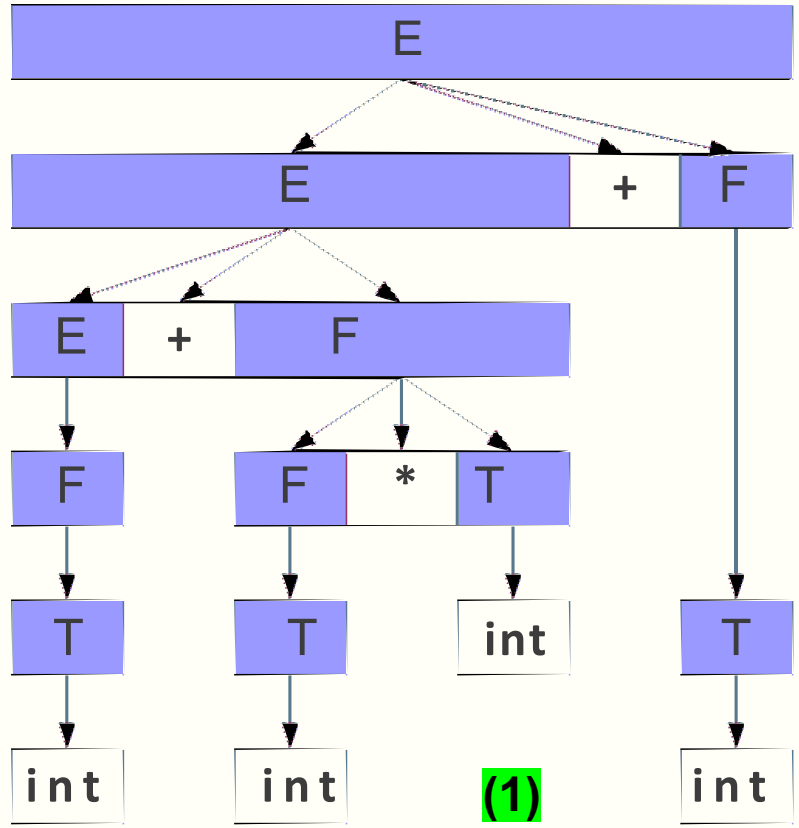
- When using a shift/reduce parser, we must decide whether to shift or reduce at each point.
- We only want to reduce when we know we have a handle.
- **Question**: How can we tell that we might be looking at a handle?
- **Answer**: Exploring the Left Side
  - The handle will always appear at the end of the string in the left side of the parser.
  - Can any string appear on the left side of the parser, or are there restrictions on what sorts of strings can appear there?
  - If we can find a pattern in the strings that can appear on the left side, we might be able to exploit it to detect handles.

Another Look at Handles

E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



Shift

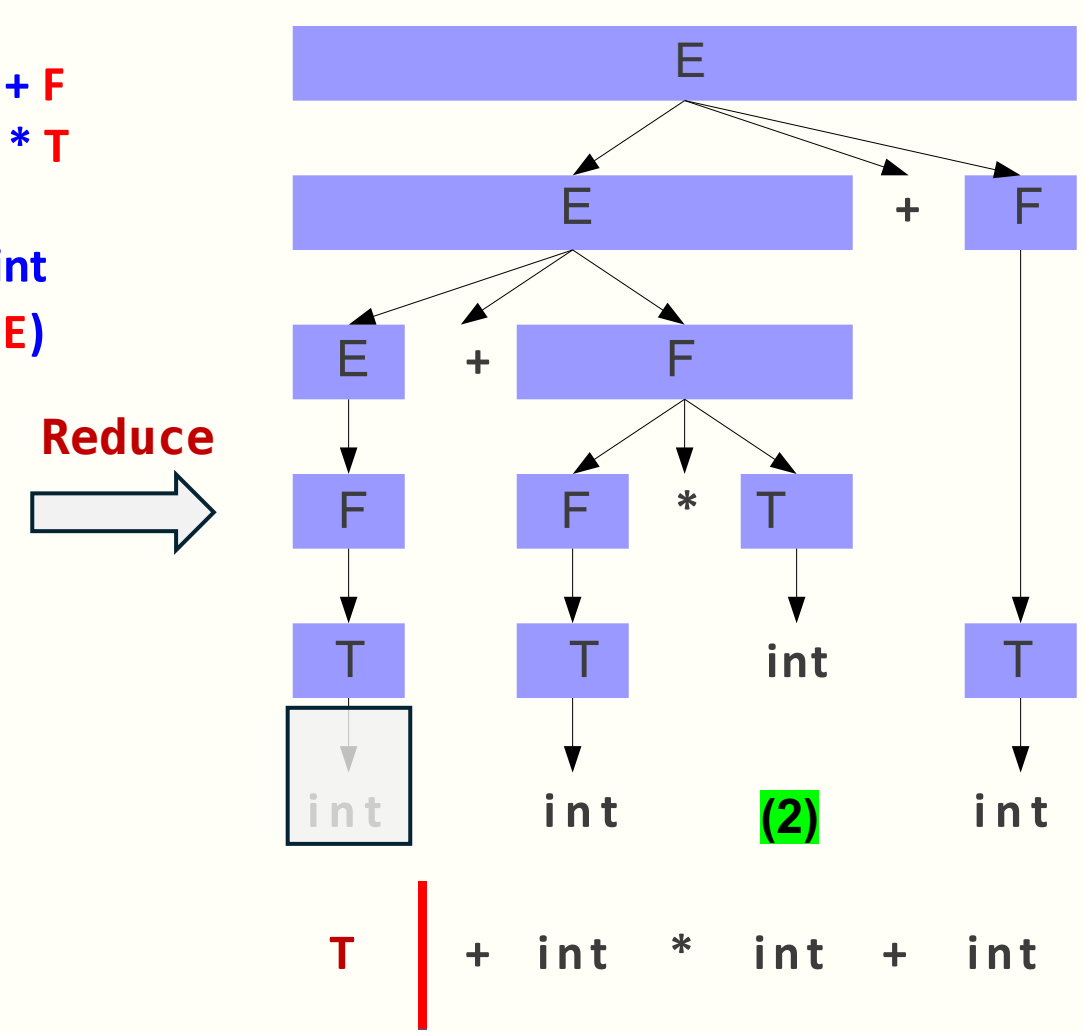


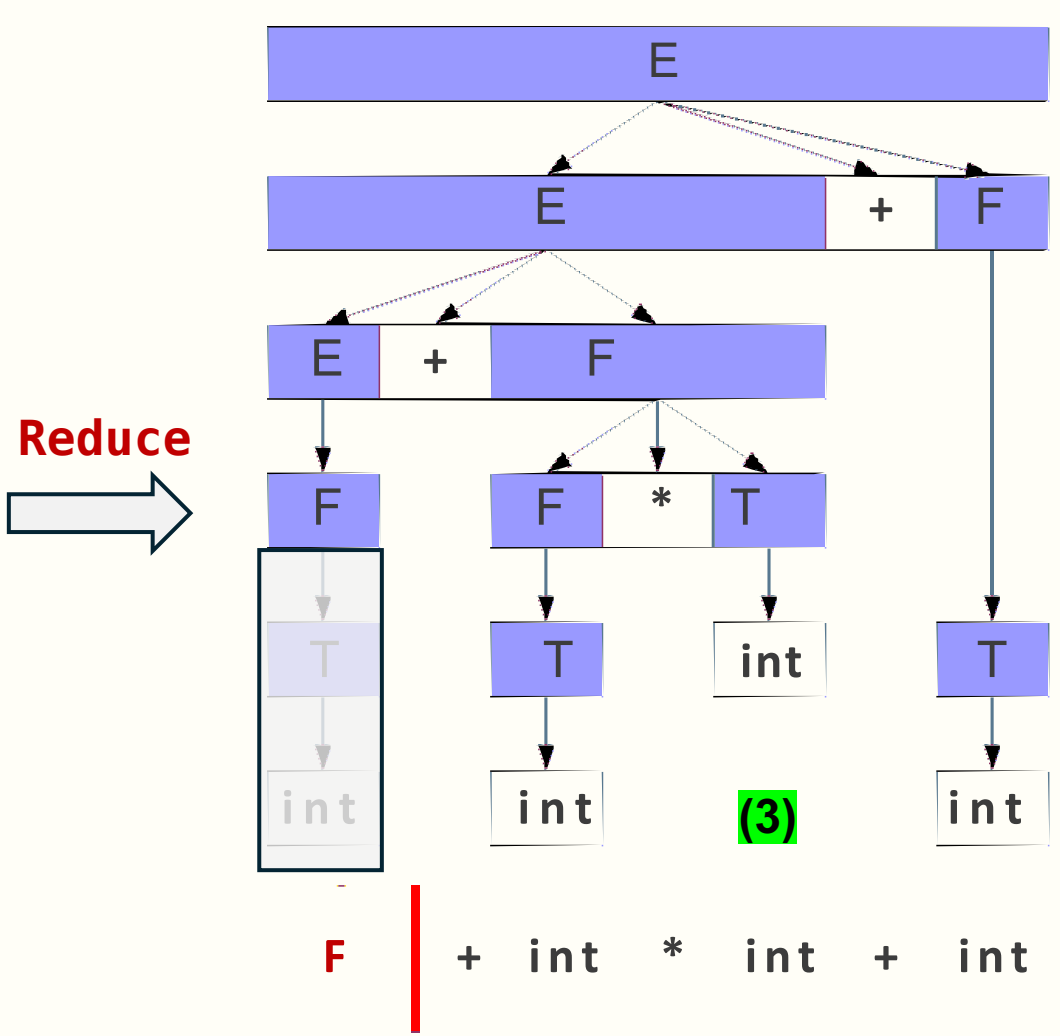
int + int * int + int

int + int * int + int

Another Look at Handles

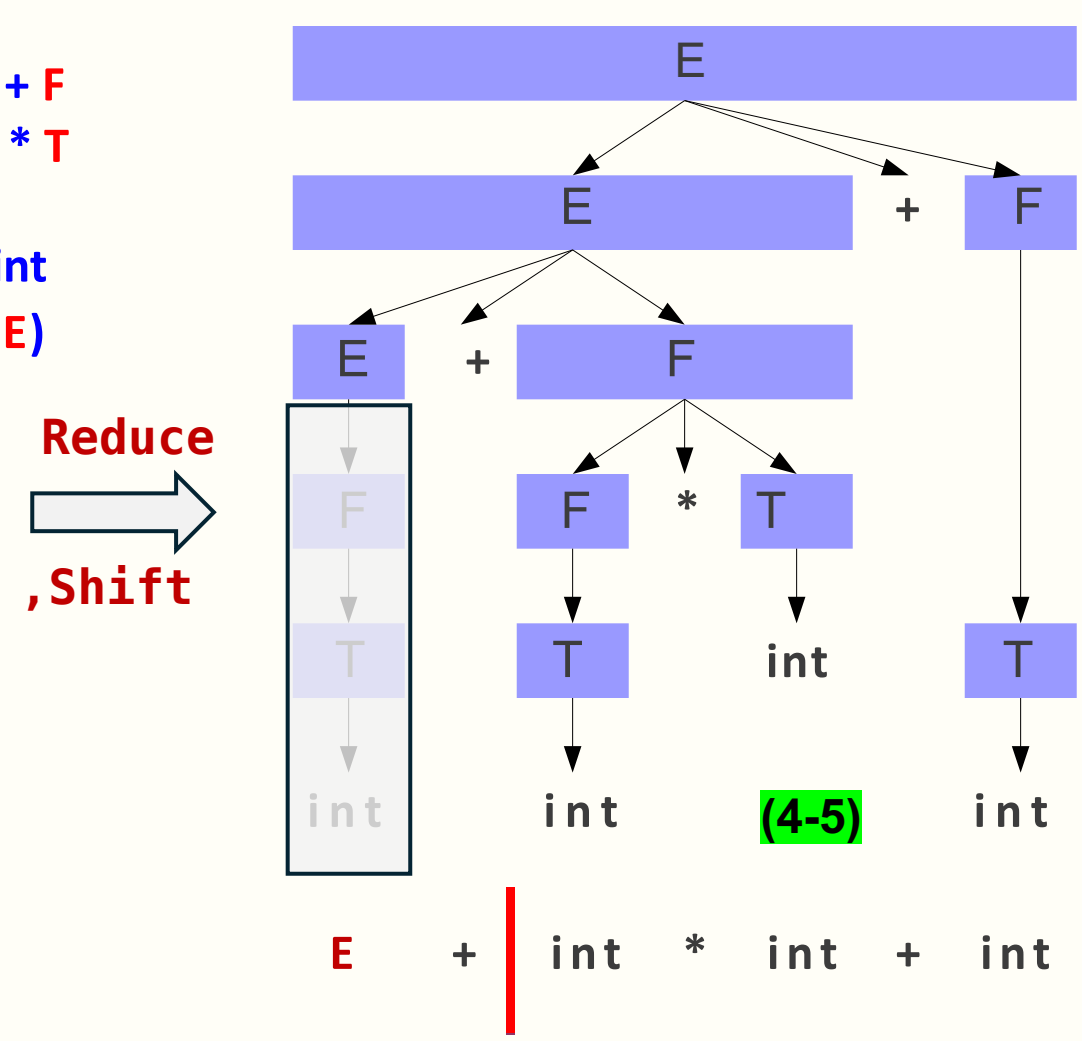
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

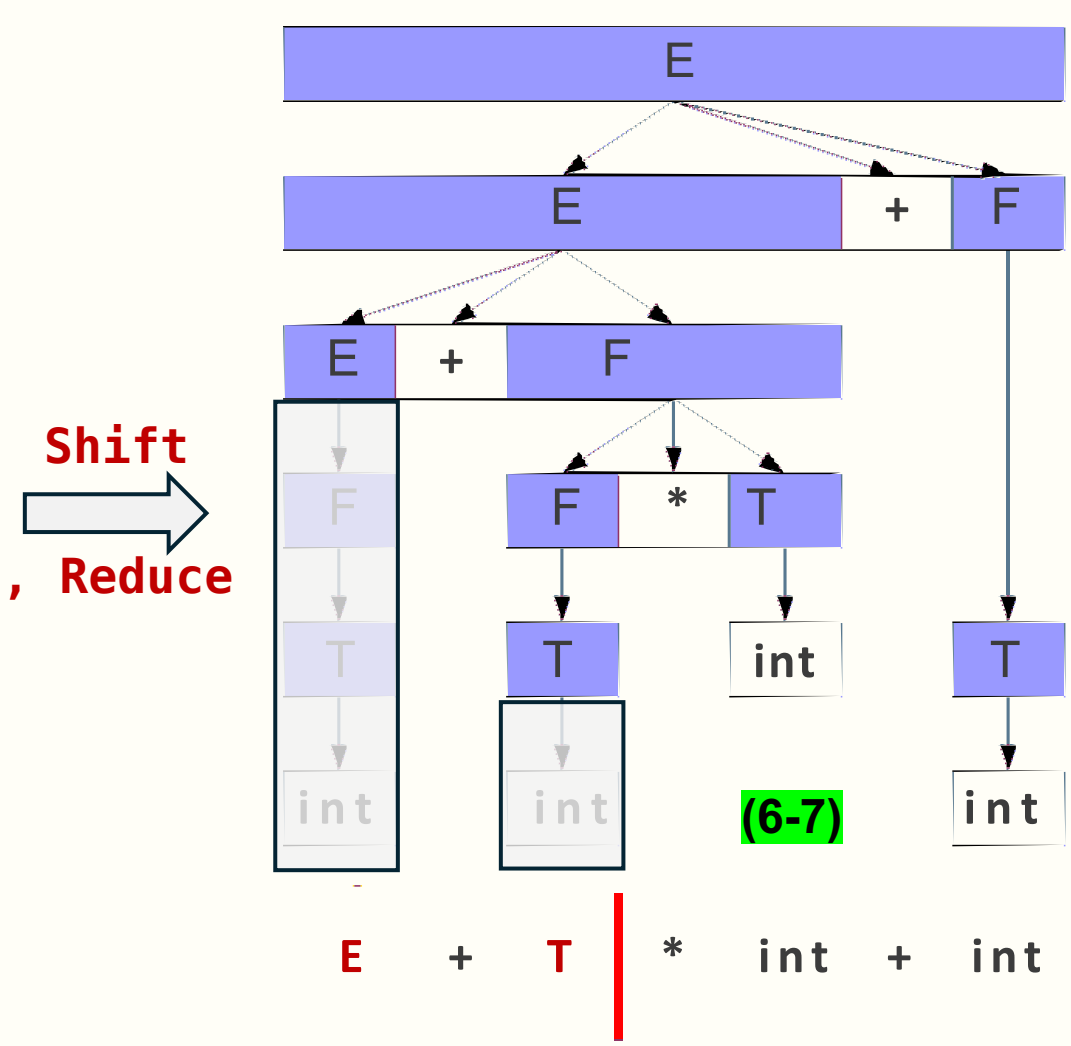




Another Look at Handles

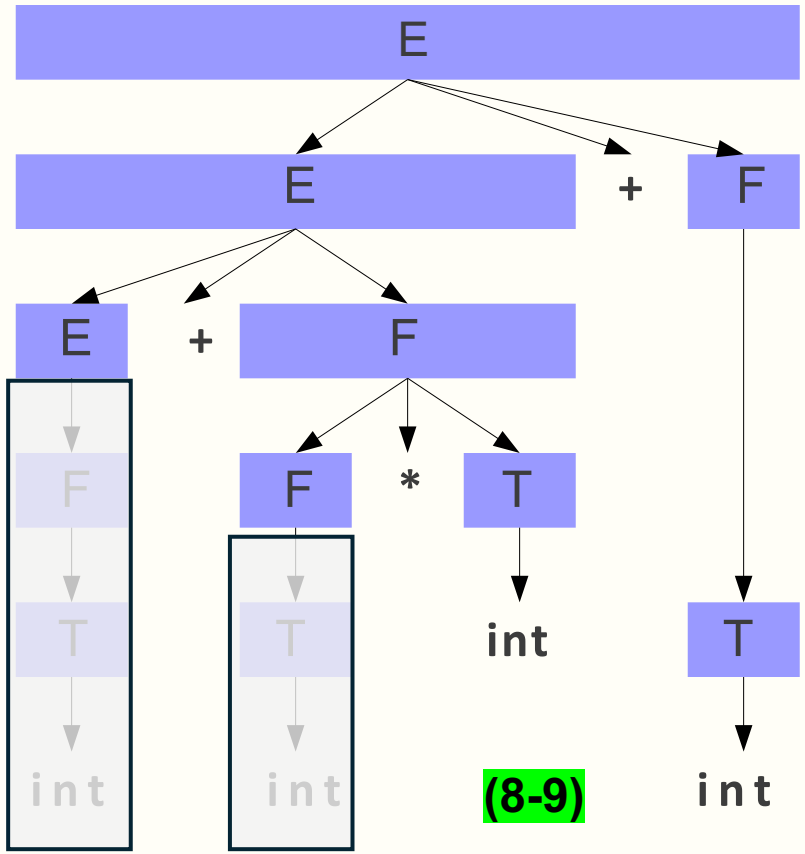
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

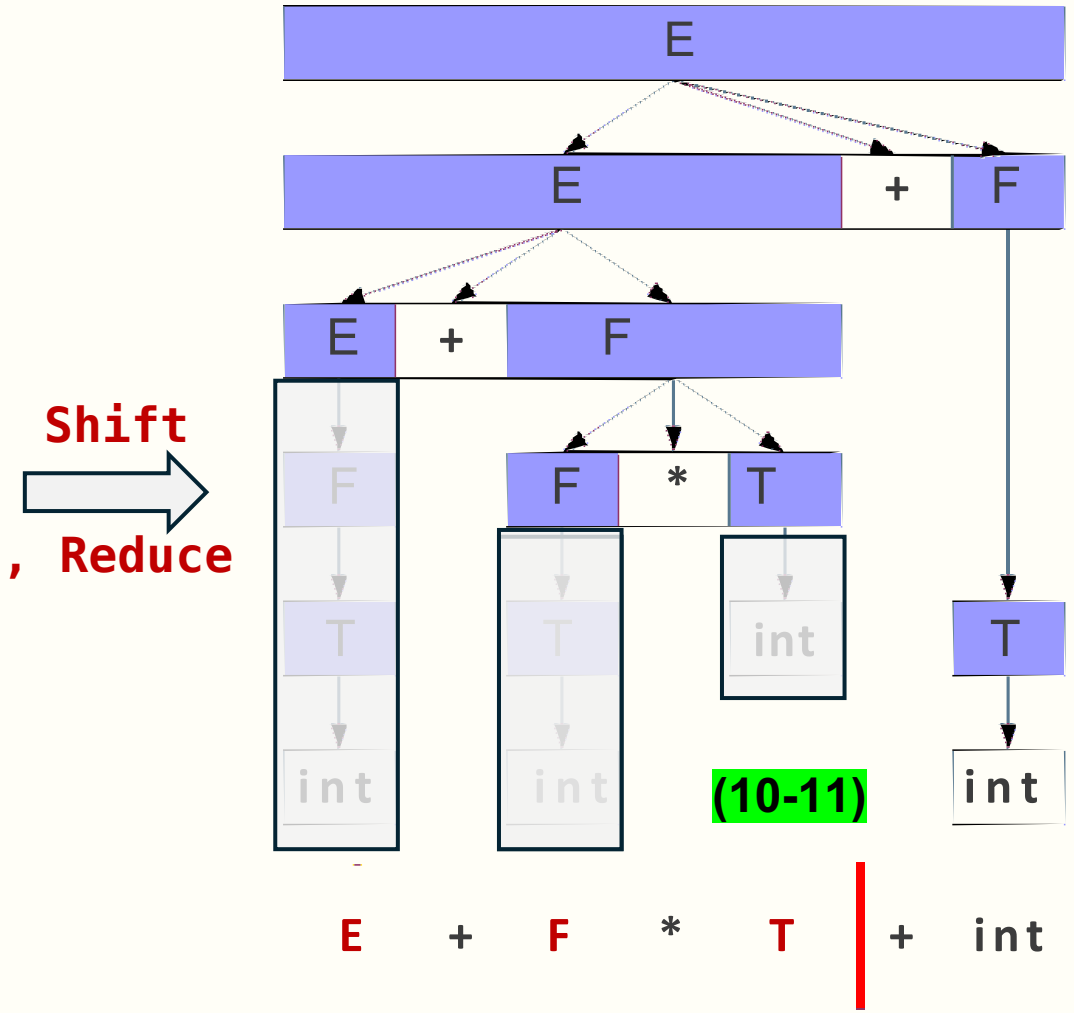




Another Look at Handles

E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



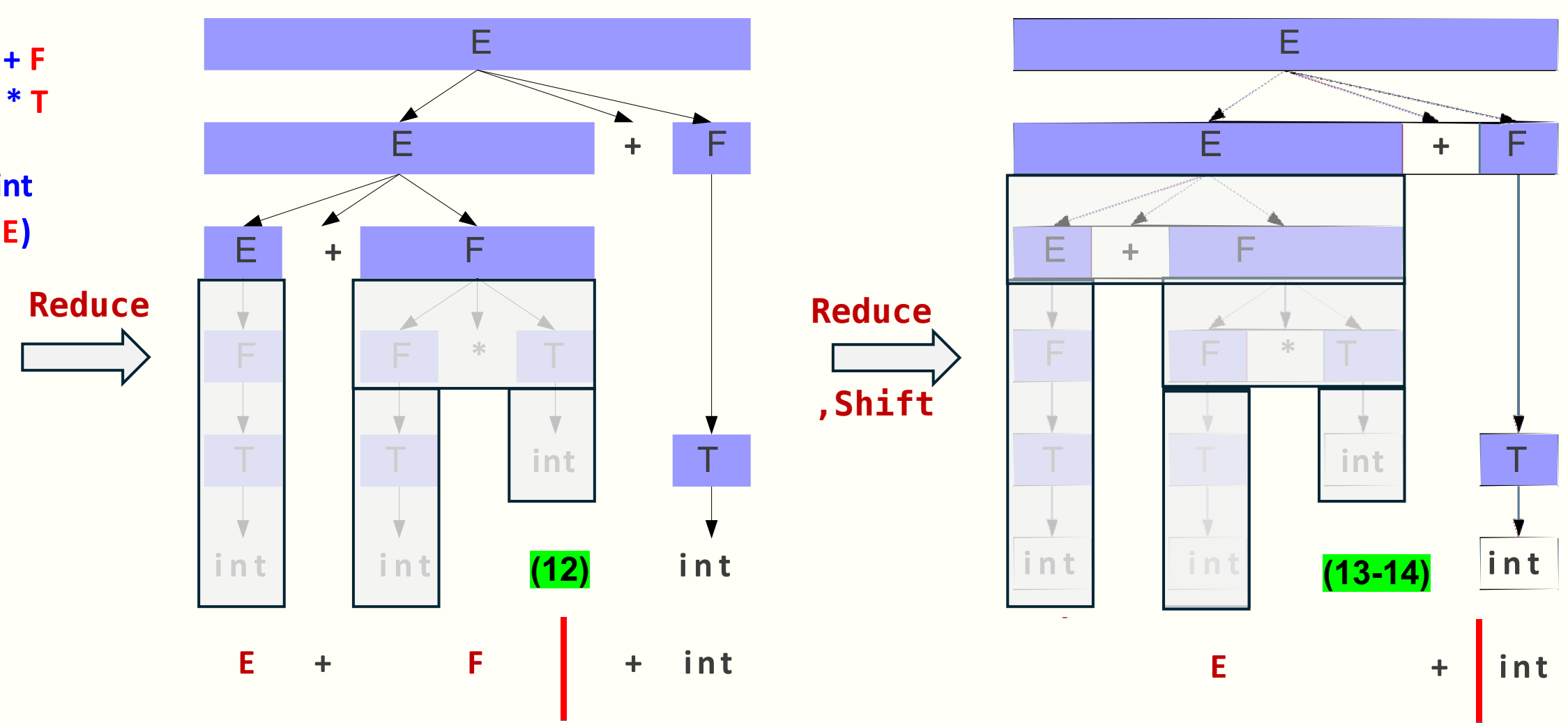


E + F * | int + int

E + F * T | + int

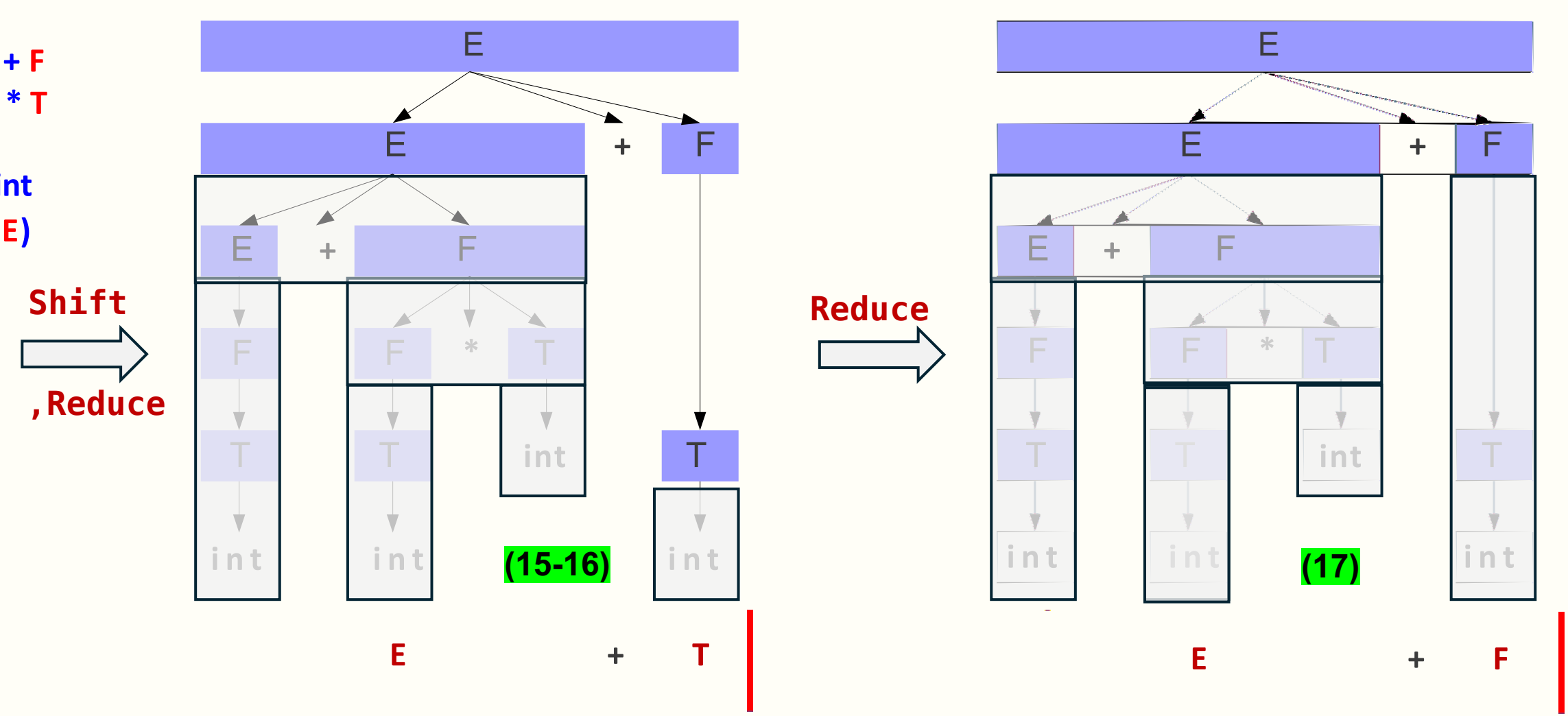
Another Look at Handles

E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



Another Look at Handles

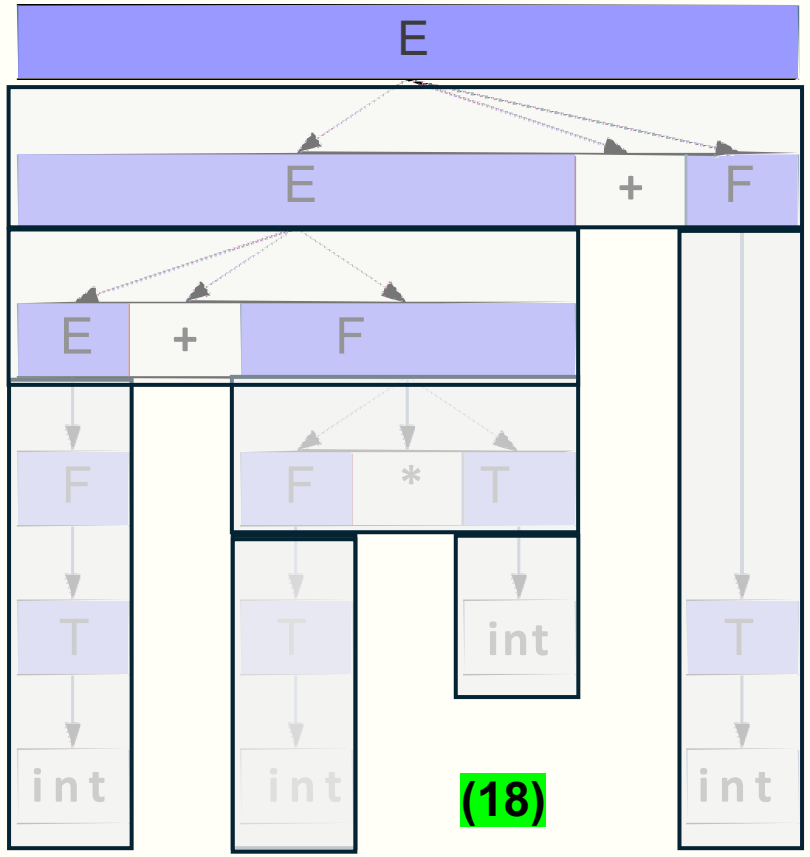
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



Another Look at Handles

E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

Reduce



E

Tracking Our Position

S → E
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

(1)

S → · E

Try to derive



(2)

S → · E
E → · E + F
E → · E + F
E → · F
F → · T
T → · int

int + int * int + int

int + int * int + int

Tracking Our Position

S → E
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

Shift

(3)

S → · E
E → · E + F
E → · E + F
E → · F
F → · T
T → int ·

Reduce

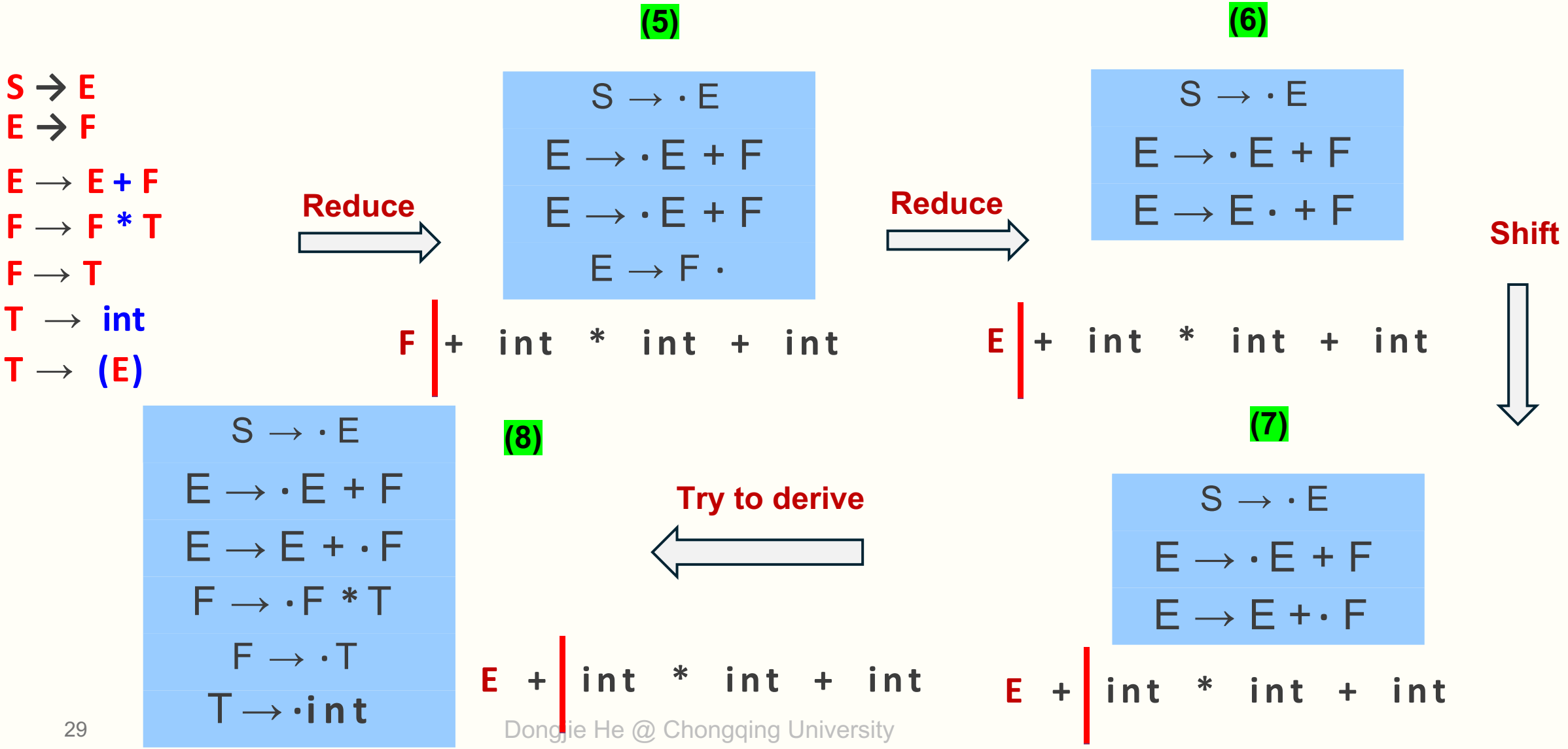
(4)

S → · E
E → · E + F
E → · E + F
E → · F
F → T ·

int | + int * int + int

T | + int * int + int

Tracking Our Position



Tracking Our Position

$S \rightarrow E$
$E \rightarrow F$
$E \rightarrow E + F$
$F \rightarrow F * T$
$F \rightarrow T$
$T \rightarrow int$
$T \rightarrow (E)$

Shift

(9)

$S \rightarrow \cdot E$
$E \rightarrow \cdot E + F$
$E \rightarrow E + \cdot F$
$F \rightarrow \cdot F * T$
$F \rightarrow \cdot T$
$T \rightarrow int \cdot$

E + int | * int + int

2 Reduces

(10)

$S \rightarrow \cdot E$
$E \rightarrow \cdot E + F$
$E \rightarrow E + \cdot F$
$F \rightarrow F \cdot * T$

E + F | * int + int

Shift

(11)

$S \rightarrow \cdot E$
$E \rightarrow \cdot E + F$
$E \rightarrow E + \cdot F$
$F \rightarrow F * \cdot T$

E + F * | int + int

Tracking Our Position

S → E
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

(12)

S → · E
E → · E + F
E → E + · F
F → F * · T

E + F * | int + int

Try to derive , Shift

(13, 14)

S → · E
E → · E + F
E → E + · F
F → F * · T
T → int ·

E + F * int | + int

Reduce

(15, 16)

S → · E
E → · E + F
E → E + F ·

E + F | + int

Reduce

(17)

S → · E
E → E · + F

E | + int

Shift

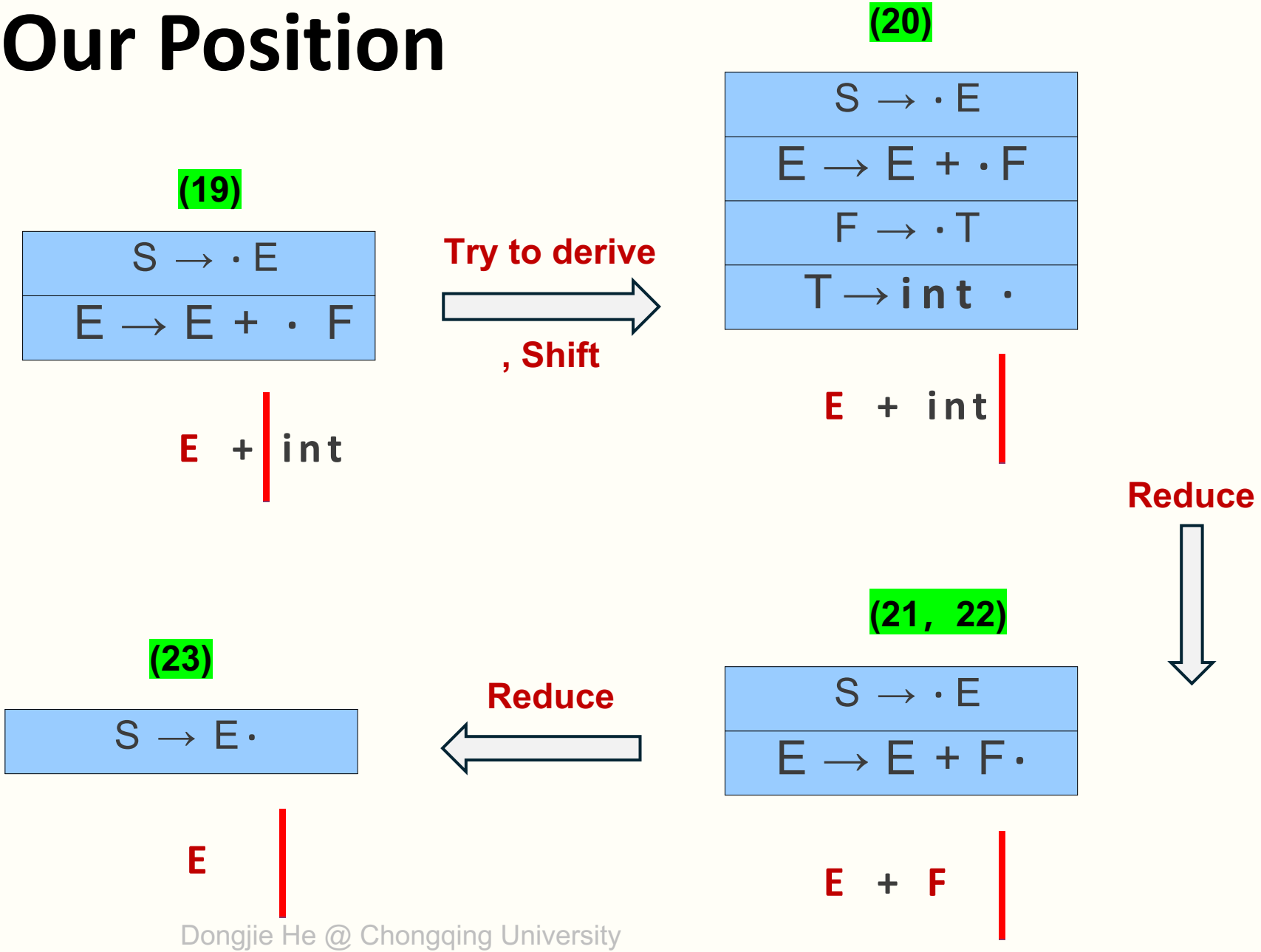
(18)

S → · E
E → E + · F

E + | int

Tracking Our Position

S → E
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

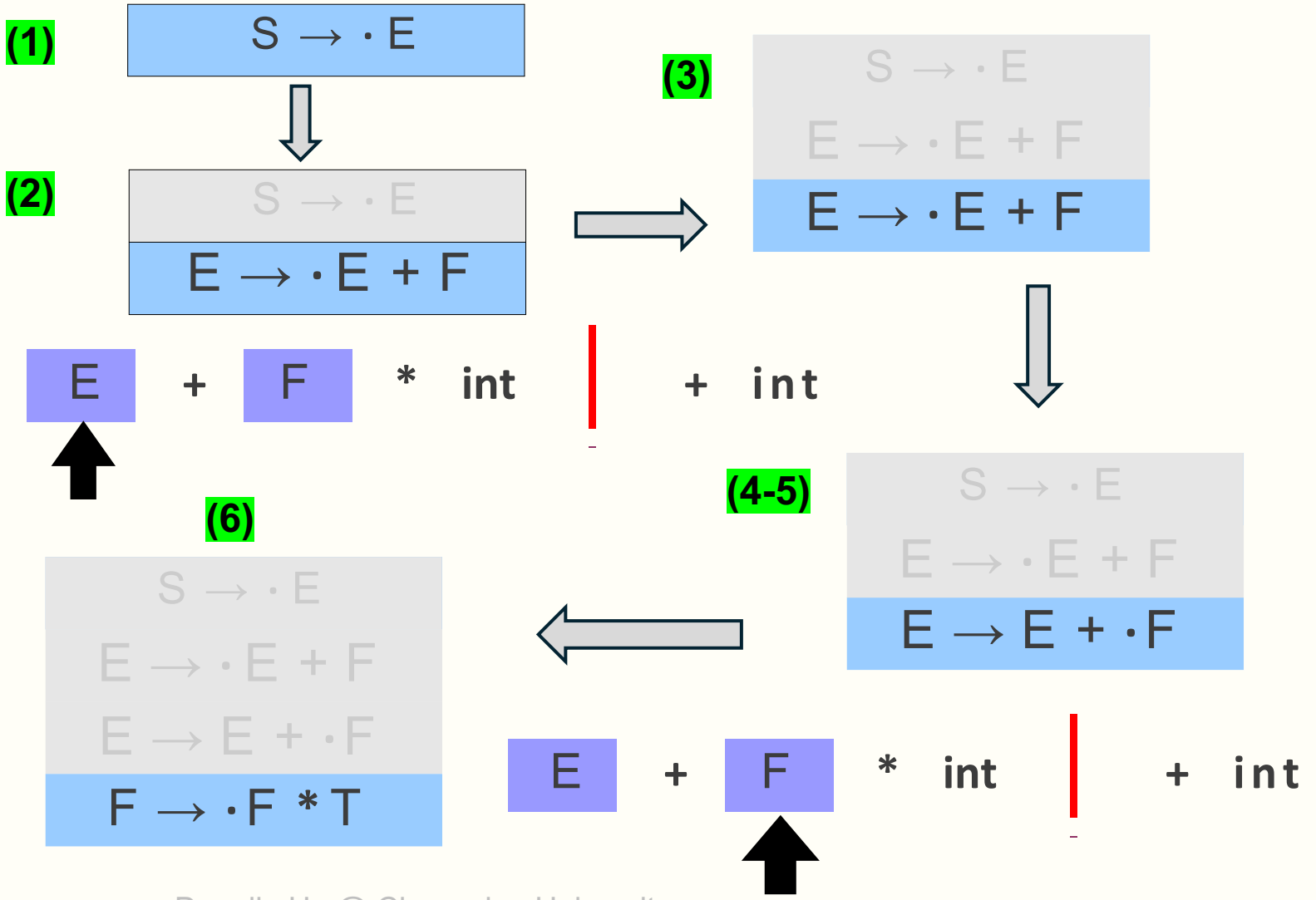


Generating and Recognizing Left-Hand Sides

- At any instant in time, the contents of the left side of the parser can be described using the following process:
  - Trace out, from the start symbol, the series of productions that have not yet been completed and where we are in each production.
  - For each production, in order, output all of the symbols up to the point where we change from one production to the next.
- Recognizing Left-Hand Sides
- **Idea**: At each point, track
  - Which production we are in, and
  - Where we are in that production.
- At each point, we can do one of two things:
  - Match the next symbol of the candidate left-hand side with the next symbol in the current production, or
  - If the next symbol of the candidate left-hand side is a nonterminal, nondeterministically guess which production to try next.

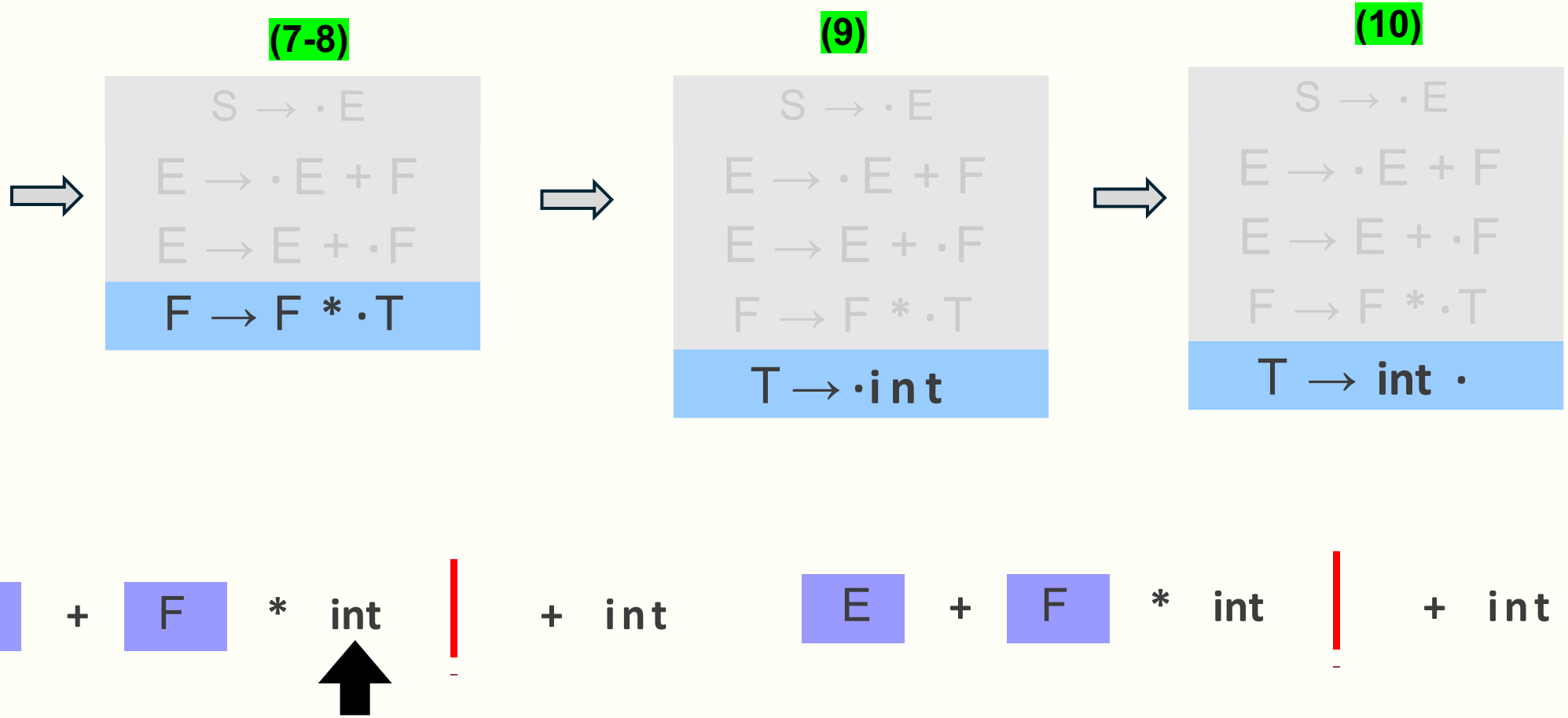
Recognizing Left-Hand Sides

S → E
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



Recognizing Left-Hand Sides

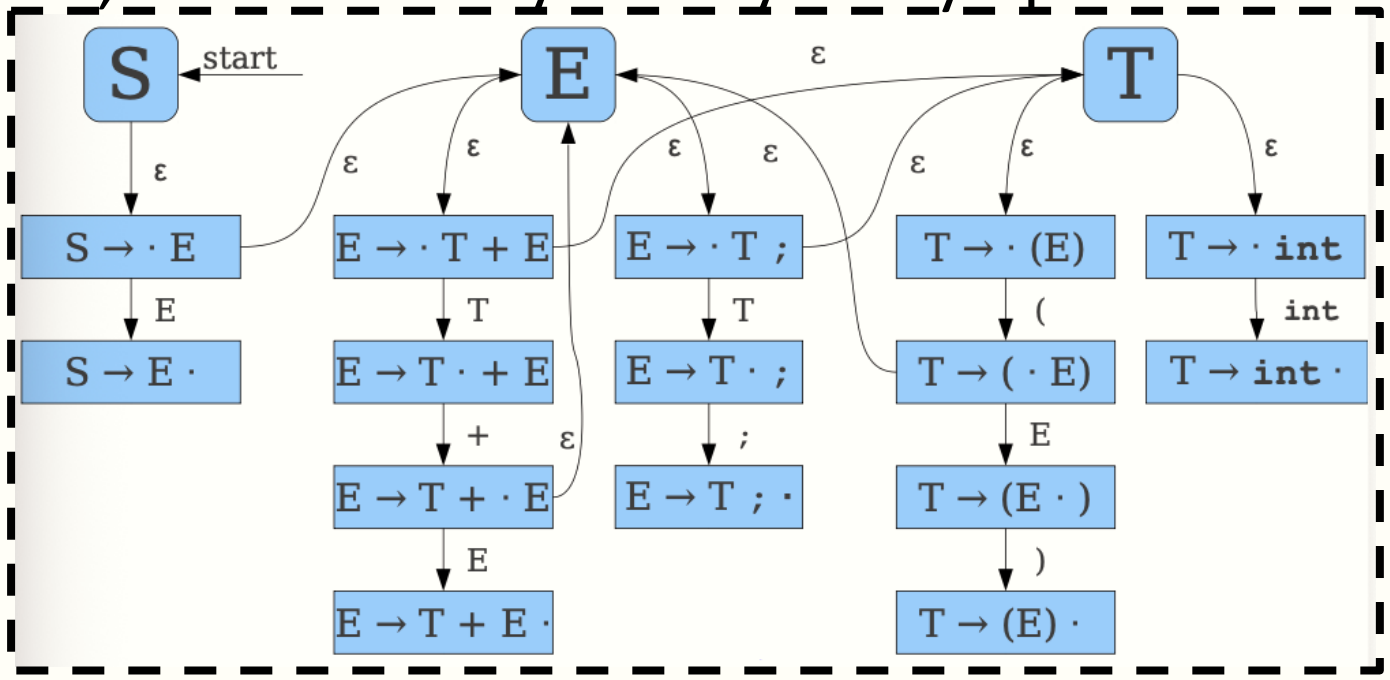
S → E
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



An Important Result

- **We can use a finite automaton as our recognizer**. Why?
  - There are only finitely many productions and positions (therein).
  - At any point, we only need to track where we are in one production.
  - At each point, there are only finitely many options.

S → E
E → T;
E → T + E
T → int
T → (E)



Constructing the Handle-Finding Automaton

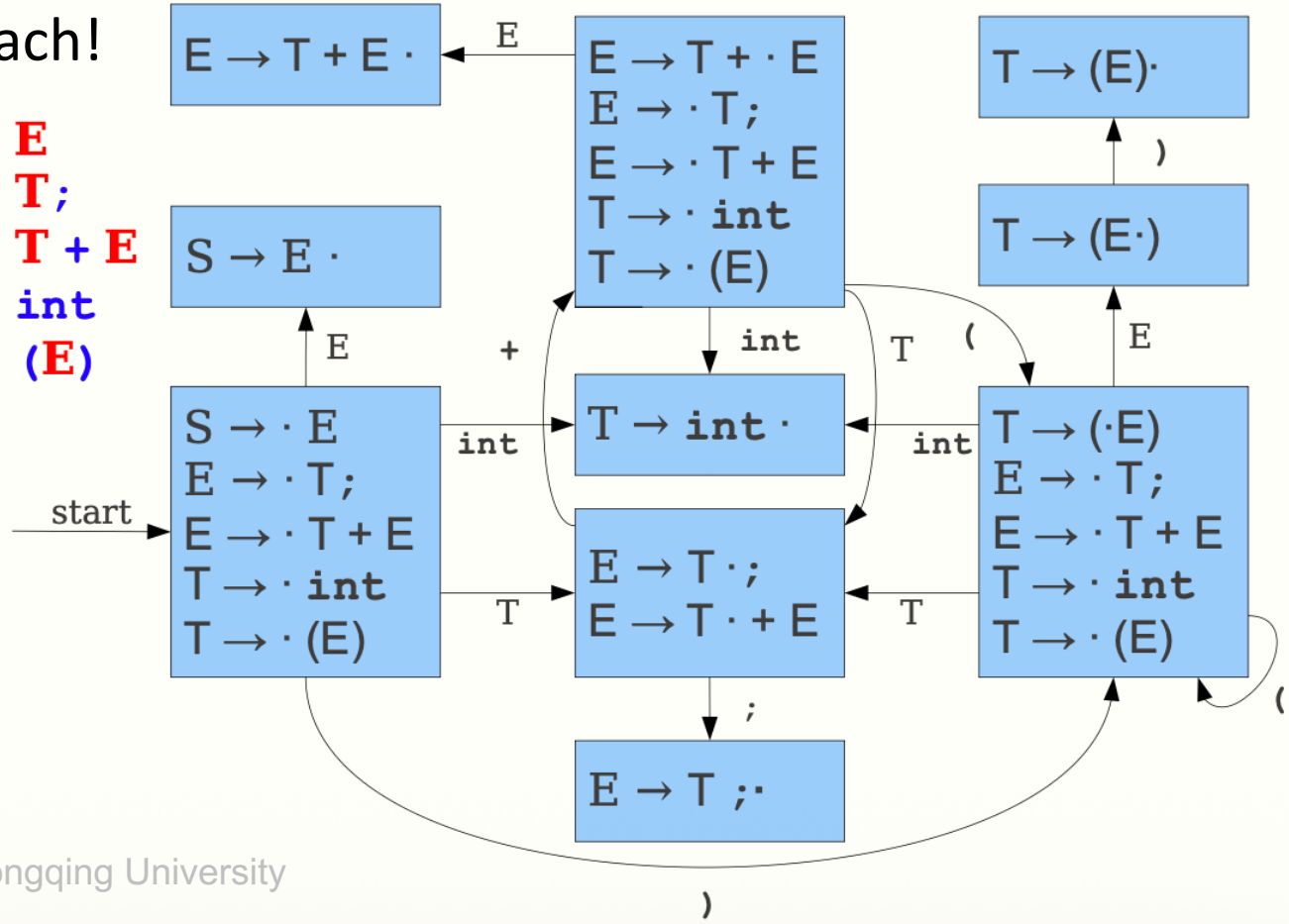
- Create a state for each nonterminal.
- For each production $\mathbf{A} \rightarrow \boldsymbol{\gamma}$:
  - Construct states $\mathbf{A} \rightarrow \boldsymbol{\alpha} \cdot \boldsymbol{\omega}$ for each possible way of splitting $\boldsymbol{\gamma}$ into two substrings $\boldsymbol{\alpha}$ and $\boldsymbol{\omega}$.
  - Add transitions on x between $\mathbf{A} \rightarrow \boldsymbol{\alpha} \cdot x\boldsymbol{\omega}$and $\mathbf{A} \rightarrow \boldsymbol{\alpha}x \cdot \boldsymbol{\omega}$.
- For each state $\mathbf{A} \rightarrow \boldsymbol{\alpha} \cdot \mathbf{B}\boldsymbol{\omega}$ for nonterminal $\mathbf{B}$, add an $\boldsymbol{\varepsilon}$-transition from $\mathbf{A} \rightarrow \alpha \cdot \mathbf{B}\omega$ to $\mathbf{B}$.

- **Why We Need the Automaton?**
  - Our goal is to identify **handles** during bottom-up parsing.
  - A state containing a completed item $\mathbf{A} \rightarrow \omega \cdot$ indicates a **possible handle**.
  - The automaton helps identify **potential handle locations**.

Making the Automaton Deterministic

- Handle-finding automata are typically implemented **deterministically**.
  - We can convert an NFA into a DFA using **subset construction**.
  - But there is a more direct approach!

S → E
E → T;
E → T + E
T → int
T → (E)



Constructing the Automaton Deterministically

- Start in a state with $\mathbf{S} \rightarrow \cdot\, \mathbf{A}$, where $\mathbf{S}$ is the augmented start symbol.
- Compute the closure of the state:
  - If $\mathbf{A} \rightarrow \alpha \cdot \mathbf{B}\omega$ is in the state, add $\mathbf{B} \rightarrow \cdot\, \boldsymbol{\gamma}$ to the state for each $\mathbf{B} \rightarrow \boldsymbol{\gamma}$.
  - Yet another fixed-point iteration!
- Repeat until no new states are added:
  - If a state contains a production $\mathbf{A} \rightarrow \boldsymbol{\alpha} \cdot x\boldsymbol{\omega}$ for symbol x, add a transition on x from that state to the state containing the closure of $\mathbf{A} \rightarrow \boldsymbol{\alpha}x \cdot \boldsymbol{\omega}$
- The DFA is equivalent to a subset construction on the NFA.
  - Could be very large, exponential to the size of the grammar.
  - Almost always generated by tools like Yacc, Bison, and Menhir.

Q3: How do we recognize handles?

- The automaton will tell us all places where a handle might be.
  - 包含$A \rightarrow \alpha \cdot$的状态表示一个potential handle (候选句柄)
- We need a way to **confirm the correct handle**.
- **Predictive bottom-up parsing** makes deterministic shift/reduce decisions.
  - LR(0), SLR(1), LALR(1), LR(1), etc.
  - Different LR algorithms use different strategies to determine when to reduce
- Different LR parsing algorithms recognize different classes of grammars.

Our First Algorithm: LR(0)

- Bottom-up predictive parsing with:
  - **L**: **L**eft-to-right scan of the input.
  - **R**: **R**ightmost derivation in reverse.
  - **(0)**: Zero tokens of lookahead.
- Use the handle-finding automaton, without any lookahead, to predict where handles are.
- Example:
  - Let us see how to parse the following with LR(0)

S → E
E → T;
E → T + E
T → int
T → (E)

int	+	(	int	+	int	;	)	;

LR(0) Parsing

(1) Stack: S0

int	+	(	int	+	int	;	)	;

Shift

(2) Stack: S0 S2

int	+	(	int	+	int	;	)	;

Reduce

可以理解成Reduce+Shift

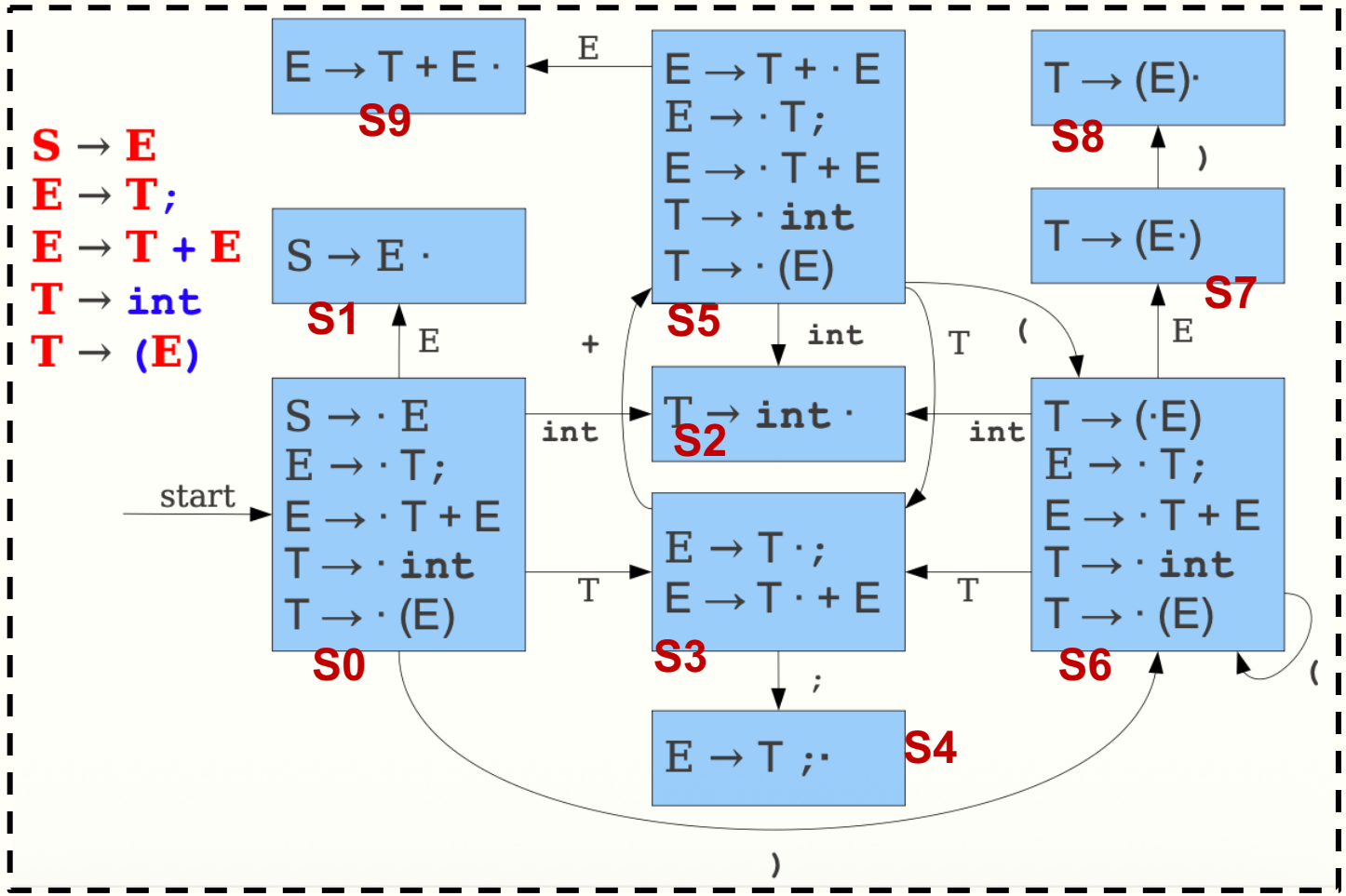
(3) Stack: S0 S3

T	+	(	int	+	int	;	)	;

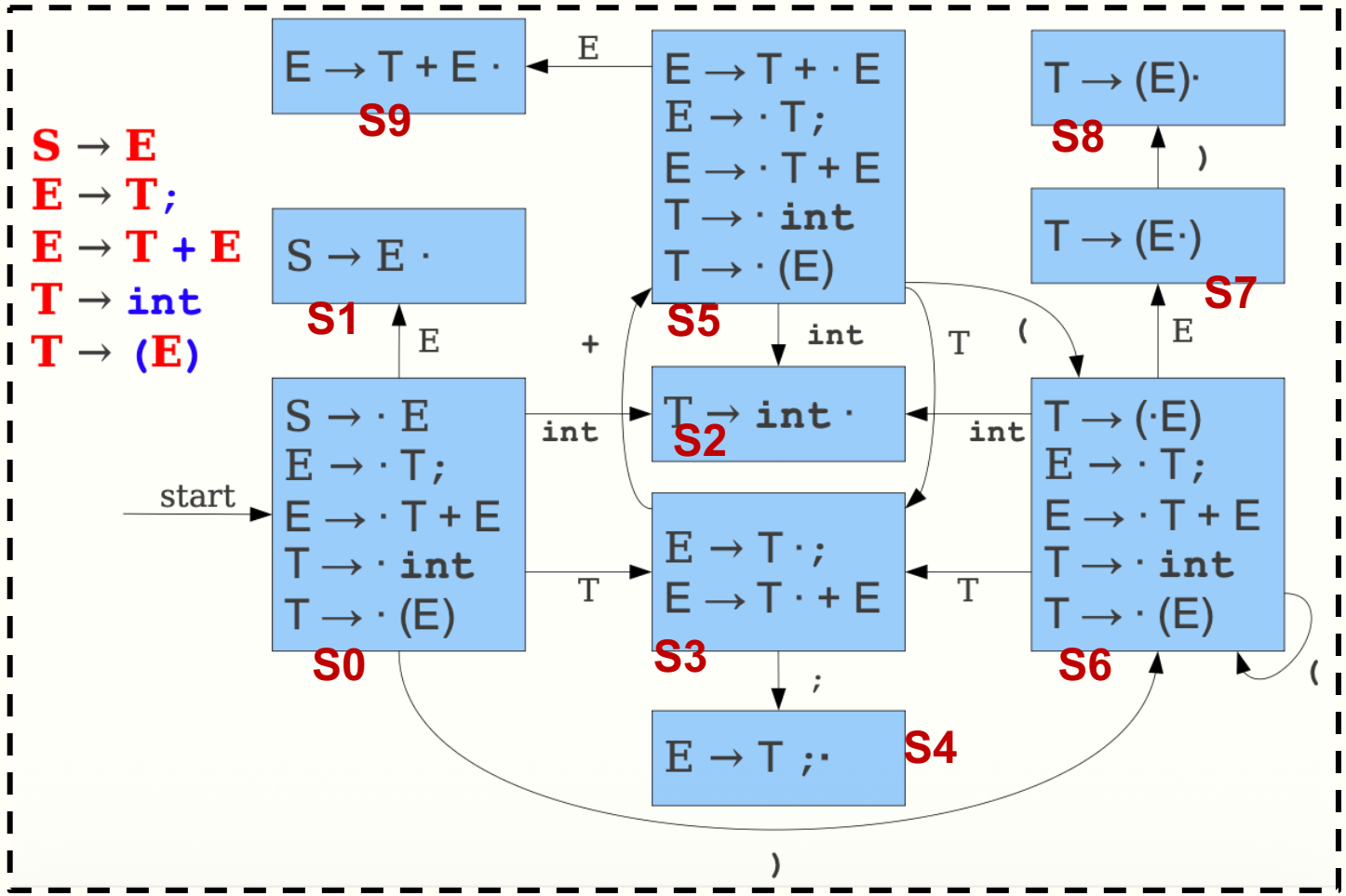
Shift

(4) Stack: S0 S3 S5

T	+	(	int	+	int	;	)	;



LR(0) Parsing



(5) Stack: S0 S3 S5 S6

Shift

T	+	(	int	+	int	;	)	;

Shift

(6) Stack: S0 S3 S5 S6 S2

T	+	(	int	+	int	;	)	;

Reduce

(7) Stack: S0 S3 S5 S6 S3

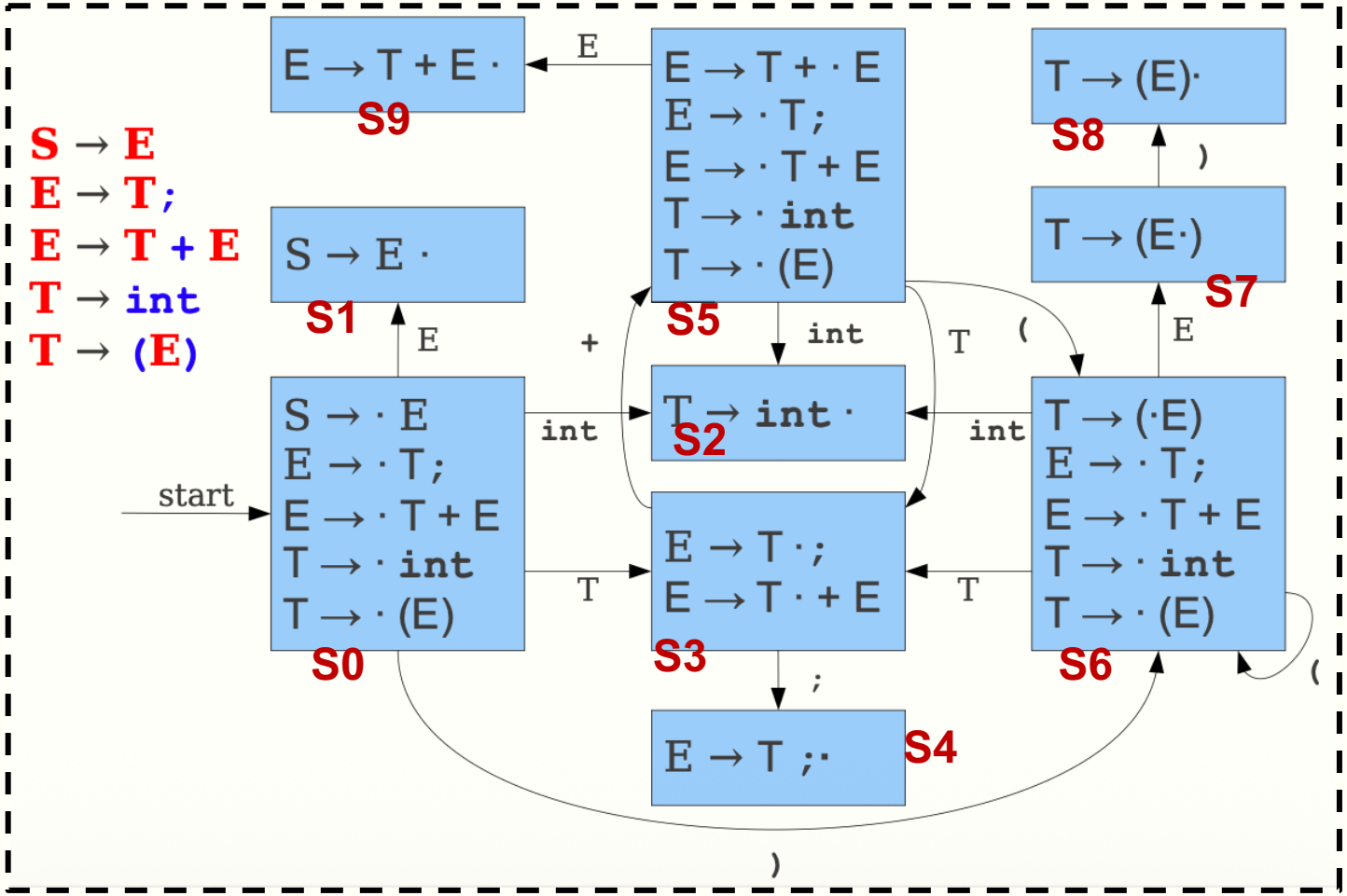
T	+	(	T	+	int	;	)	;

2 Shifts

(8-9) Stack: S0 S3 S5 S6 S3 S5 S2

T	+	(	T	+	int	;	)	;

LR(0) Parsing



Reduce

(10) Stack: S0 S3 S5 S6 S3 S5 S3

T	+	(	T	+	T	;	)	;

Shift

(11) Stack: S0 S3 S5 S6 S3 S5 S3 S4

T	+	(	T	+	T	;	)	;

Reduce

(11) Stack: S0 S3 S5 S6 S3 S5 S9

T	+	(	T	+	E	)	;

Reduce

(12) Stack: S0 S3 S5 S6 S7

T	+	(	E	)	;

LR(0) Parsing

Shift

(13) Stack: S0 S3 S5 S6 S7 S8

T	+	(	E	)	;

Reduce

(14) Stack: S0 S3 S5 S3

T	+	T	;

Shift

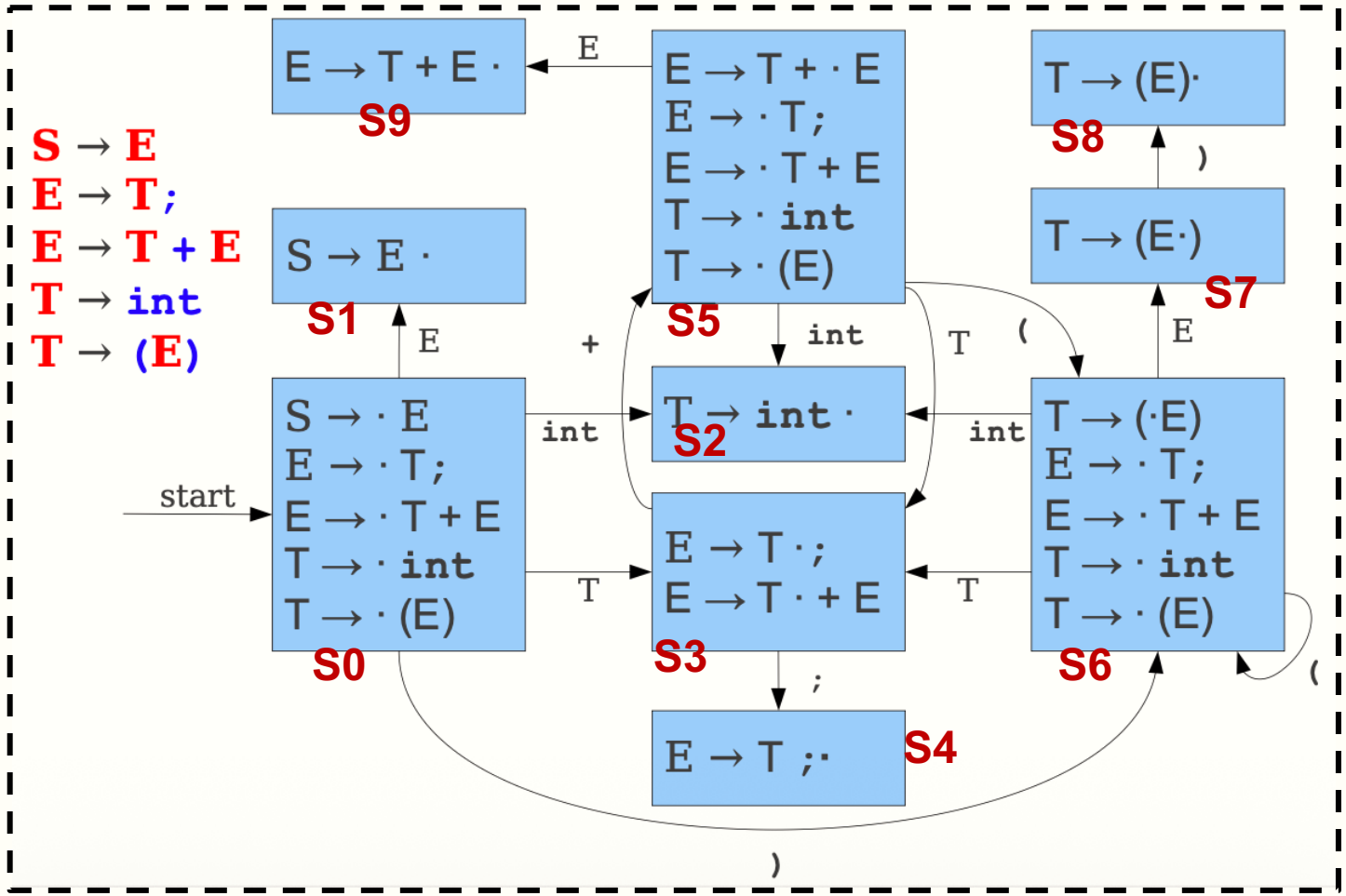
(15) Stack: S0 S3 S5 S3 S4

T	+	T	;

Reduce

(16) Stack: S0 S3 S5 S9

T	+	E

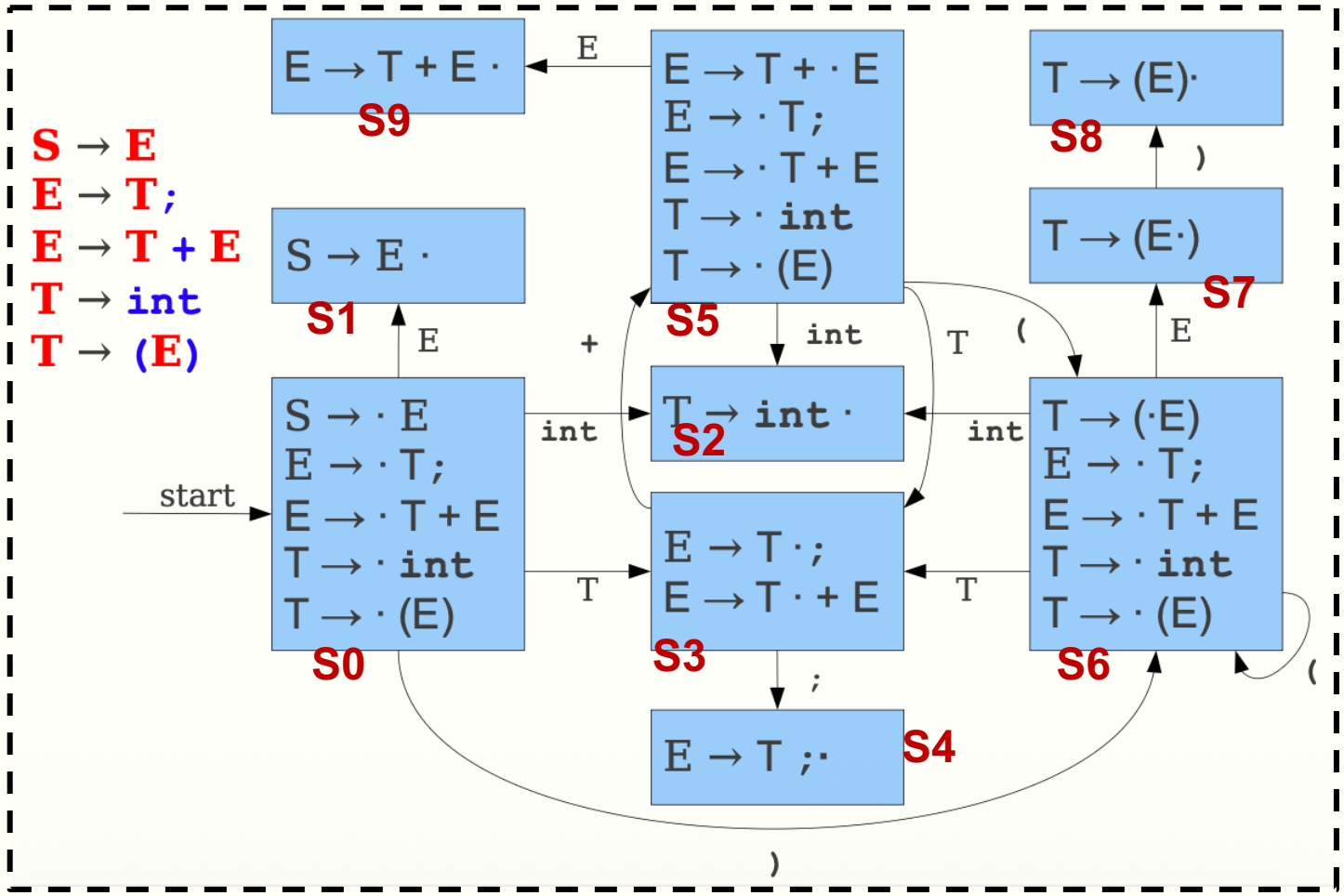


LR(0) Parsing

Reduce

(17) Stack: S0 S1





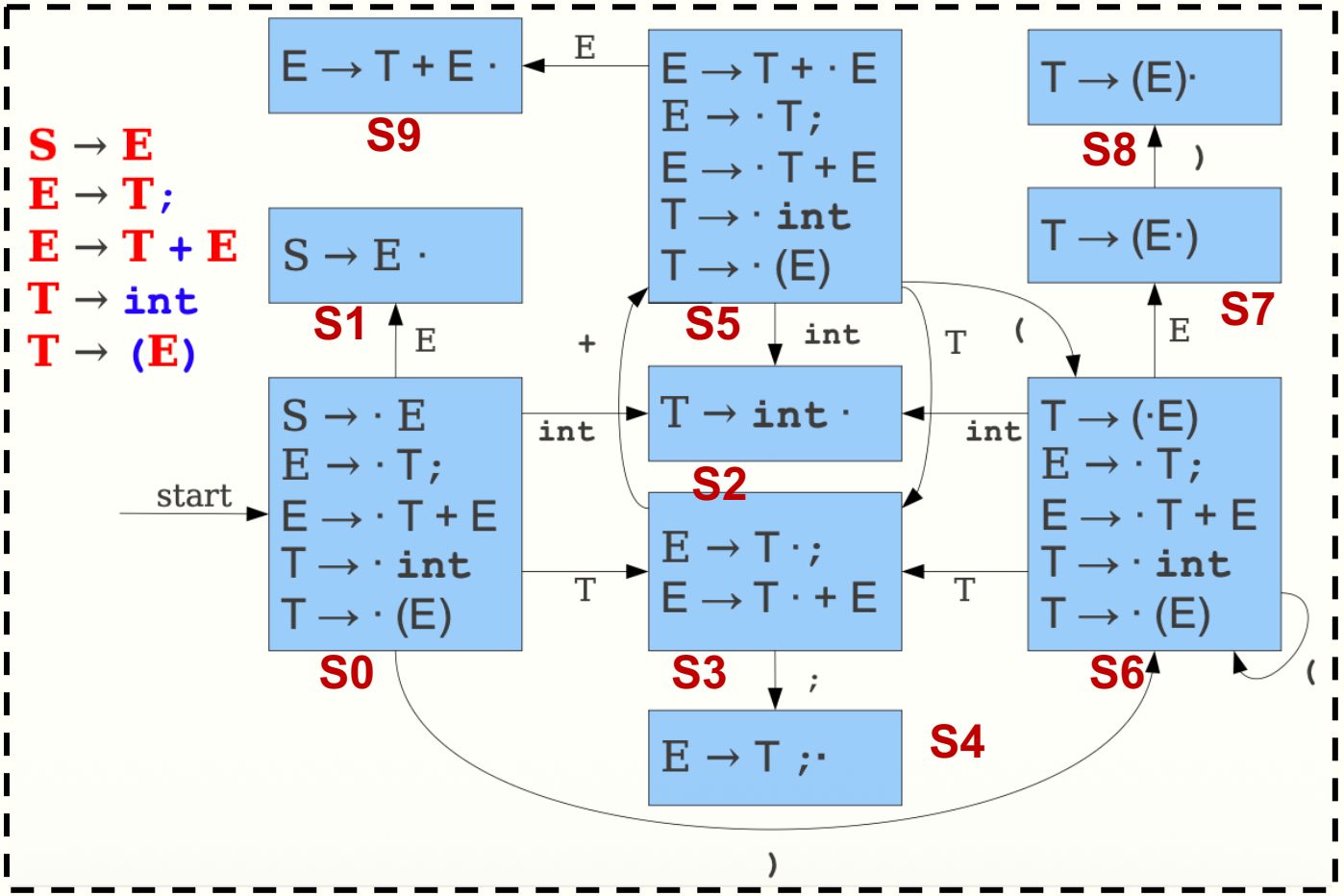
S1 is an accepting state!

We are done.

Representing the Automaton

- LR(0) parsers are usually represented via two tables:
  - an **action** table and a **goto** table.
- The **action** table maps each state to an action:
  - **shift**, which shifts the next terminal to the stack, and
  - **reduce** $\mathbf{A} \rightarrow \boldsymbol{\omega}$, which performs reduction $\mathbf{A} \rightarrow \boldsymbol{\omega}$.
  - Any state of the form $\mathbf{A} \rightarrow \boldsymbol{\gamma} \cdot$ does that reduction; everything else shifts.
- The **goto** table maps state/symbol pairs to a next state.
  - This is just the transition table for the automaton.
- Usually, we have two stacks: symbol stack S_1 and state stack S_2
  - **Shift**: move an input token **t** to S_1 , and move **goto**(S_2^{top}, **t**) to S_2
  - **Reduce** $\mathbf{A} \rightarrow \boldsymbol{\gamma} \cdot$: pop tokens in $\boldsymbol{\gamma}$ in reverse from S_1, and push A into S_1; pop $|\boldsymbol{\gamma}|$ states from S_2, and push **goto**(S_2^{top}, **A**) into S_2
  - One stack is also fine, maintain a stack of pair (t, s) with $t \in S_1$ and $s \in S_2$

Building LR(0) Tables: Example



	int	+	;	(	)	E	T	Action
0	2				8	1	3	Shift
1								Accept
2								Reduce T → int
3		5	4					Shift
4								Reduce E → T ;
5	2			6		9	3	Shift
6	2			6		7	3	Shift
7					8			Shift
8								Reduce T → (E)
9								Reduce E → T + E

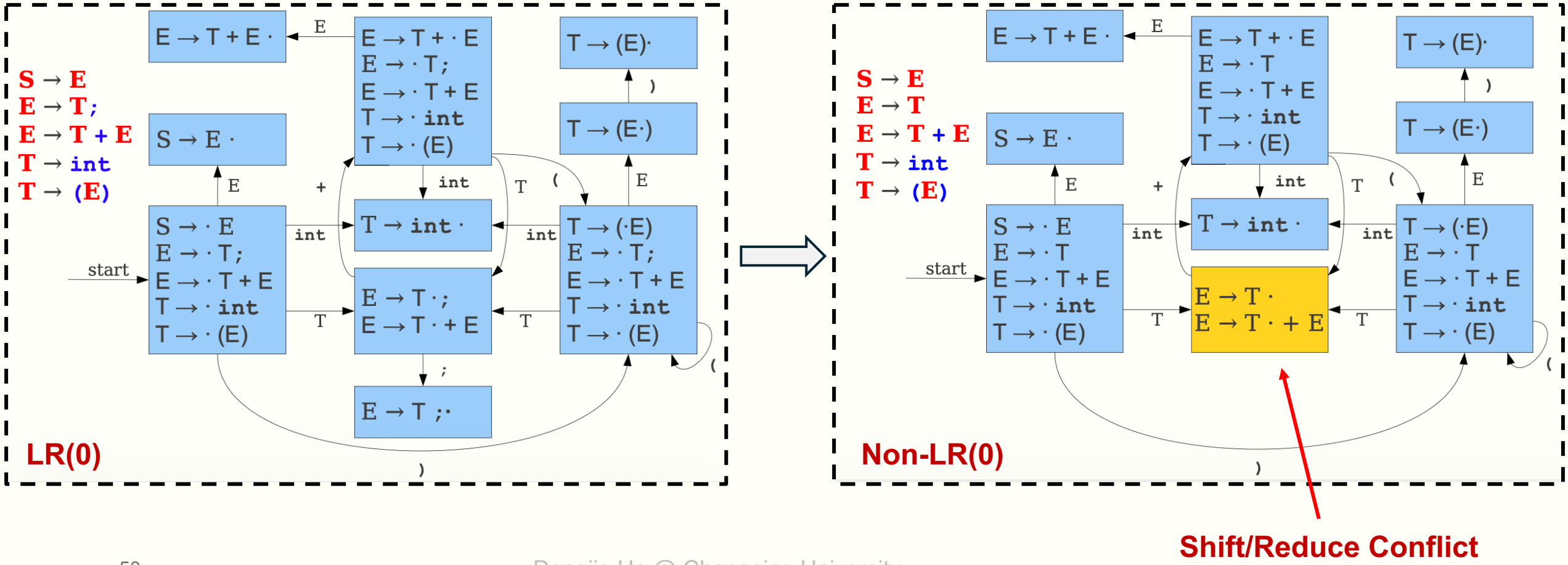
GOTO

The LR(0) Algorithm

- Maintain a stack of (symbol, state) pairs
  - which is initially (**?**, 1) for some dummy symbol **?**.
- While the stack is not empty:
  - Let S_{top} be the top state.
  - If **action**[S_{top}] is **shift**:
    - Let **t** be the next symbol in the input.
  - Push (**t**, **goto**[S_{top}, **t**]) atop the stack.
  - If **action**[S_{top}] is **reduce** $\mathbf{A} \rightarrow \boldsymbol{\omega}$:
    - Remove |**ω** | symbols from the top of the stack.
    - Let **top-state** be the state on top of the stack now.
    - Push (**A**, **goto**[**top-state**, **A**]) atop the stack.
  - Otherwise, report an error.

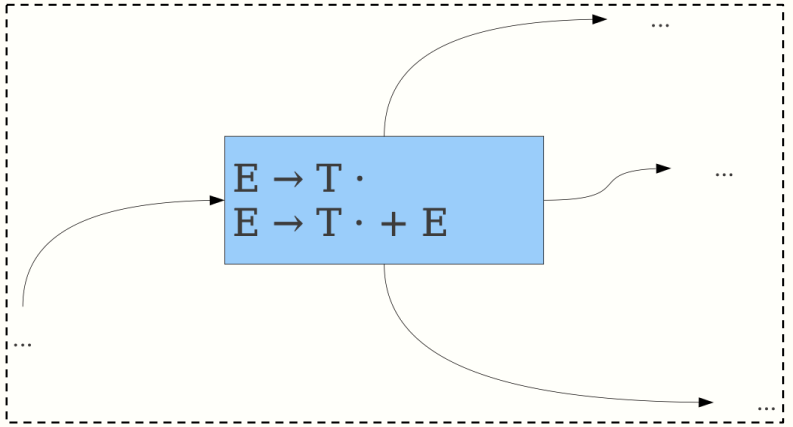
The Limits of LR(0)

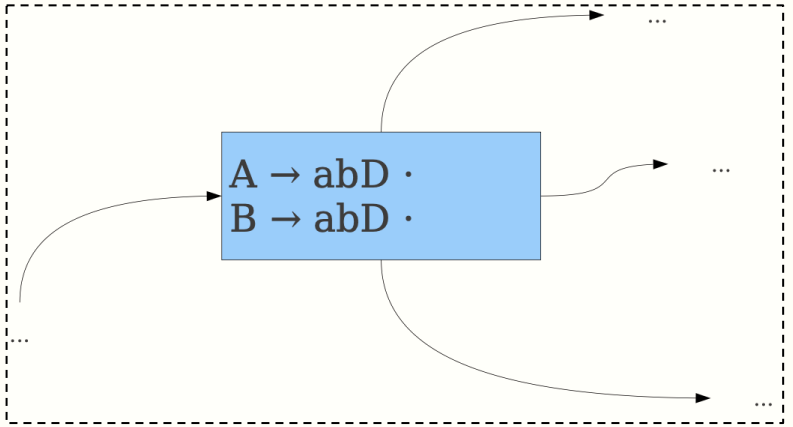
- A Non-LR(0) Grammar and its handle-finding Automaton



LR Conflicts

- **Shift/Reduce conflict**:
  - An error where a shift/reduce parser cannot tell whether to shift a token or perform a reduction.
  - A state where a handle might occur, but we might actually need to keep searching.
  - Often happens when two productions overlap.
- **Reduce/Reduce conflict**:
  - An error where a shift/reduce parser cannot tell which of many reductions to perform.
  - A state where we know we have found the handle, but can't tell which reduction to apply.
  - Often the result of ambiguous grammars.
- Can you have a **Shift/Shift conflict**?





The Limits of LR(0)

- A grammar whose handle-finding automaton contains a shift/reduce conflict or a reduce/reduce conflict is not LR(0).
- **LR(0) is Weak**
  - LR(0) only accepts languages where the handle can be found with no right context.
  - Our shift/reduce parser only looks to the left of the handle, not to the right.
- How do we exploit the tokens after a possible handle to determine what to do?

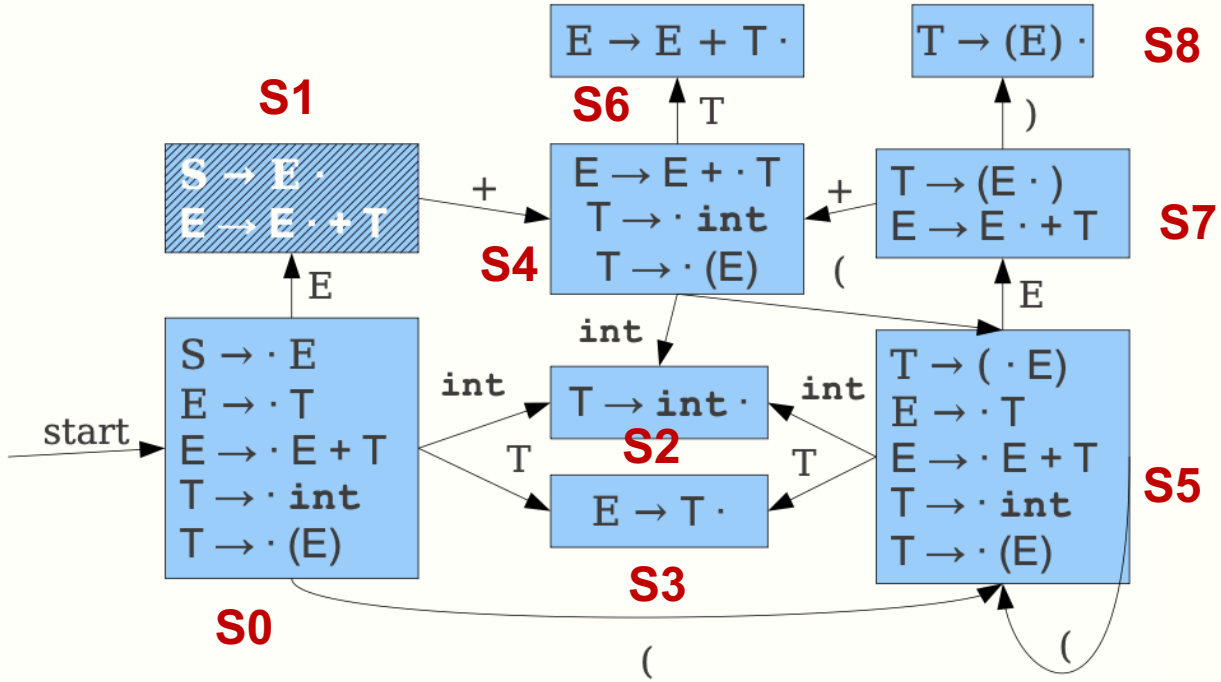
A Powerful Parser: LR(1)

- Bottom-up predictive parsing with
  - **L**: **L**eft-to-right scan
  - **R**: **R**ightmost derivation
  - **(1)**: One token lookahead
- Why LR(1)?
  - Substantially more powerful than previous parsing methods.
  - Use lookahead to make smarter parsing decisions.
- **Key Idea**
  - Track possible productions during parsing.
  - Use one **lookahead token** to decide shift or reduce.
    - 当前产生式完成归约时允许出现在输入中的下一个 token

LR(1) Parsing: The Intuition

$S \rightarrow E$
$E \rightarrow T$
$E \rightarrow E + T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$

LR(0) Handle-Finding DFA



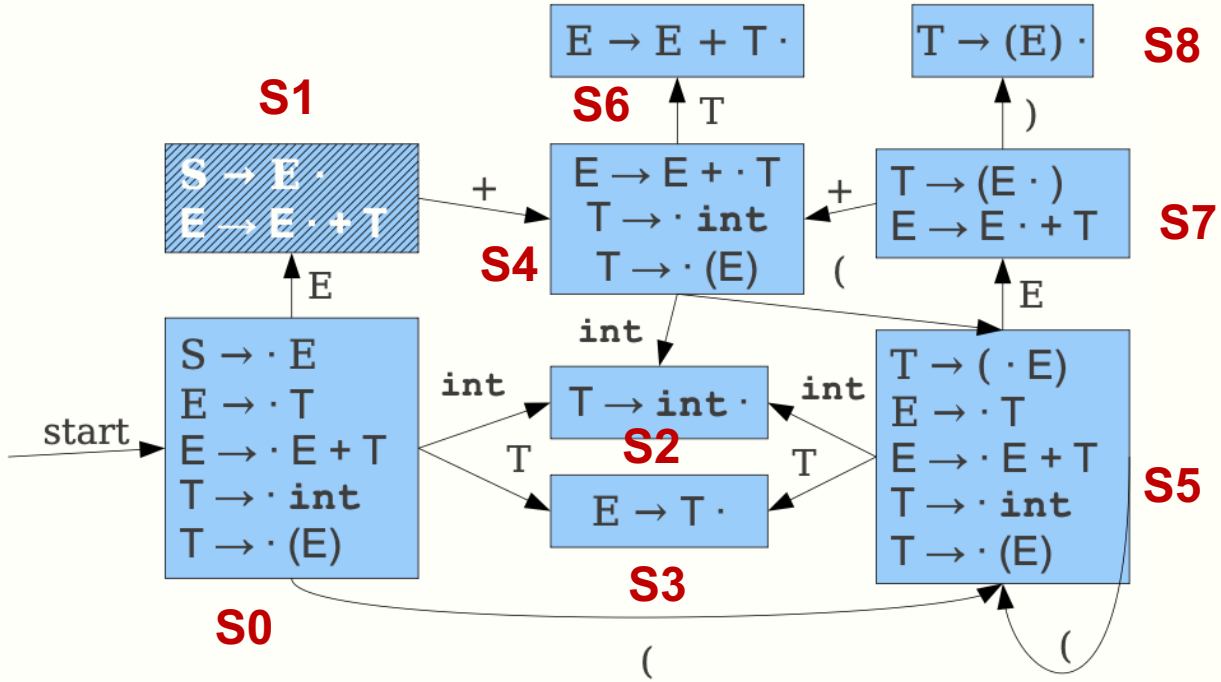
Let us try to parse the following tokens.

int	+	(	int	+	int	+	int	)	$

LR(1) Parsing: The Intuition

$S \rightarrow E$
$E \rightarrow T$
$E \rightarrow E + T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$

LR(0) Handle-Finding DFA



$S \rightarrow \cdot E$	$
$E \rightarrow \cdot E + T$	$
$E \rightarrow \cdot T$	+
$T \rightarrow \cdot \texttt{int}$	+

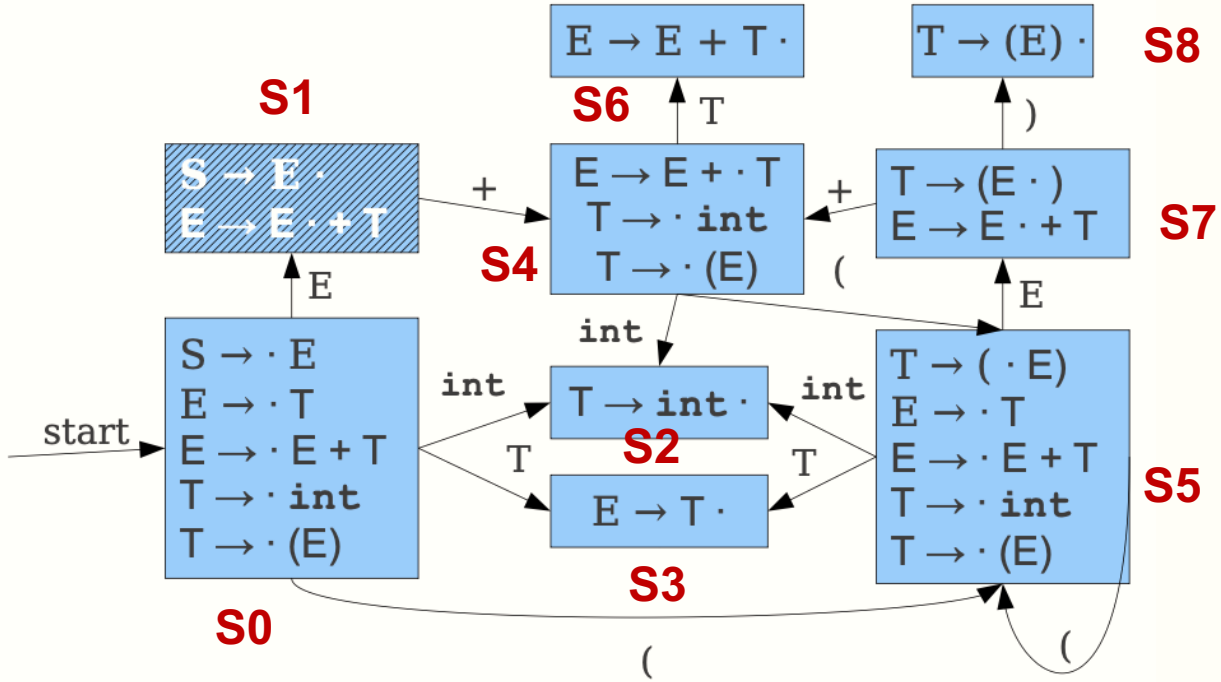
⇐ **Track the possible productions during parsing.**

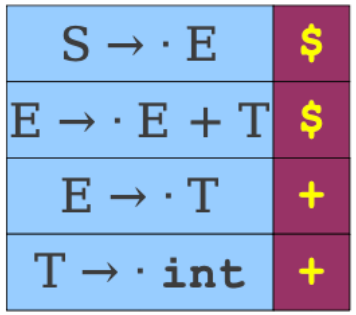
int	+	(	int	+	int	+	int	)	$

LR(1) Parsing: The Intuition

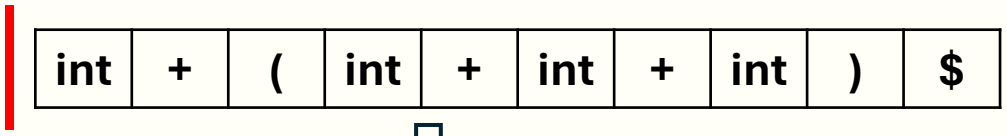
$S \rightarrow E$
$E \rightarrow T$
$E \rightarrow E + T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$

LR(0) Handle-Finding DFA

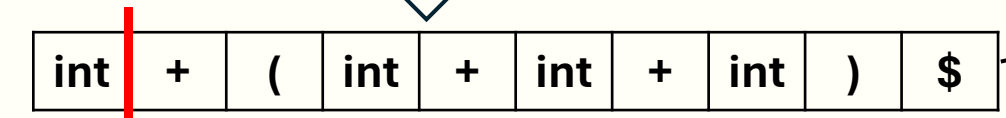




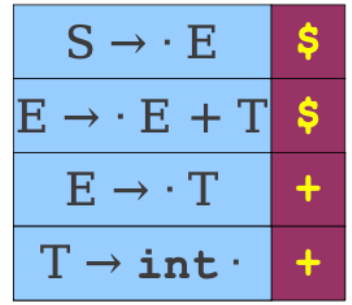
(1) Stack: S0



Shift



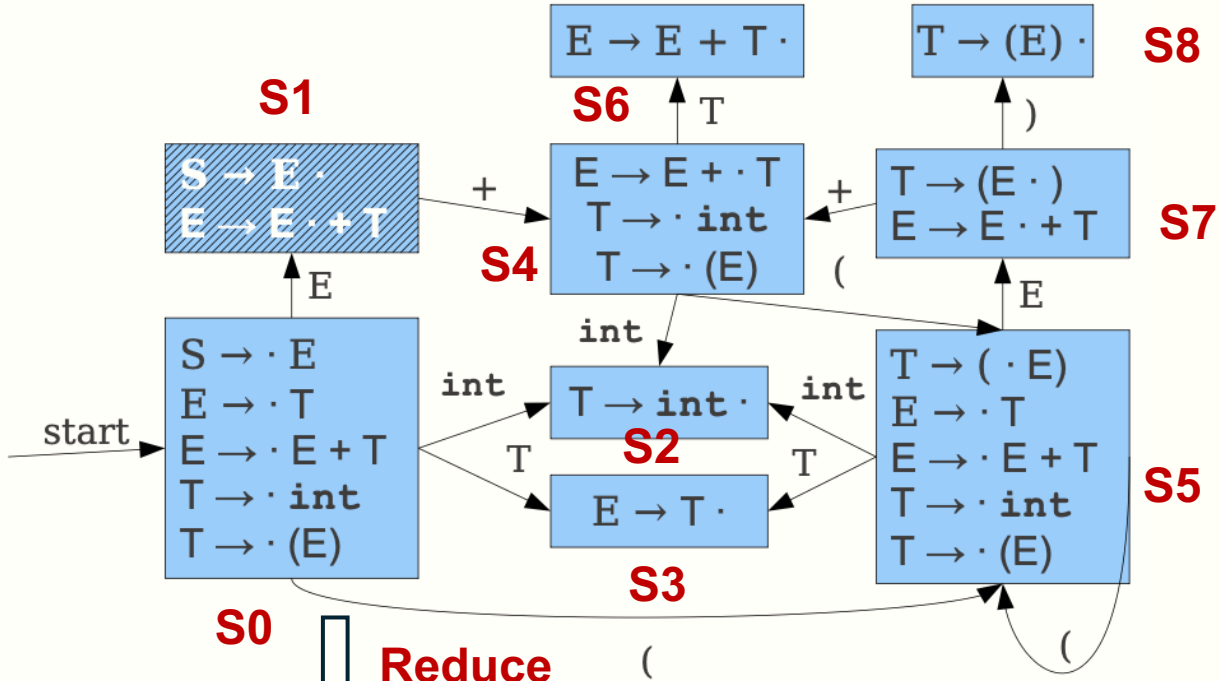
(2) Stack: S0 S2



LR(1) Parsing: The Intuition

$S \rightarrow E$
$E \rightarrow T$
$E \rightarrow E + T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$

LR(0) Handle-Finding DFA



$S \rightarrow \cdot E$	$
$E \rightarrow \cdot E + T$	$
$E \rightarrow T \cdot$	+

(3) Stack: S0 S3

T	+	(	int	+	int	+	int	)	$

Reduce

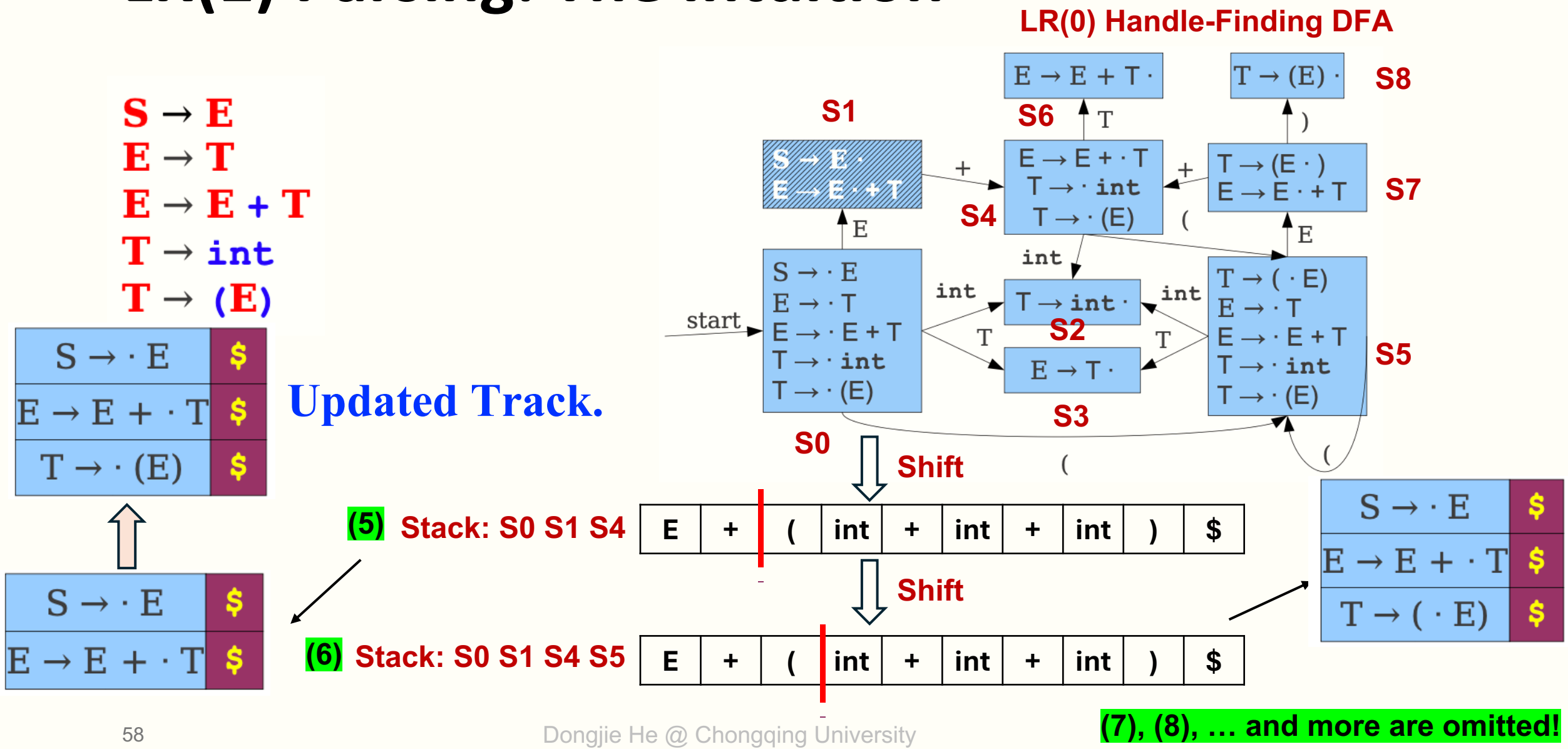
(4) Stack: S0 S1

E	+	(	int	+	int	+	int	)	$

$S \rightarrow \cdot E$	$
$E \rightarrow E \cdot + T$	$

Next, we will do a Shift.

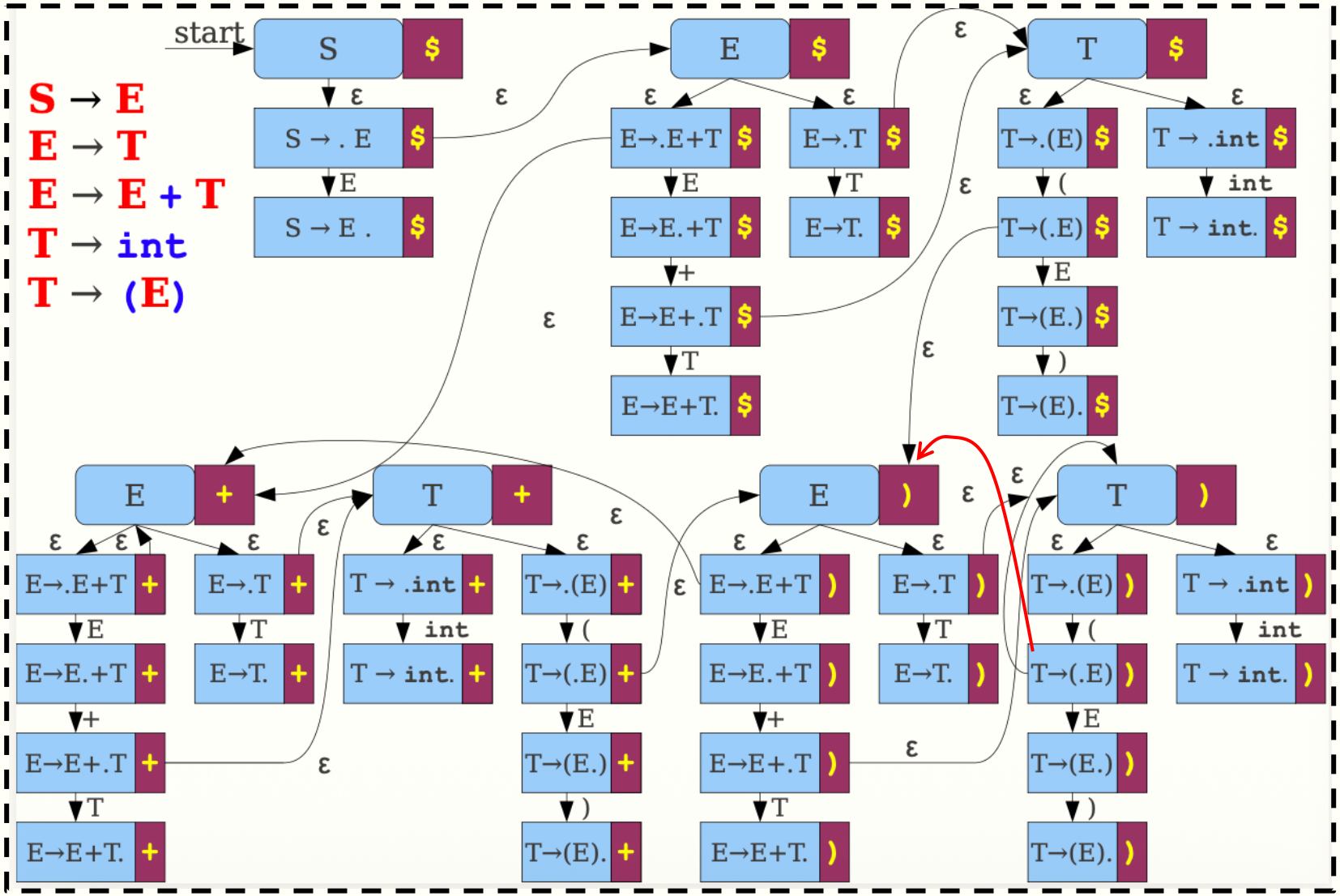
LR(1) Parsing: The Intuition



Tracking Lookaheads

- How do we know what lookahead to expect at each state?
- Observation:
  - There are only **finitely many productions** we can be in at any point.
  - There are only **finitely many positions** we can be in each production.
  - There are only **finitely many lookahead sets** at each point.
- Construct an automaton to track lookaheads!

Constructing LR(1) Automata: Example

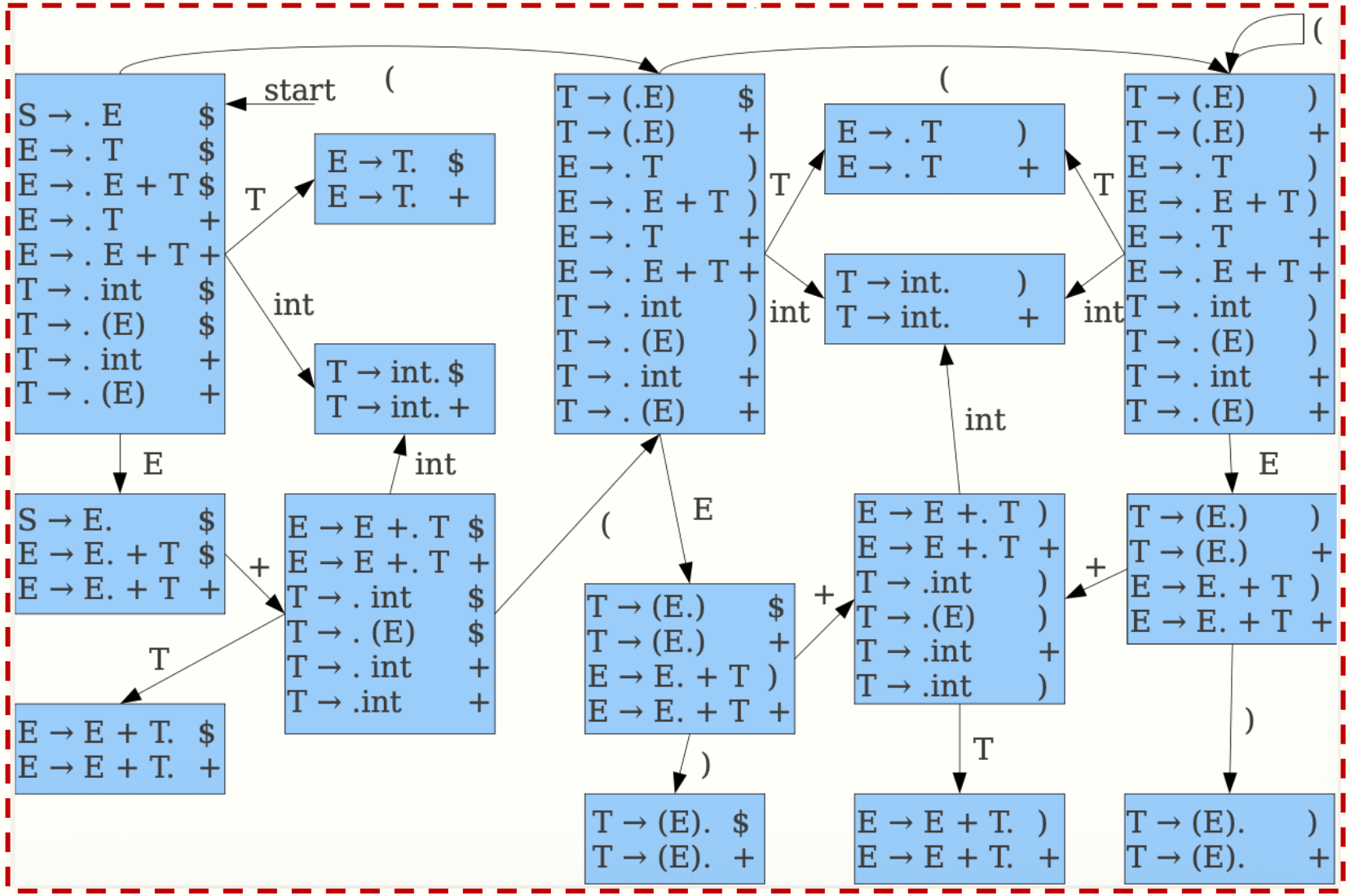


Constructing LR(1) Automata

- Begin with a state **S** [**$**].
- For each state **A** [**t**], for each production $\mathbf{A} \to \boldsymbol{\gamma}$:
  - Construct states $\mathbf{A} \to \boldsymbol{\alpha} \cdot \boldsymbol{\omega}$ [**t**] for all possible ways of splitting $\boldsymbol{\gamma} = \boldsymbol{\alpha\omega}$.
  - Add an ε-transition from **A** [**t**] to each initial item $\mathbf{A} \to \cdot\, \boldsymbol{\gamma}$ [**t**].
  - Add transitions on x between $\mathbf{A} \to \boldsymbol{\alpha} \cdot x\boldsymbol{\omega}$[**t**] and $\mathbf{A} \to \boldsymbol{\alpha}x \cdot \boldsymbol{\omega}$[**t**]
- For each state $\mathbf{A} \to \boldsymbol{\alpha} \cdot \mathbf{B}\boldsymbol{\omega}$ [**t**], add an ε-transition from $\mathbf{A} \to \boldsymbol{\alpha} \cdot \mathbf{B}\boldsymbol{\omega}$[**t**] to **B** [**r**] for each terminal **r** ∈ **FIRST***(**ωt**).
  - Review **FIRST*** definition**:**
    - **FIRST***(t$\boldsymbol{\omega}$) = { t }
    - If **Nullable**(**A**)=False: **FIRST***(**A**$\boldsymbol{\omega}$) = **FIRST**(**A**)
    - If **Nullable**(**A**)=True: **FIRST***(**A**$\boldsymbol{\omega}$) = **FIRST**(**A**) ∪ **FIRST***($\boldsymbol{\omega}$)

Deterministic LR(1) Automata

$S \rightarrow E$
$E \rightarrow T$
$E \rightarrow E + T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$

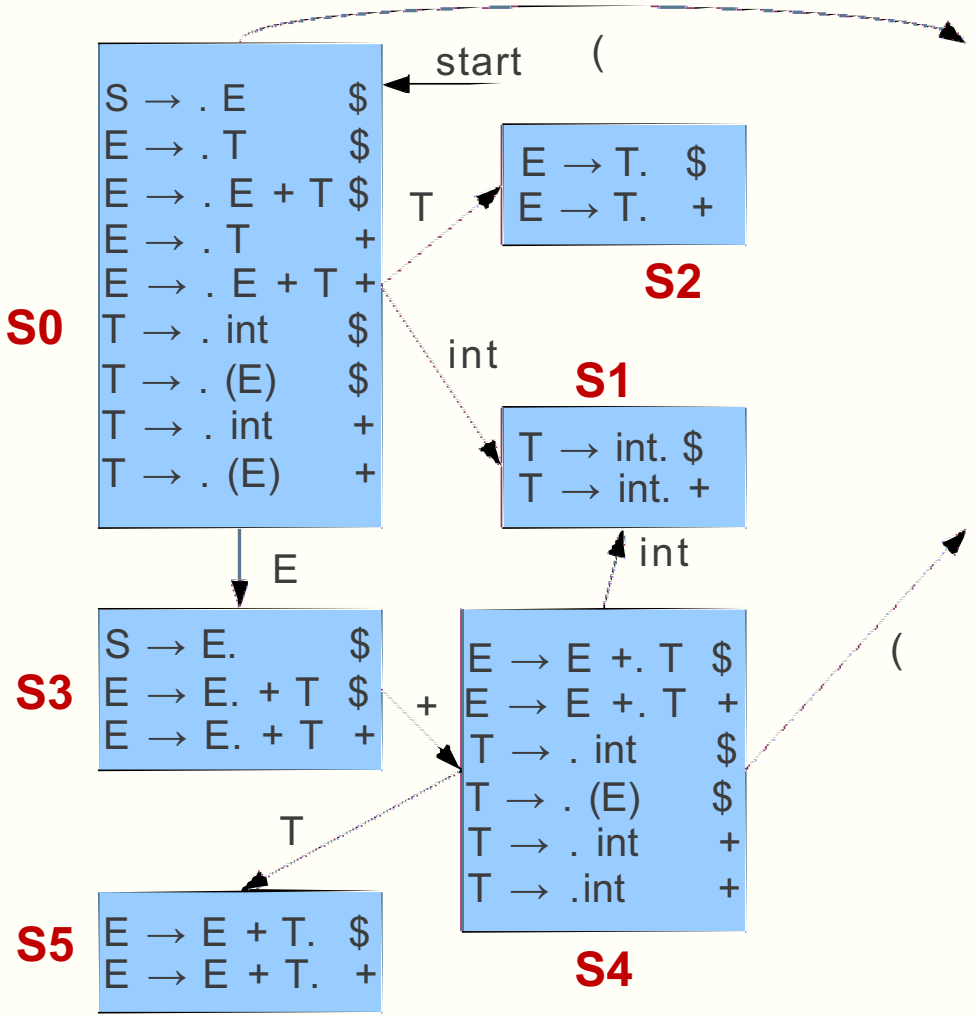


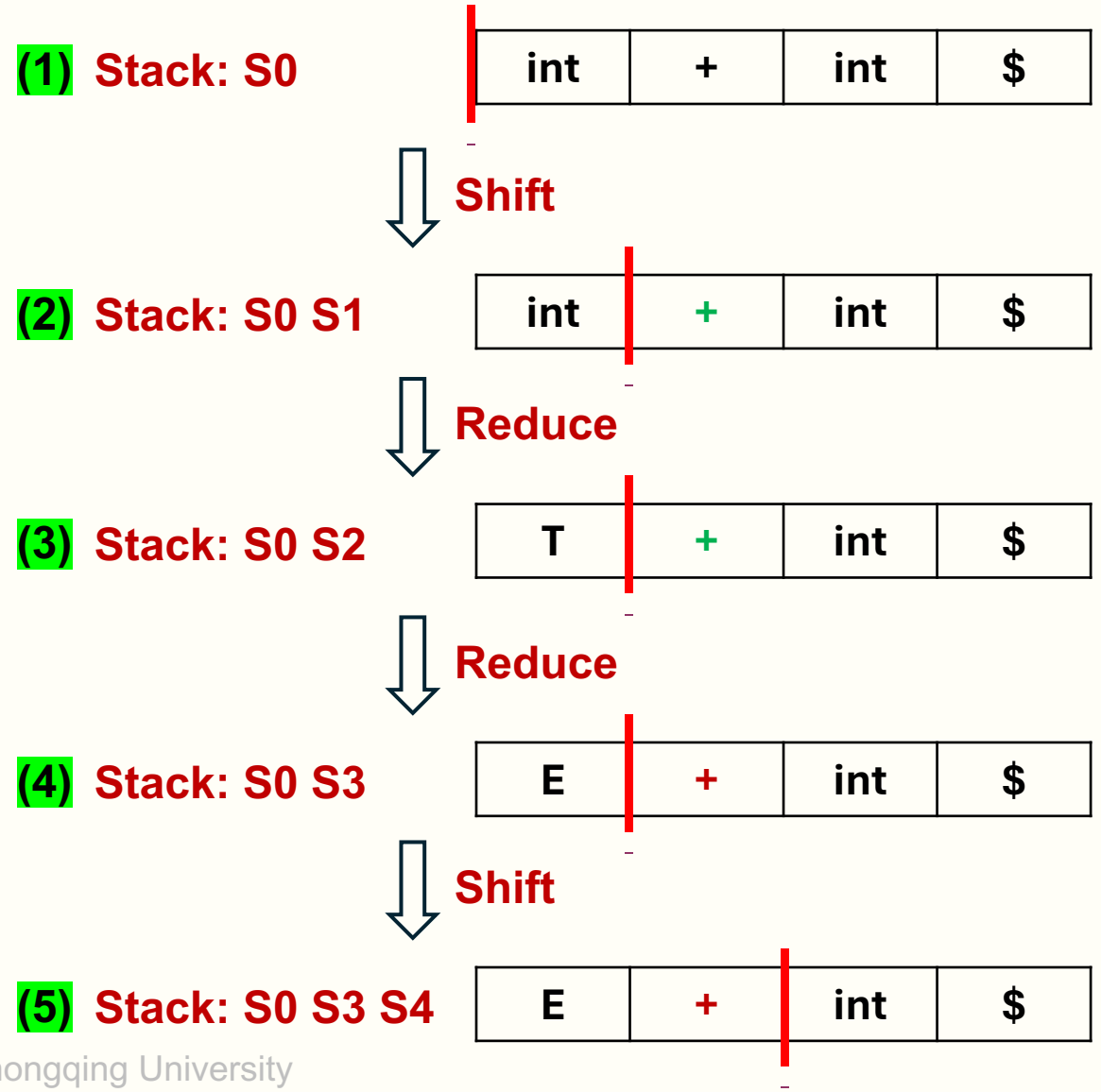
Constructing LR(1) Automata, Deterministically

- Begin in a state containing $\mathbf{S} \rightarrow \cdot\ \mathbf{E}[\$]$, where $\mathbf{S}$ is the start symbol.
- Compute the **closure** of the state:
  - If $\mathbf{A} \rightarrow \boldsymbol{\alpha} \cdot \mathbf{B}\boldsymbol{\omega}[\mathbf{t}]$ is in the state, add $\mathbf{B} \rightarrow \cdot\ \boldsymbol{\gamma}[\mathbf{t}]$ to the state for each production $\mathbf{B} \rightarrow \boldsymbol{\gamma}$ and for each terminal $\mathbf{t} \in \mathbf{FIRST} * (\boldsymbol{\omega}\mathbf{t})$
- Repeat until no new states are added:
  - If a state contains a production $\mathbf{A} \rightarrow \boldsymbol{\alpha} \cdot x\boldsymbol{\omega}[\mathbf{t}]$, add a transition on x from that state to the state containing the closure of $\mathbf{A} \rightarrow \boldsymbol{\alpha}x \cdot \boldsymbol{\omega}[\mathbf{t}]$.

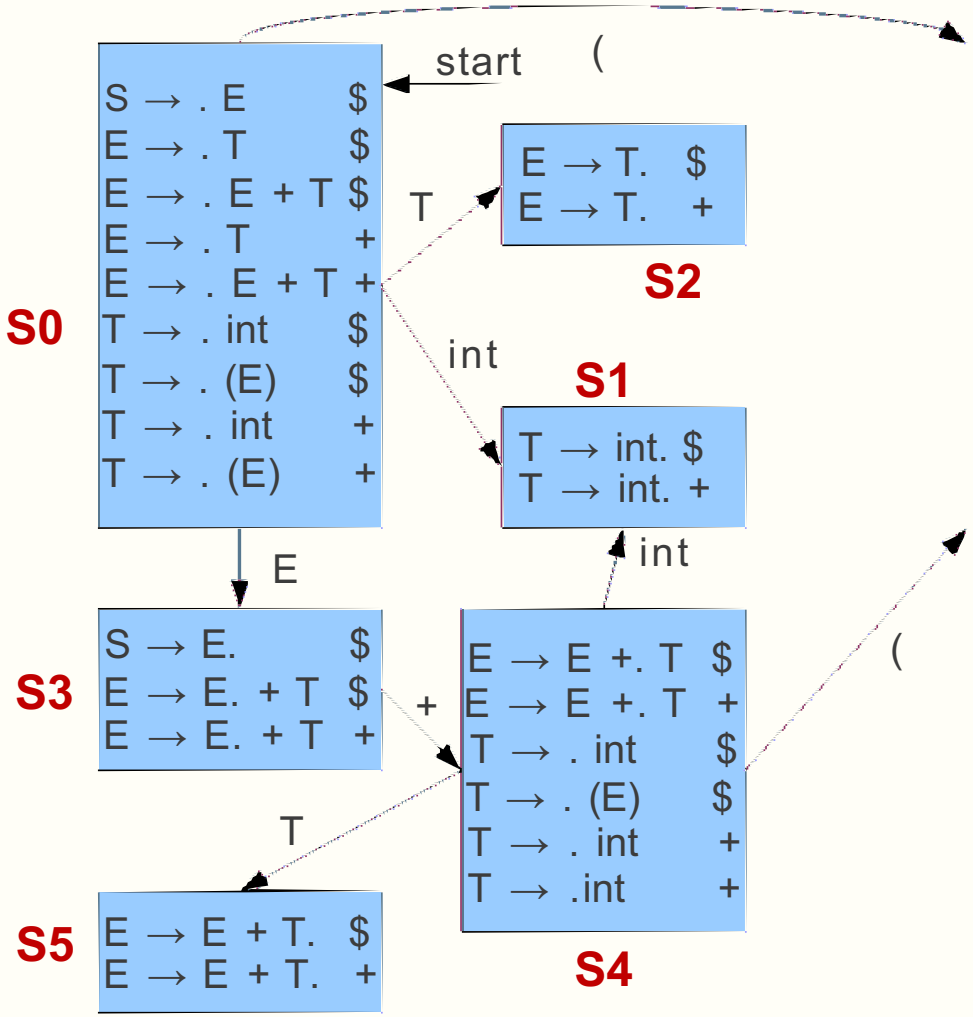
- LR(1) automaton simulates two processes simultaneously:
  - An LR(0) automaton for finding handles.
  - A lookahead tracker for determining what the lookahead is.

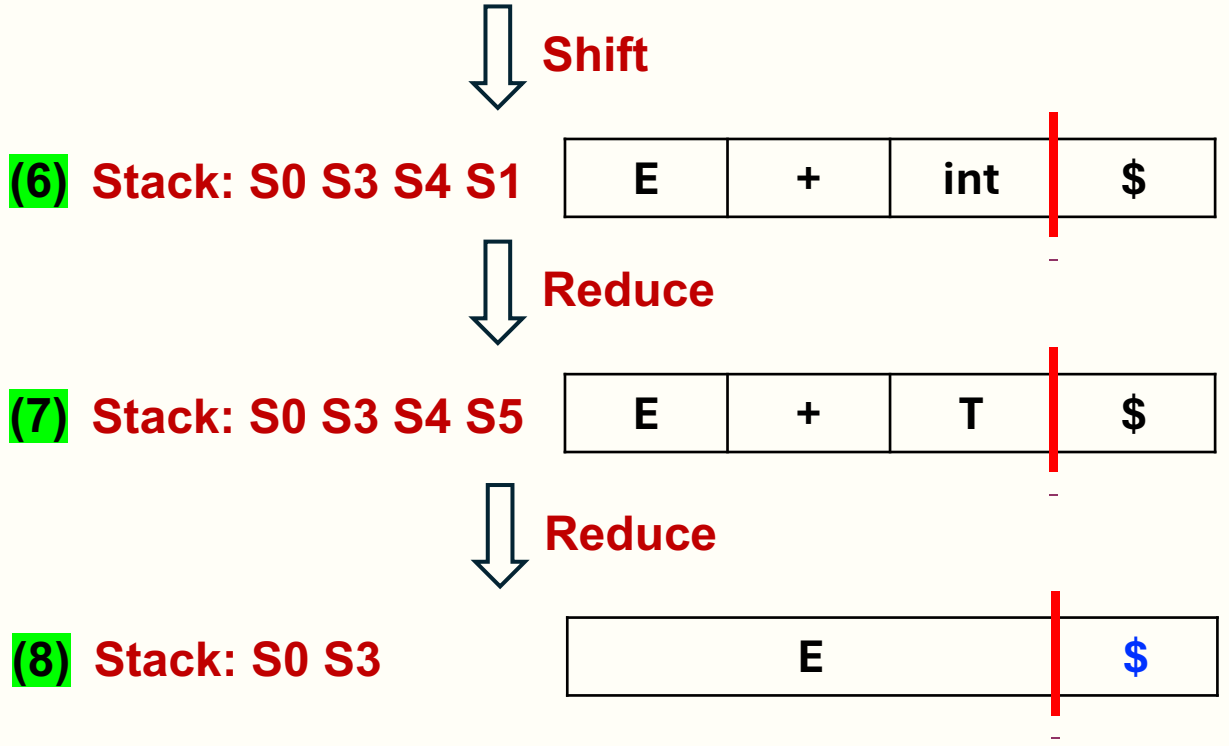
Use Deterministic LR(1) Automata in Parsing





Use Deterministic LR(1) Automata in Parsing

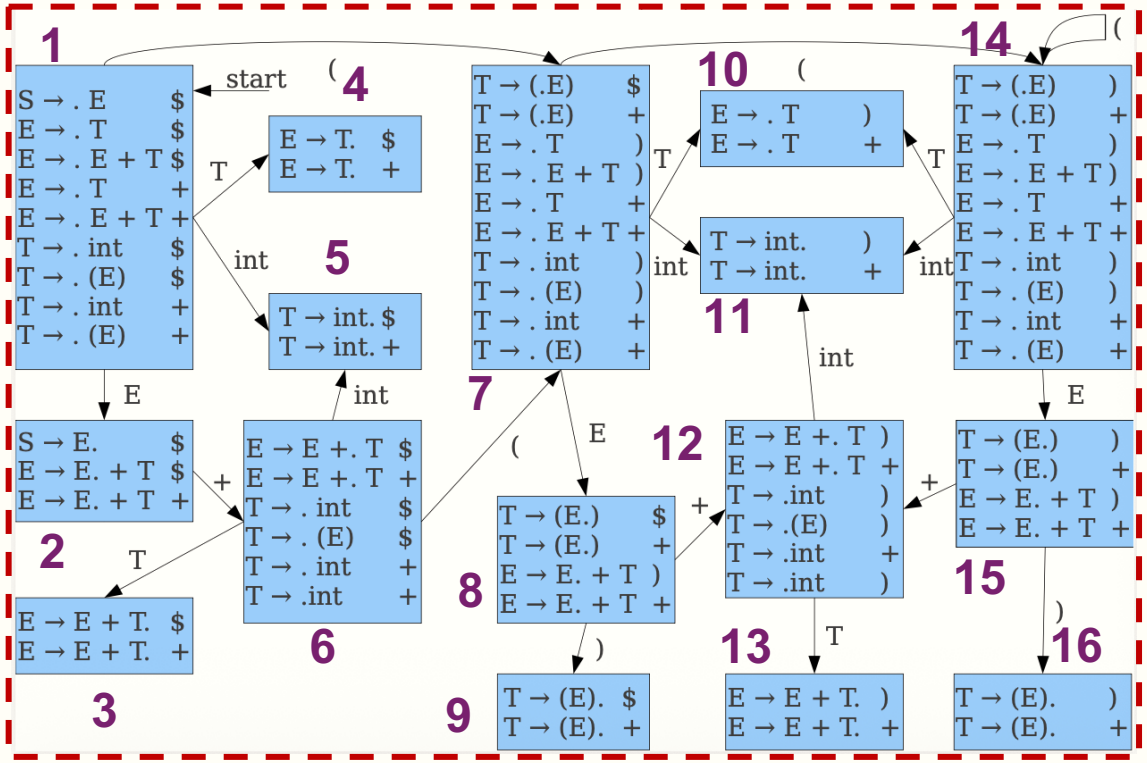




Accept it!

Representing LR(1) Automata

- **goto table:** encodes transition as map from (state, token) to states.
- **action table:** maps pairs (state, lookahead) to actions
- Commonly combined into a single **action/goto table**.



S → E (1)
E → T (2)
E → E + T (3)
T → int (4)
T → (E) (5)

	int	(	)	+	$	T	E
1	s5					s4	s2
2				s6	ACCEPT		
3				r3	r3		
4				r2	r2		
5				r5	r5		
6	s5	s7				s3	
7	s11	s14				s10	s8
8			s9	s12			
9				r5	r5		
10			r2	r2			
11			r4	r4			
12	s11					s13	
13			r3	r3			
14	s11		s14			s10	s15
15			s16	s12			
16			r5	r5			

Constructing LR(1) Parse Tables

- For each state **X**:
  - If there is a production $\mathbf{A} \rightarrow \boldsymbol{\omega} \cdot [\mathbf{t}]$, set **action**[**X**, **t**] = **reduce** $\mathbf{A} \rightarrow \boldsymbol{\omega}$.
  - If there is the special production $\mathbf{S} \rightarrow \mathbf{E} \cdot [\$]$, where **S** is the start symbol, set **action**[**X**, **t**] = **accept**.
  - If there is a transition out of **X** on symbol **t**, set **action**[**X**, **t**] = **shift**.
- Set all other actions to **error**.
- Non-LR(1) Grammar:
  - If any table entry contains two or more actions, the grammar is not LR(1).

The LR(1) Parsing Algorithm

- Begin with an empty stack and the input set to $\omega\$$, where ω is the string to parse. Set the **state** to the initial state.
- Repeat the following:
  - Let the next symbol of the input be **t**.
  - If **action**[**state**, **t**] is **shift**, then shift the input and set **state** = **goto**[**state**, **t**].
  - If **action**[**state**, t] is **reduce** $\mathbf{A} \rightarrow \omega$:
    - Pop $|\omega|$ symbols off the stack; replace them with **A**.
    - Let the state atop the stack be **top-state**.
    - Set **state** = **goto**[**top-state**, **A**]
  - If **action**[**state**, **t**] is **accept**, then the parse is done.
  - If **action**[**state**, **t**] is **error**, report an error.

The Power of LR(1)

- A language can have multiple grammars
- Every LR(0) grammar is LR(1).
- Every LL(1) grammar is LR(1).
- Every ***deterministic CFL*** has an LR(1) grammar.
  - i.e., a CFL parseable by a deterministic pushdown automaton
- Every LL(k) language is LR(1), but its grammar may not be.
- Every LR(k) language is LR(1), but its grammar may not be.

Why is LR(1) so powerful?

- Two key reasons:
- **Lookahead improves reduction decisions.**
  - A Shift/Reduce conflict is a state that looks like this: $\mathbf{A} \rightarrow \boldsymbol{\gamma} \cdot,\ \mathbf{B} \rightarrow \boldsymbol{\alpha} \cdot \boldsymbol{\beta}$
  - In LR(0), this is simply not allowed.
  - In LR(1), it uses lookahead to decide when to shift or reduce.
- **More states capture more context.**
  - LR(1) states encode additional parsing information.
  - This enables more precise decisions.

- However, LR(1) parsers are ***rarely used in practice***. **Why?**

LR(1) Automata are Huge

- LR(1) automata can be exponentially larger than LR(0) automata.
  - Replicate each state of an LR(0) automata with all $O(2^n)$possible lookaheads
  - n is the terminals in the grammar.
- LR(1) tables for practical programming languages can have hundreds of thousands or even millions of states.
- Therefore, LR(1) parsers are rarely used in practice.

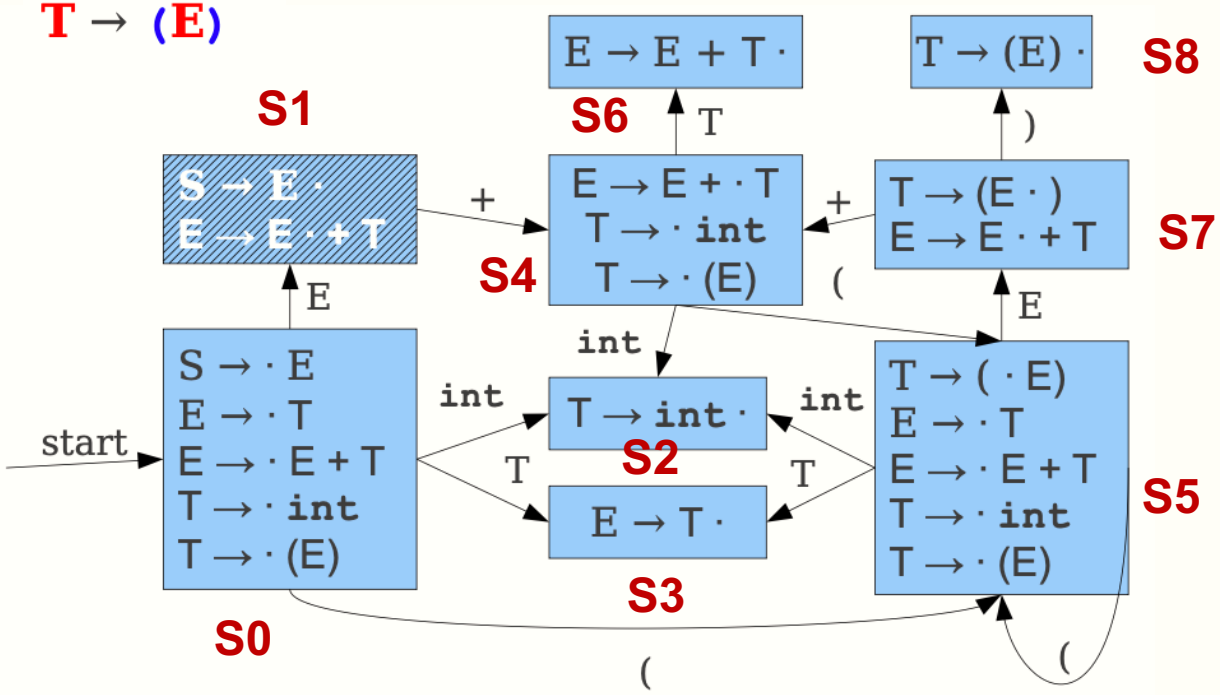
Is there a way to get the power of LR(1) without the huge table size?

SLR(1)

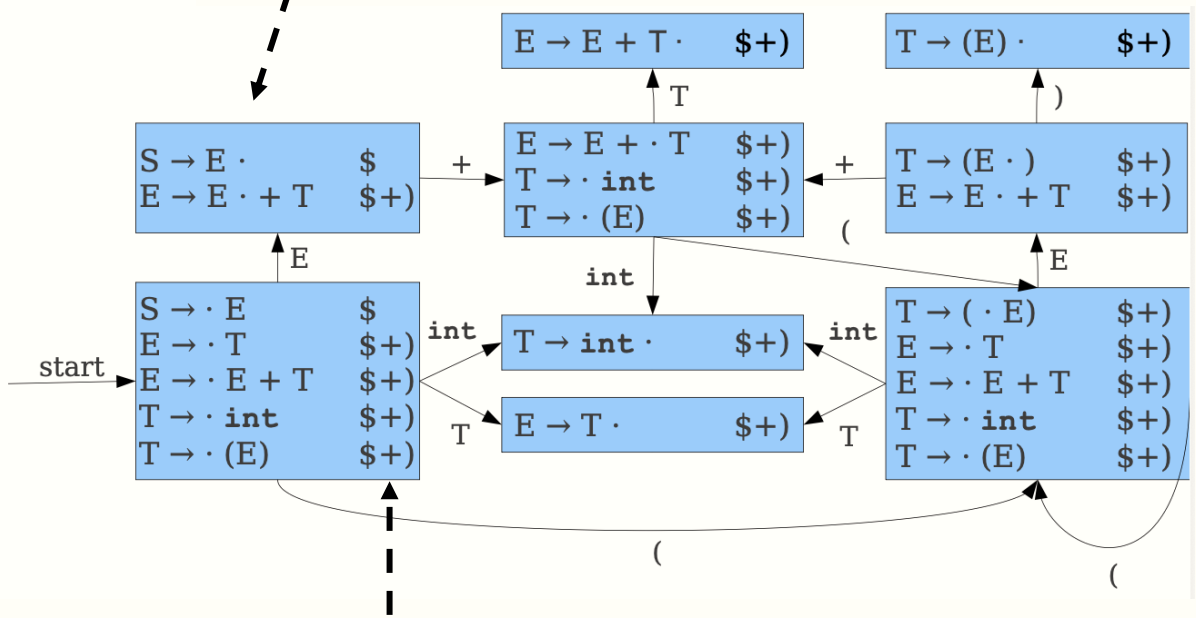
- SLR stands for Simple LR
- Idea:
  - Only reduce $\mathbf{A} \rightarrow \boldsymbol{\omega}$ if the next token **t** is in **FOLLOW**(**A**)
- SLR(1) Automaton
  - Automaton is identical to LR(0) automaton
  - The only change is that, for each production with non-terminal **A** in a state, the lookahead set is **FOLLOW**(**A**)
  - Parsing is similar to LR(1)

SLR(1) Automaton: Example

$S \rightarrow E$
$E \rightarrow T$
$E \rightarrow E + T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$



Do the **reduction** only if the lookahead token is in the **FOLLOW** set; here, it should be **$**.

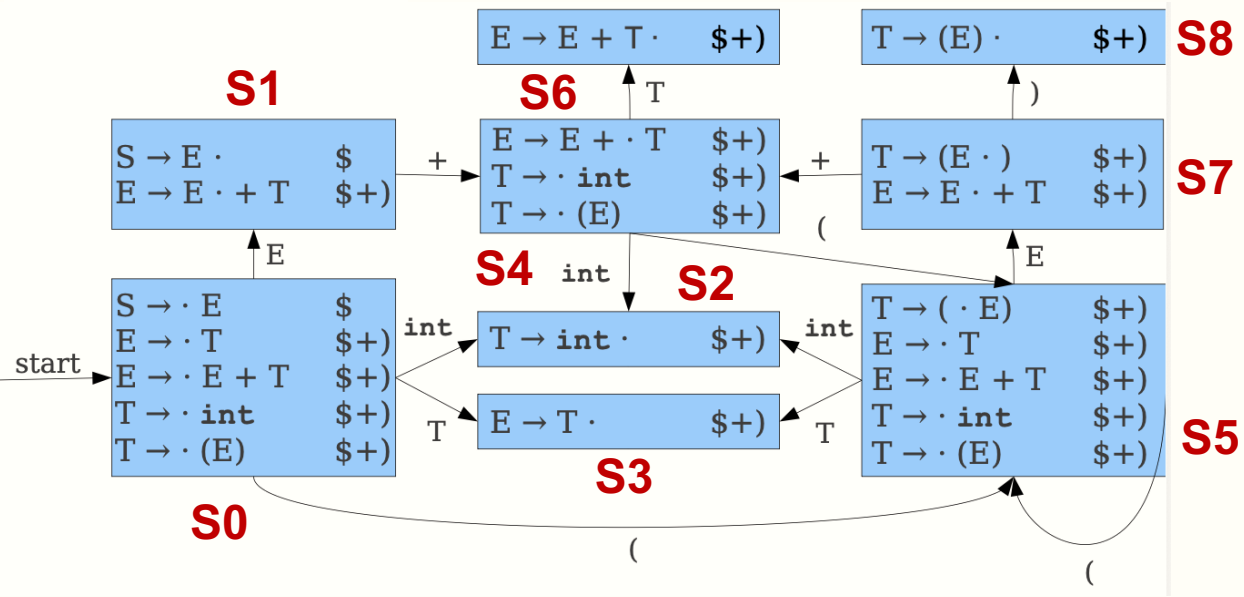


The lookahead set is the FOLLOW set of the corresponding Non-terminal.

SLR(1) Parsing: Example

$S \rightarrow E$
$E \rightarrow T$
$E \rightarrow E + T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$

SLR(1) Handle-Finding DFA



(1) Stack: S0

int	+	(	int	+	int	+	int	)	$

Shift

(2) Stack: S0 S2

int	+	(	int	+	int	+	int	)	$

Reduce

(3) Stack: S0 S3

T	+	(	int	+	int	+	int	)	$

Reduce

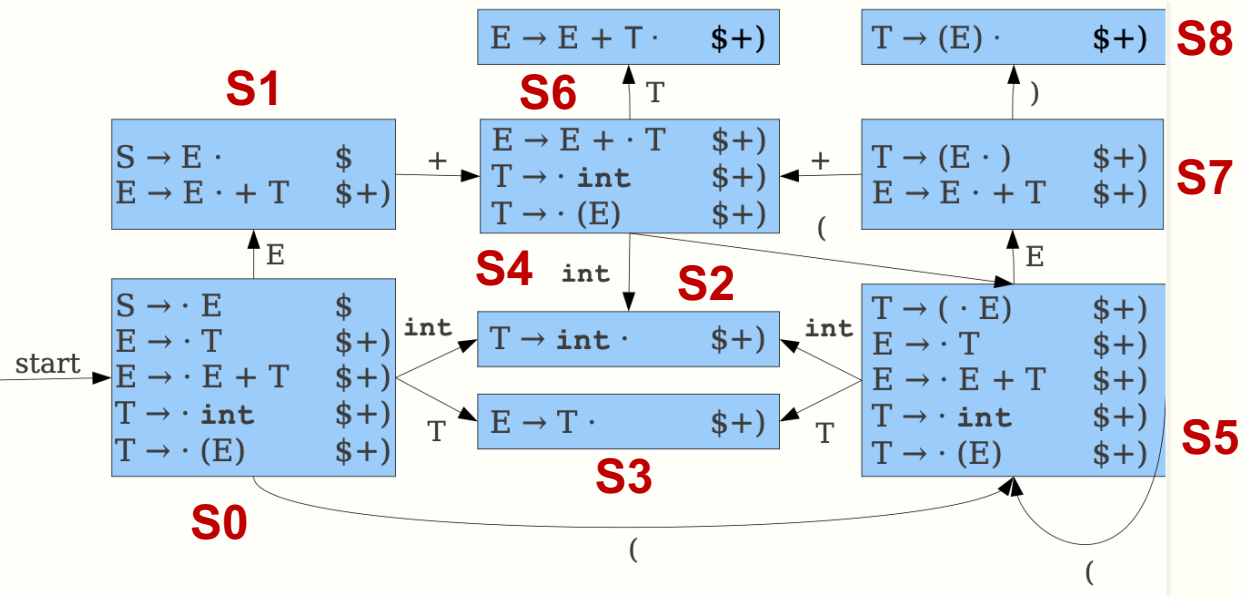
(4) Stack: S0 S1

E	+	(	int	+	int	+	int	)	$

SLR(1) Parsing: Example

$S \rightarrow E$
$E \rightarrow T$
$E \rightarrow E + T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$

SLR(1) Handle-Finding DFA



(5) Stack: S0 S1 S4

Shift

E	+	(	int	+	int	+	int	)	$

Shift

(6-7) Stack: S0 S1 S4 S5 S2

E	+	(	int	+	int	+	int	)	$

Reduce

(8) Stack: S0 S1 S4 S5 S3

E	+	(	T	+	int	+	int	)	$

Reduce

.....

More parsing steps are omitted.

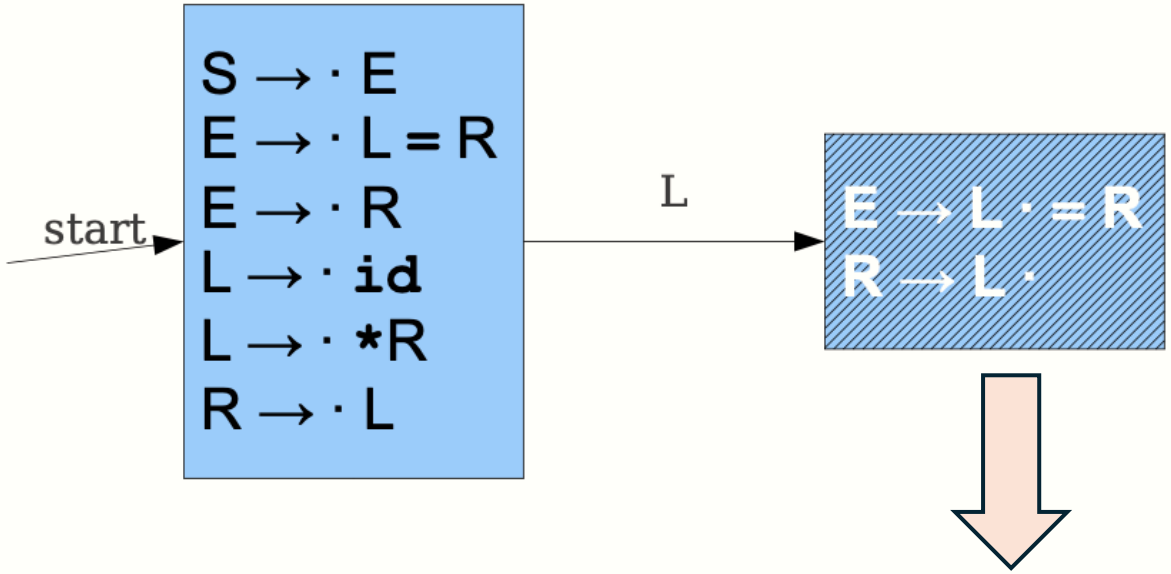
Analysis of SLR(1)

- Exploits lookahead in a small space.
  - Small automaton – same number of states as in LR(0).
  - Works on many more grammars than LR(0)
- Too weak for most grammars:
  - Loses context from not having extra states.
  - Weaker than LR(1)

- Let us use an example to illustrate SLR(1)'s limitation.

The Limits of SLR(1)

$S \to E$
$E \to L = R$
$E \to R$
$L \to id$
$L \to *R$
$R \to L$



$E \to L \cdot = R$ tells us to shift on seeing $=$
$R \to L \cdot$ tells us to reduce on FOLLOW(R).

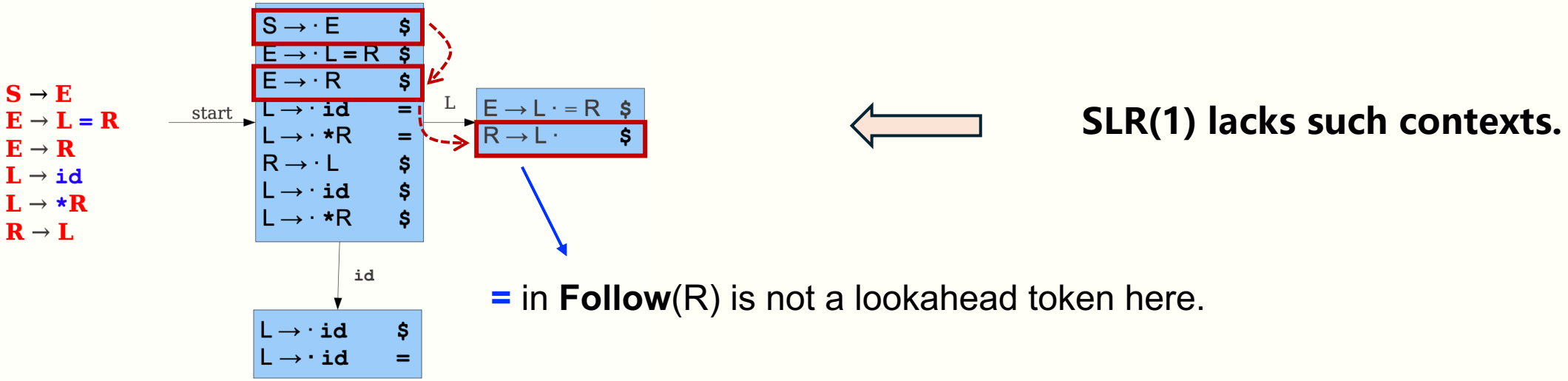
We have a conflict!

$E \to L = R$
$L \to * R$

$\Rightarrow \quad = \in \mathbf{FOLLOW}(L) \subseteq \mathbf{FOLLOW}(R)$

Why is SLR(1) Weak?

- With LR(1), incredible contextual information.
  - Lookaheads at each state only possible after applying the productions that could get us there.
- With SLR(1), *minimal* context.
  - **FOLLOW**(**A**) means *"what could follow **A** somewhere in the grammar?"* even if in a particular state, **A** couldn't possibly have that symbol after it.



LALR(1)

- SLR(1) is weak because it has no contextual information.
- LR(1) is impractical because its automaton is too big.
- **Can we preserve LR(1) context with fewer states**?
- A Surprisingly Powerful Idea:
  - Two LR(1) states share the same **core** if they differ only in lookaheads
  - In an LR(1) automaton, we have multiple states with the same core but different lookaheads.
  - What if we merge all these states?
  - This is called **LALR(1)**
    - **Lookahead(1) LR(0)**

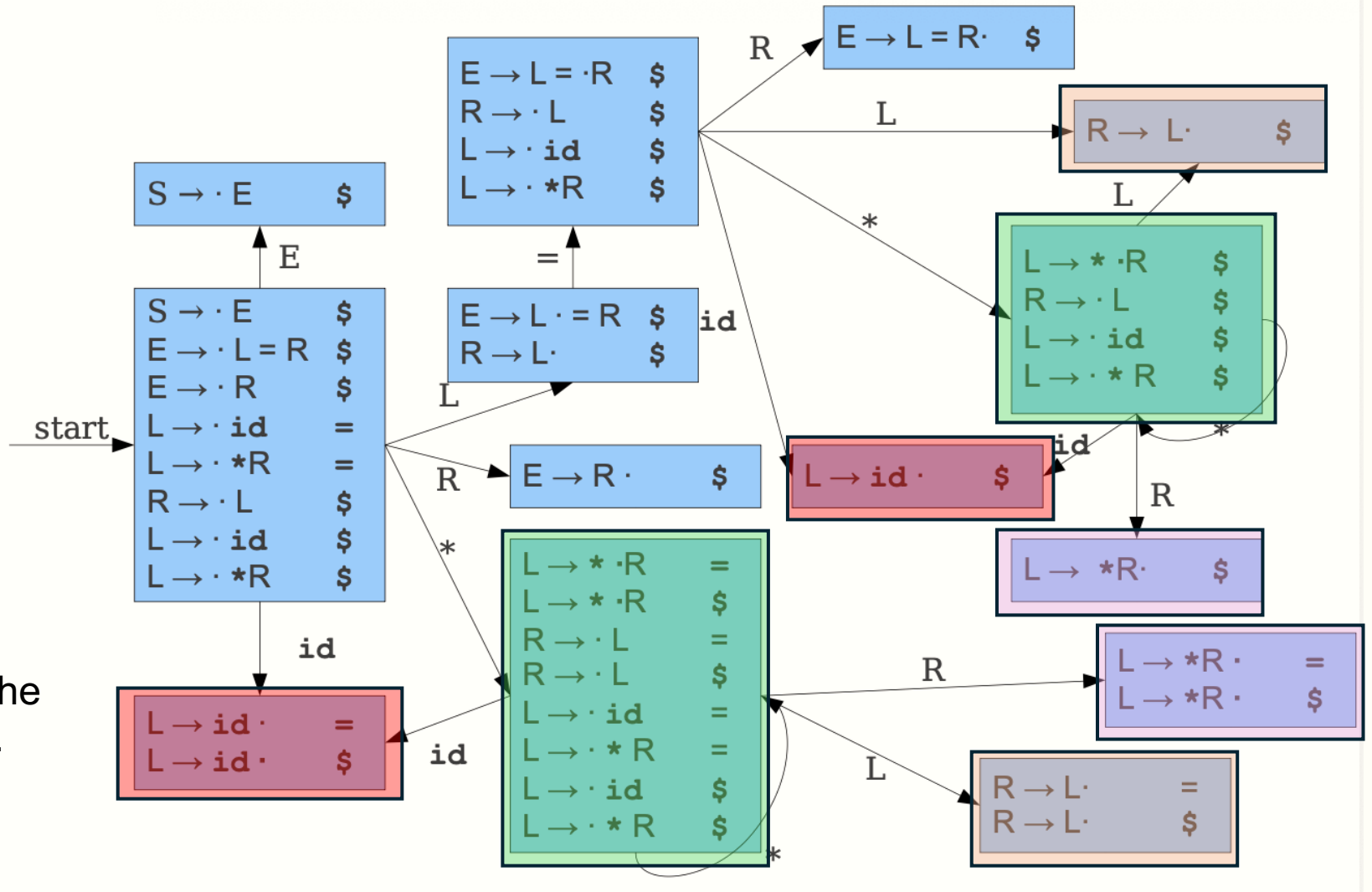
T → (·E)	$
E → ·E + T	)
E → ·T	)
T → ·int	)
T → ·(E)	)

T → (·E)	)
E → ·E + T	)
E → ·T	)
T → ·int	)
T → ·(E)	)

From LR(1) to LALR(1)

S → E
E → L = R
E → R
L → id
L → *R
R → L

LR(1) automaton

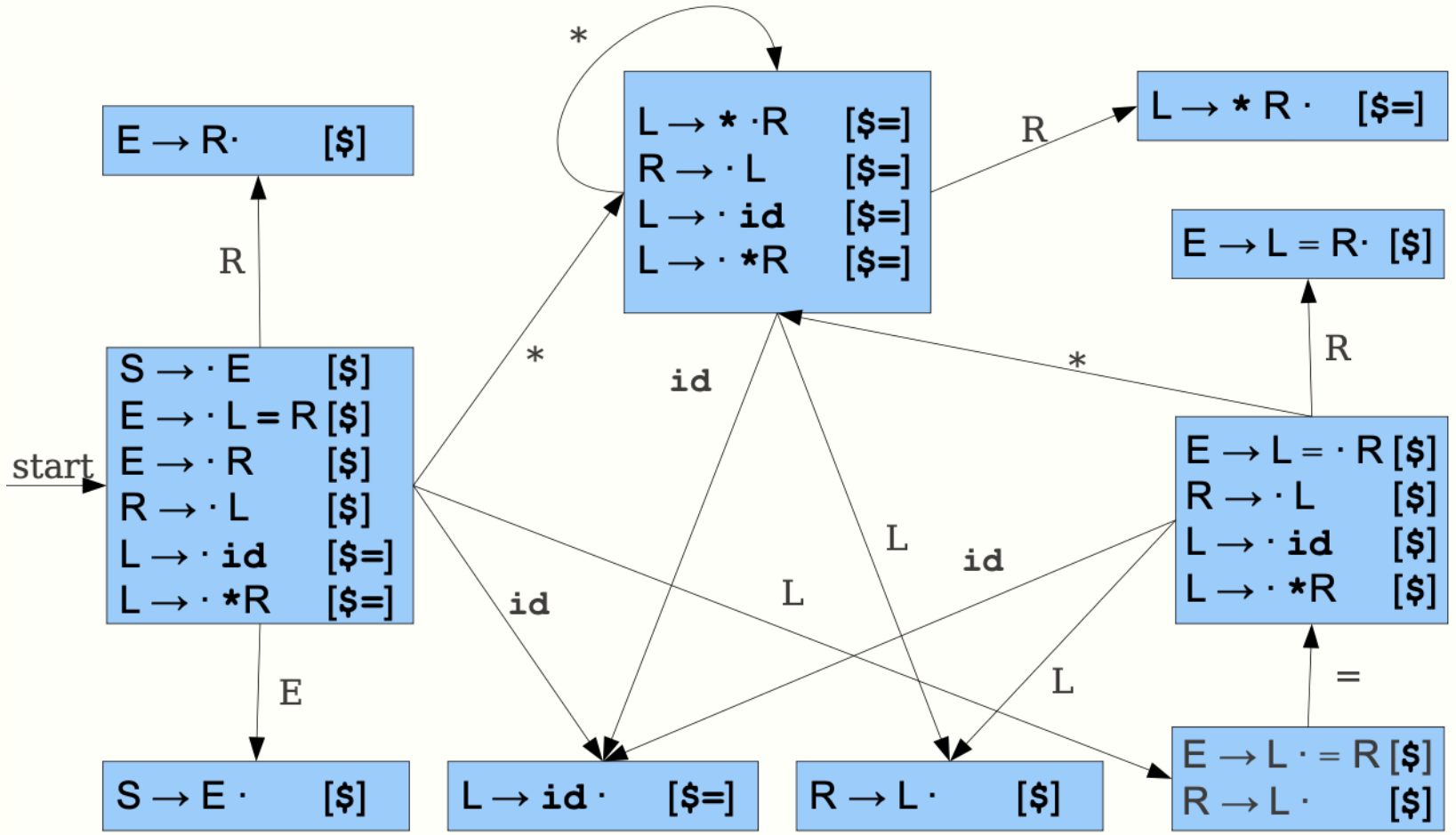


Merge the states with the same **Core (Color)**.

From LR(1) to LALR(1)

$S \rightarrow E$
$E \rightarrow L = R$
$E \rightarrow R$
$L \rightarrow id$
$L \rightarrow *R$
$R \rightarrow L$

LALR(1) automaton



Advantages of LALR(1)

- Maintains context.
  - Lookup sets are based on the fine-grained LR(1) automaton.
  - Each state's lookup is relevant only for that state.
- Keeps the automaton small.
  - Resulting automaton has the same size as the LR(0) automaton.

- LALR(1) is **Powerful**
  - Every LR(0) grammar is LALR(1).
  - Every SLR(1) grammar is LALR(1)
  - Most (but not all) LR(1) grammars are LALR(1).

LALR(1) isn't LR(1)

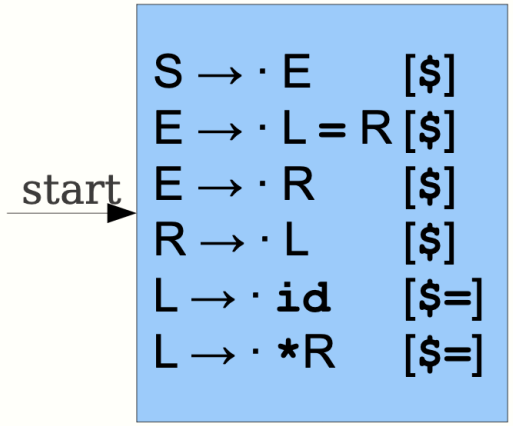
- Merging LR(1) states **cannot** introduce a **shift/reduce conflict**. **Why**?
  - Since the items have the same core, a shift/reduce conflict in an LALR(1) state would have to also exist in one of the LR(1) states it was merged from.
- Merging LR(1) states **can** introduce a **reduce/reduce conflict**.
  - Often these conflicts appear without any good reason
  - This is one limitation of LALR(1).

Constructing LALR(1) Automata

- Building LALR(1) from LR(1) is not good. **Why**?
  - LR(1) automata are impractically large.
- Can we construct LALR(1) automata more efficiently?
  - Yes, we will show two.
- The "**Lazy Merging**" Technique
  - **Idea**: Merge LR(1) states during construction; Start with the initial state in a worklist; Process states one by one; If a new state shares a core with an existing state, merge them and reprocess the updated state.

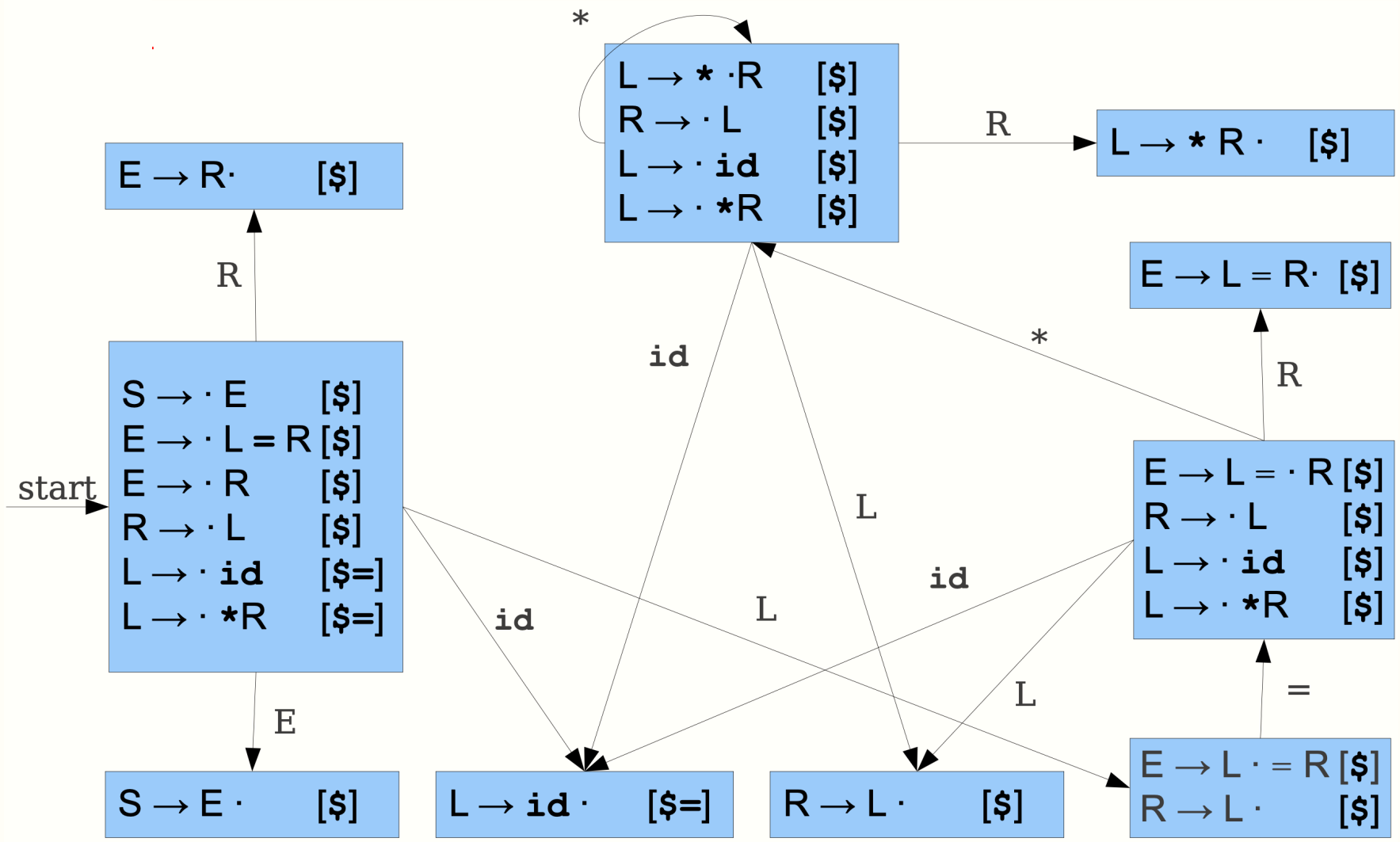
LALR(1) Construction: “Lazy Merging” Technique

$S \rightarrow E$
$E \rightarrow L = R$
$E \rightarrow R$
$L \rightarrow \texttt{id}$
$L \rightarrow *R$
$R \rightarrow L$



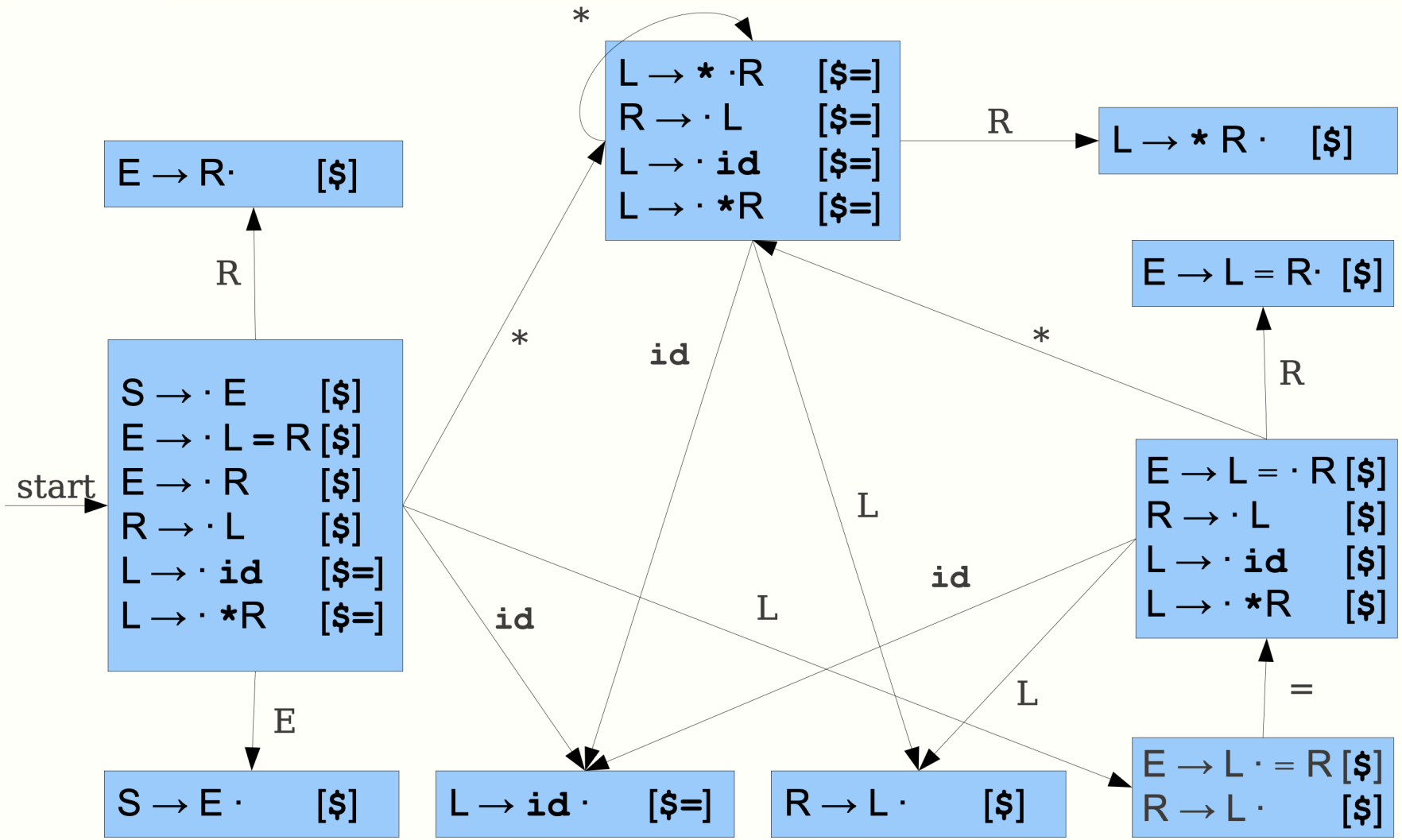
LALR(1) Construction: “Lazy Merging” Technique

$S \rightarrow E$
$E \rightarrow L = R$
$E \rightarrow R$
$L \rightarrow id$
$L \rightarrow *R$
$R \rightarrow L$



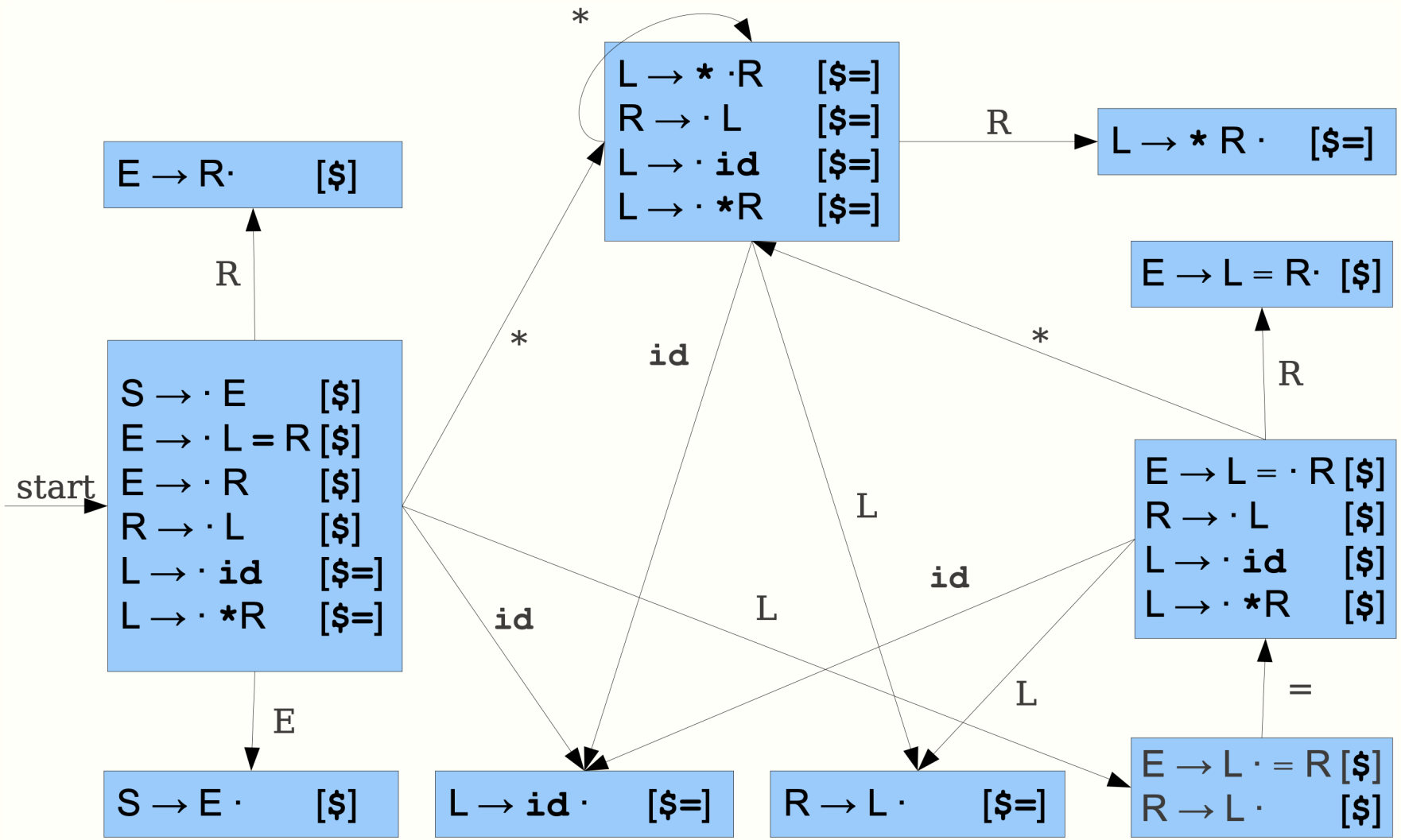
LALR(1) Construction: “Lazy Merging” Technique

$S \rightarrow E$
$E \rightarrow L = R$
$E \rightarrow R$
$L \rightarrow id$
$L \rightarrow *R$
$R \rightarrow L$



LALR(1) Construction: “Lazy Merging” Technique

$S \rightarrow E$
$E \rightarrow L = R$
$E \rightarrow R$
$L \rightarrow id$
$L \rightarrow *R$
$R \rightarrow L$



Analysis of the “Lazy Merging” Algorithm

- Since we merge as we go, the size of the partial automaton never exceeds the size of the overall automaton.
- However, this algorithm could be **very slow** in practice.
  - We might still have to generate all the LR(1) states, even if they immediately get merged.
  - This can be very slow.
- Is there any better method?

SLR versus LALR

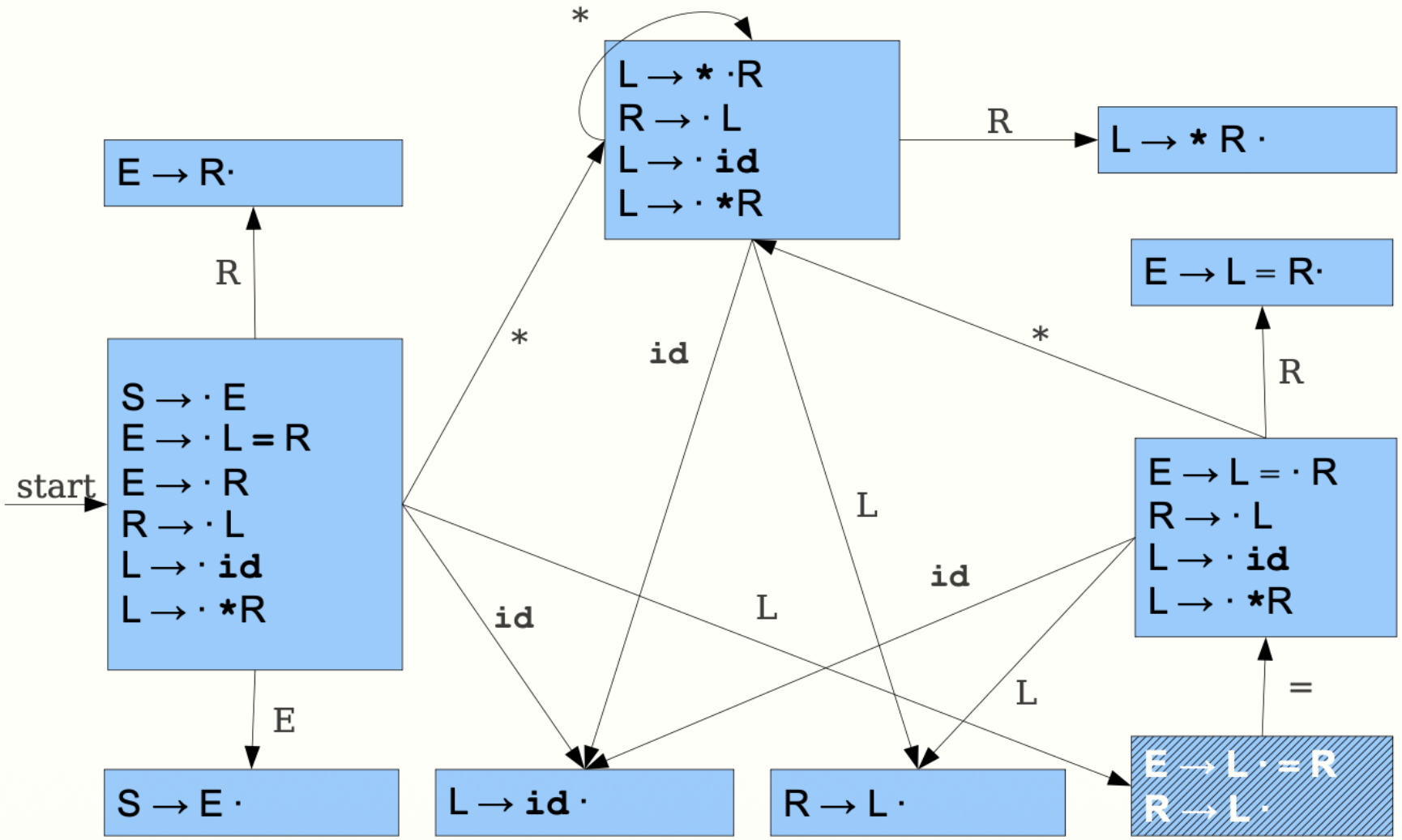
- SLR uses **FOLLOW** sets
  - $\mathbf{FOLLOW(A)} = \{\mathbf{t} | \mathbf{S} \Rightarrow * \boldsymbol{\alpha}\mathbf{A}\mathbf{t}\boldsymbol{\omega}\}$ is the set of terminals that can follow **A** in a derivation
  - SLR is LR(0), with reductions augmented using **FOLLOW** sets.
- LALR uses **LA** Sets
  - Given an LR(0) state q and a production $\mathbf{A} \rightarrow \boldsymbol{\gamma}$, the **lookahead set** $\mathbf{LA}(q, \mathbf{A} \rightarrow \boldsymbol{\gamma}) = \{\mathbf{t} | \mathbf{S} \Rightarrow * \boldsymbol{\alpha}\mathbf{A}\mathbf{t}\boldsymbol{\omega}\}$ and $\boldsymbol{\alpha}\boldsymbol{\gamma}$ reaches q }.
    - $\boldsymbol{\alpha}\boldsymbol{\gamma}$ reaches q: run the LR(0) automaton on $\boldsymbol{\alpha}\boldsymbol{\gamma}$, reaches state q
  - Intuitively, if we're in some state q and are going to reduce $\mathbf{A}$ to $\boldsymbol{\gamma}$, $\mathbf{LA}(q, \mathbf{A} \rightarrow \boldsymbol{\gamma})$ is the set of terminals that could actually follow **A** at this point, given that we're reducing $\mathbf{A} \rightarrow \boldsymbol{\gamma}$.
  - Much more precise than **FOLLOW** sets, $\mathbf{LA}(q, \mathbf{A} \rightarrow \boldsymbol{\gamma}) \subseteq \mathbf{FOLLOW}(\mathbf{A})$.
- If we can compute **LA** from **FOLLOW**, we can construct an LALR(1) parser.

LALR(1) Construction: “Augmented Grammar”

- One of the most beautiful constructions of the quarter...
  - Fast and simple construction of LALR(1) lookaheads
- Key Steps:
  - Construct the LR(0) automaton for the grammar.
  - Construct the augmented grammar by replacing nonterminals with new nonterminals based on the LR(0) transitions.
  - Compute the **FOLLOW** sets for these new nonterminals.
  - Propagate changes through the LR(0) automaton.
- **Theorem** (Bermudez and Logothetis):
  - This correctly computes LALR(1) lookaheads.

Step 1: Compute LR(0) Automaton

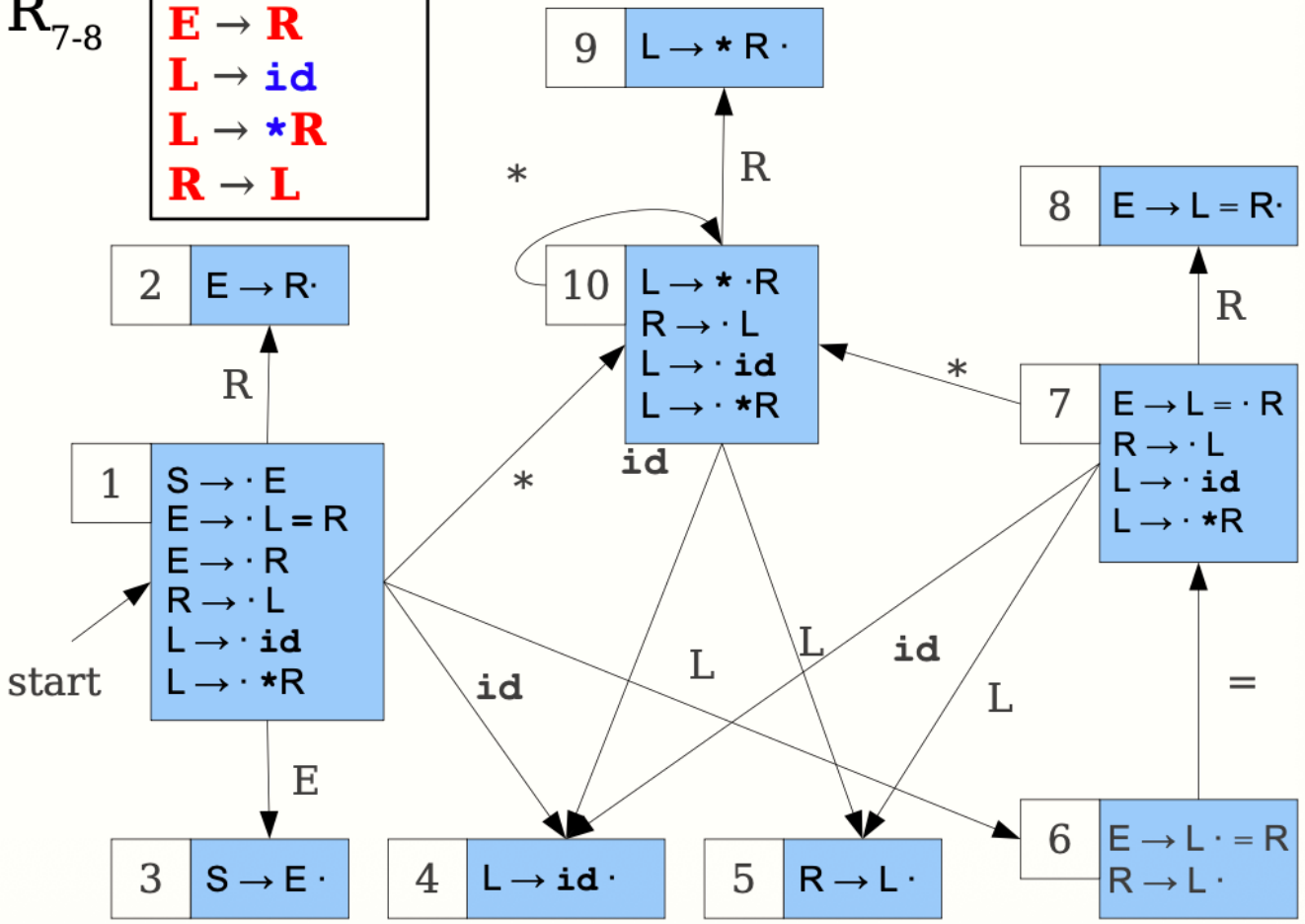
$S \rightarrow E$
$E \rightarrow L = R$
$E \rightarrow R$
$L \rightarrow id$
$L \rightarrow *R$
$R \rightarrow L$



Step 2: Construct the Augmented Grammars

$S_1 \rightarrow E_{1\text{-}3}$
$E_{1\text{-}3} \rightarrow L_{1\text{-}6} = R_{7\text{-}8}$
$E_{1\text{-}3} \rightarrow R_{1\text{-}2}$
$R_{1\text{-}2} \rightarrow L_{1\text{-}6}$
$L_{1\text{-}6} \rightarrow \mathbf{id}$
$L_{1\text{-}6} \rightarrow * R_{10\text{-}9}$
$R_{7\text{-}8} \rightarrow L_{7\text{-}5}$
$L_{7\text{-}5} \rightarrow \mathbf{id}$
$L_{7\text{-}5} \rightarrow *R_{10\text{-}9}$
$R_{10\text{-}9} \rightarrow L_{10\text{-}5}$
$L_{10\text{-}5} \rightarrow \mathbf{id}$
$L_{10\text{-}5} \rightarrow *R_{10\text{-}9}$

S → E
E → L = R
E → R
L → id
L → *R
R → L



Step 2: Construct the Augmented Grammars

- For each item $\mathbf{A} \rightarrow \cdot\ \boldsymbol{\omega}$ in some state q:
  - Trace out the path $\boldsymbol{\omega}$ takes through the LR(0) automaton starting at q.
  - Replace each nonterminal in $\boldsymbol{\omega}$ with a nonterminal annotated with the state transitioned to and from by the edge labeled with that nonterminal.
  - Replace **A** with a nonterminal annotated with the start and end state of the transition on **A** out of q.
- Result is a larger grammar with more precise productions.

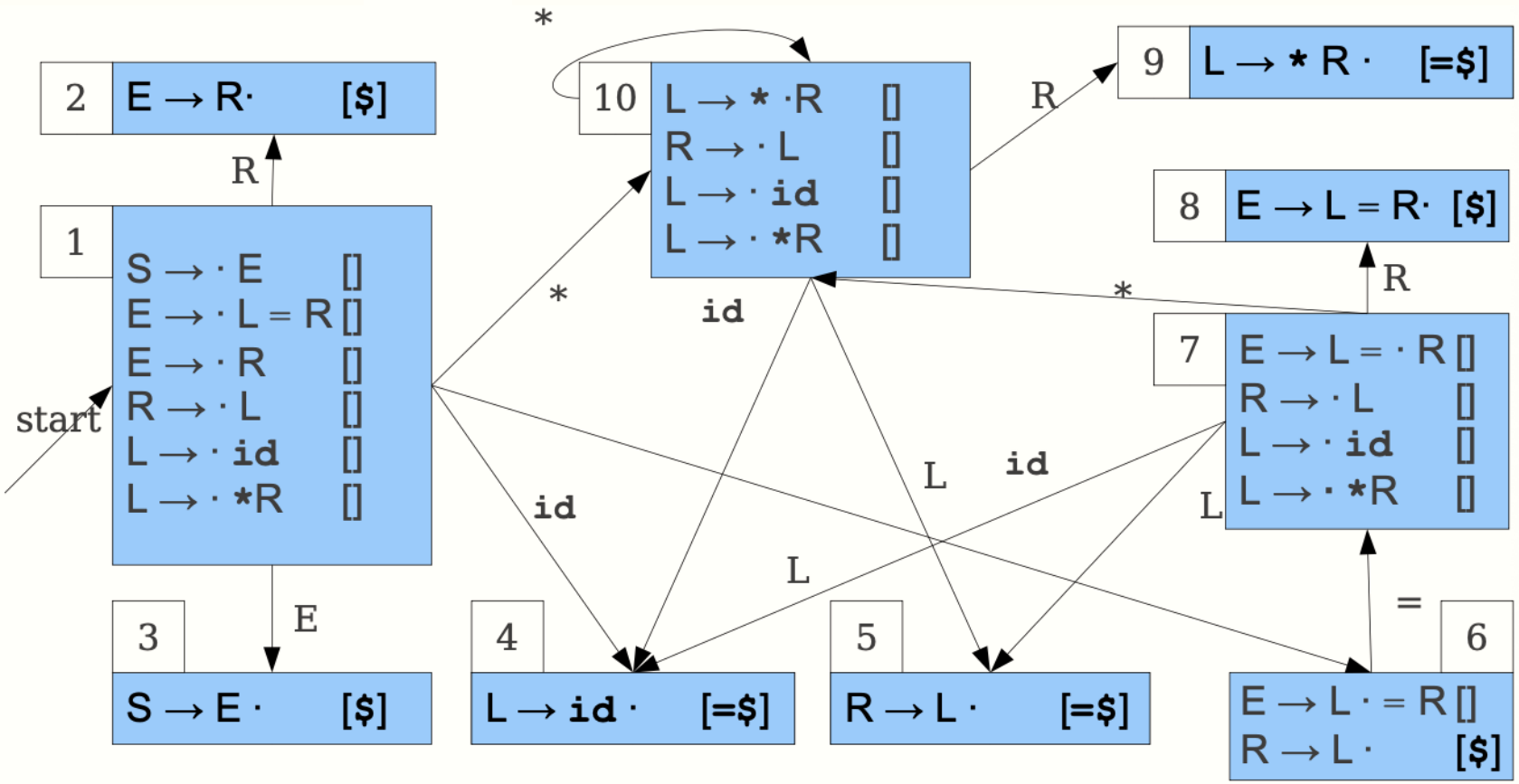
- **Why is this augmented grammars useful**?
  - At a high-level, separates nonterminals by their context.
  - This makes the **FOLLOW** sets more precise for their nonterminals.
  - In fact, the **FOLLOW** sets are surprisingly precise.

Step 3: Compute the Augmented FOLLOW Sets

$S_1 \rightarrow E_{1\text{-}3}$
$E_{1\text{-}3} \rightarrow L_{1\text{-}6} = R_{7\text{-}8}$
$E_{1\text{-}3} \rightarrow R_{1\text{-}2}$
$R_{1\text{-}2} \rightarrow L_{1\text{-}6}$
$L_{1\text{-}6} \rightarrow \mathbf{id}$
$L_{1\text{-}6} \rightarrow * R_{10\text{-}9}$
$R_{7\text{-}8} \rightarrow L_{7\text{-}5}$
$L_{7\text{-}5} \rightarrow \mathbf{id}$
$L_{7\text{-}5} \rightarrow *R_{10\text{-}9}$
$R_{10\text{-}9} \rightarrow L_{10\text{-}5}$
$L_{10\text{-}5} \rightarrow \mathbf{id}$
$L_{10\text{-}5} \rightarrow *R_{10\text{-}9}$

S_1	$E_{1\text{-}3}$	$L_{1\text{-}6}$	$L_{7\text{-}5}$	$L_{10\text{-}5}$	$R_{1\text{-}2}$	$R_{7\text{-}8}$	$R_{10\text{-}9}$
$	$	= $	$	= $	$	$	= $

Step 4: Construct the LALR(1) automaton using FOLLOW sets



S_1	E_{1-3}	L_{1-6}	L_{7-5}	L_{10-5}	R_{1-2}	R_{7-8}	R_{10-9}
$	$	= $	$	= $	$	$	= $

Step 4: Construct the LALR(1) automaton using FOLLOW sets

- For each item $\mathbf{A} \rightarrow \cdot\, \boldsymbol{\omega}$ in a state q:
  - Let $\mathbf{A}_{\mathrm{q}\rightarrow\mathrm{r}}$ be the nonterminal corresponding to $\mathbf{A}$ following the transition out of q into some state r.
  - Trace through the automaton along the path labeled by $\boldsymbol{\omega}$.
    - This will lead to a state containing an item $\mathbf{A} \rightarrow \boldsymbol{\gamma}\, \cdot$.
- Add to the lookahead of $\mathbf{A} \rightarrow \boldsymbol{\gamma} \cdot$ the contents of $\mathbf{FOLLOW}\left(\mathbf{A}_{\mathrm{q}\rightarrow\mathrm{r}}\right)$

Summary of LALR(1)

- Along with LL(k), one of the most popular parsing algorithms in use today.
- Produced by the bison/Menhir/YACC parser generator.
  - Rarely generated by hand.
- Can handle most, but not all, LR(1) languages.

Two Practical Concerns

❑Ambiguity

- Real grammars for programming languages are often ambiguous.
- Programmers are terrible at eliminating it.
- The predictive parsers we have seen so far (LL(1), LR(0), SLR(1), LALR(1), LR(1)) only work on unambiguous grammars.
  - Intuitively: if grammar is ambiguous, cannot uniquely guess which production/reduction to use.
- How do you build a parser to try to combat it?

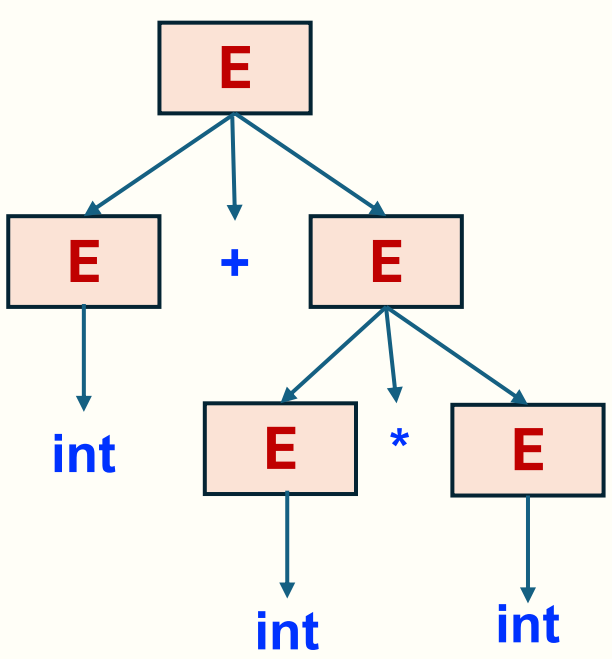
❑Error-handling

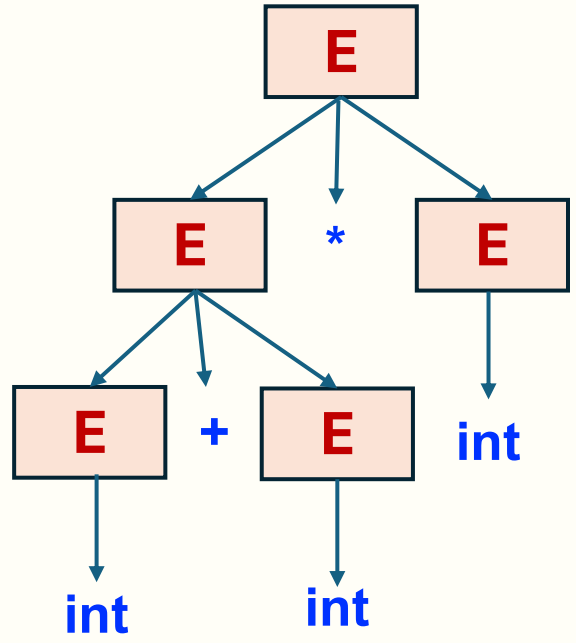
- How do you report errors intelligently?
- How do you continue parsing after an error?

Parsing Ambiguous Grammars

- Consider this simple grammar for arithmetic expressions:
  - It is ambiguous, e.g., has two parse trees for `int + int * int`

$$S \rightarrow E$$
$$E \rightarrow E + E$$
$$E \rightarrow E * E$$
$$E \rightarrow \texttt{int}$$
$$E \rightarrow (E)$$





- **What happens if we try parsing it?**

SLR(1) Parsing with Ambiguity

1
S → · E E → · E + E E → · E * E E → · int E → · (E)

1. **S → E**
2. **E → E + E**
3. **E → E * E**
4. **E → (E)**
5. **E → int**

2
S → E · E → E ·+ E E → E ·* E

3
E → E + ·E E → · E + E E → · E * E E → · int E → · (E)

4
E → E * ·E E → · E + E E → · E * E E → · int E → · (E)

5
E → E + E · E → E · + E E → E · * E

6
E → E * E · E → E · + E E → E · * E

7
E → (·E) E → · E + E E → · E * E E → · int E → · (E)

8
E → (E ·) E → E ·+ E E → E ·* E

9
E → (E) ·

10
E → int ·

FOLLOW(**S**) = { **$** }

FOLLOW(**E**) = { **+**, *****, **)** , **$** }

	int	+	*	(	)	$	E
1	s10			s7			s2
2		s3	s4			acc	
3	s10			s7			s5
4	s10			s7			s6
5		s3 r2	s4 r2		r2	r2	
6		s3 r3	s4 r3		r3	r3	
7	s10			s7			s8
8		s3	s4		s9		
9		r4	r4		r4	r4	
10		r5	r5		r5	r5	

Resolving Ambiguity

- Although the grammar is ambiguous, there is clearly one intended parse tree because of operator precedence.
- How can we use this precedence information to avoid LR conflicts?
  - Tell the parser generator about the **associativity** and **precedence** of certain rules.
- Operators can be left-associative, right- associative, or non-associative.
- Productions can have their priorities ranked against one another.
- When choosing whether to reduce on **t** or shift the terminal **r**:
  - If **t** has higher priority, **reduce**; or If **r** has higher priority, **shift**.
  - If **t** and **r** have the same priority:
    - If **t** is left-associative, **reduce; i**f **t** is right-associative, **shift**; if **t** is non-associative, **error**.

Resolving Ambiguity in SLR(1) Parsing

1. **S → E**
2. **E → E + E** (Left-assoc, pri. 0)
3. **E → E * E** (Left-assoc, pri. 1)
4. **E → (E)**
5. **E → int**

1

S → · E
E → · E + E
E → · E * E
E → · int
E → · (E)

2

S → E ·
E → E ·+ E
E → E ·* E

3

E → E + ·E
E → · E + E
E → · E * E
E → · int
E → · (E)

4

E → E * ·E
E → · E + E
E → · E * E
E → · int
E → · (E)

5

E → E + E ·
E → E · + E
E → E · * E

6

E → E * E ·
E → E · + E
E → E · * E

7

E → (·E)
E → · E + E
E → · E * E
E → · int
E → · (E)

8

E → (E ·)
E → E ·+ E
E → E ·* E

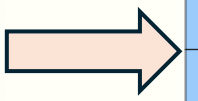
9

E → (E) ·

10

E → int ·

	int	+	*	(	)	$	E
1	s10			s7			s2
2		s3	s4			acc	
3	s10			s7			s5
4	s10			s7			s6
5		s3 r2	s4 r2		r2	r2	
6		s3 r3	s4 r3		r3	r3	
7	s10			s7			s8
8		s3	s4		s9		
9		r4	r4		r4	r4	
10		r5	r5		r5	r5	



	int	+	*	(	)	$	E
1	s10			s7			s2
2		s3	s4			acc	
3	s10			s7			s5
4	s10			s7			s6
5		r2	s4		r2	r2	
6		r3	r3		r3	r3	
7	s10			s7			s8
8		s3	s4		s9		
9		r4	r4		r4	r4	
10		r5	r5		r5	r5	

Resolving Ambiguity in SLR(1) Parsing II

1. **S** → **E**
2. **E** → **E + E** (Rgt.-assoc, pri. 0)
3. **E** → **E * E** (Rgt.-assoc, pri. 1)
4. **E** → **(E)**
5. **E** → **int**

1

S → · E
E → · E + E
E → · E * E
E → · int
E → · (E)

2

S → E ·
E → E ·+ E
E → E ·* E

3

E → E + ·E
E → · E + E
E → · E * E
E → · int
E → · (E)

4

E → E * ·E
E → · E + E
E → · E * E
E → · int
E → · (E)

5

E → E + E ·
E → E · + E
E → E · * E

6

E → E * E ·
E → E · + E
E → E · * E

7

E → (·E)
E → · E + E
E → · E * E
E → · int
E → · (E)

8

E → (E ·)
E → E ·+ E
E → E ·* E

9

E → (E) ·

10

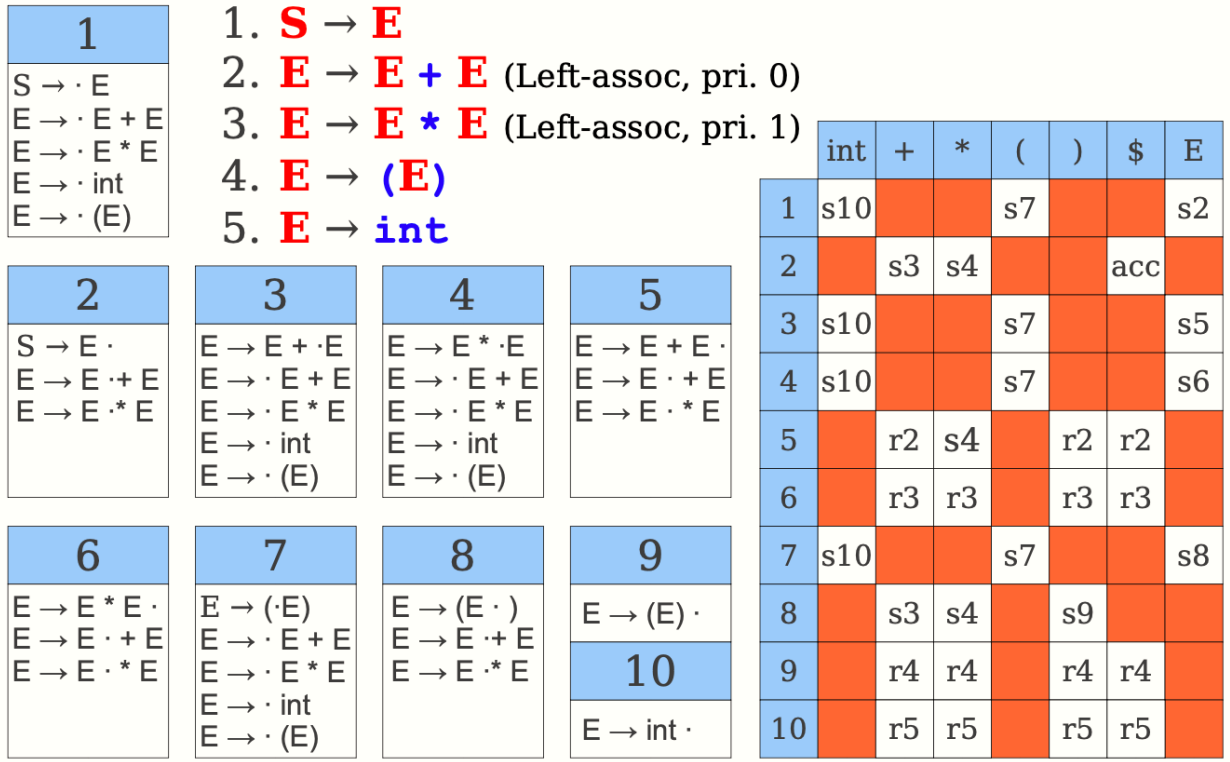
E → int ·

	int	+	*	(	)	$	E
1	s10			s7			s2
2		s3	s4			acc	
3	s10			s7			s5
4	s10			s7			s6
5		s3 r2	s4 r2		r2	r2	
6		r3	s4 r3		r3	r3	
7	s10			s7			s8
8		s3	s4		s9		
9		r4	r4		r4	r4	
10		r5	r5		r5	r5	

	int	+	*	(	)	$	E
1	s10			s7			s2
2		s3	s4			acc	
3	s10			s7			s5
4	s10			s7			s6
5		s3	s4		r2	r2	
6		r3	s4		r3	r3	
7	s10			s7			s8
8		s3	s4		s9		
9		r4	r4		r4	r4	
10		r5	r5		r5	r5	

Error Handling

- What should the parser do when it encounters an error? (**red entries**)
- Could just say "**syntax error**," but we'd like more detailed messages.
- How do we **resume parsing** after an error?



	int	+	*	(	)	$	E
1	s10			s7			s2
2		s3	s4			acc	
3	s10			s7			s5
4	s10			s7			s6
5		r2	s4		r2	r2	
6		r3	r3		r3	r3	
7	s10			s7			s8
8		s3	s4		s9		
9		r4	r4		r4	r4	
10		r5	r5		r5	r5	

Error Handling

❑**Idea** 1: add error productions to recognize common mistakes.

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow \texttt{int}$

$E \rightarrow (E)$

$E \rightarrow E\ E$ *(error production)* ⟶ Missing operator

$E \rightarrow E\ +$ *(error production)* ⟶ Missing operand

$E \rightarrow E\ *$ *(error production)* ⟶ Missing operand

- Useful for diagnosing common programmer mistakes.
  - For example, using **implements** instead of **extends** in Java.
- Increases the risk of parsing problems.
  - More likelihood for ambiguity.
  - More likelihood grammar won't be accepted by parser generator (i.e. not LALR(1))
- Forces the parser generator to anticipate errors.

Error Handling

- Idea 2: Panic-Mode Error Recovery
  - Skip invalid input and resume parsing at a synchronising token.
  - **Error productions**: add rules containing the special error token.

$$E \rightarrow E + E$$
$$E \rightarrow E * E$$
$$E \rightarrow (E)$$
$$E \rightarrow \texttt{int}$$

$$E \rightarrow \underline{\text{error}}\ \texttt{int}$$
$$E \rightarrow (\underline{\text{error}})$$

  - Tokens after errors are called **synchronising tokens**.
- Recovery Steps:
  - Pop stack entries until the parser can shift **error**
  - Discard input tokens until a synchronising token is found
  - Resume normal parsing

练习1: 使用LR(0)解析“(x, x)”

❑LR(0)文法如下：

```
S' → S eof
S → ( L )
S → x
L → S
L → L, S
```

- 请画出其LR(0) 自动机（DFA）
- 构建其动作/转移（action/goto)表
- 根据动作/转移表解析“(x, x)”， 构建出parse tree
- 思考，如何在构建解析树的过程中把AST也构建出来?

练习2: LR(1)

- 给定如下LR(1)文法G：

$$S \to Aa \mid bAc \mid Bc \mid bBa$$
$$A \to d$$
$$B \to d$$

- 构造出其LR(1)自动机
- 构建其动作/转移（action/goto）表
- 根据动作/转移表，解析“bda”，构建解析树

练习3: LALR(1)

- 给定如下文法G:

$$S \to Aa \mid bAc \mid Bc \mid bBa$$
$$A \to d$$
$$B \to d$$

- 用状态合并（Lazy technique）和增广文法（Argumented Grammar）两种方式，构建其LALR(1)自动机
- 该文法是LALR(1)文法吗?
- 若是，请用构建好的自动机解析“bda”， 并构建解析树